Operation Mapping

Chia-Chi Tsai (蔡家齊)
cctsai@gs.ncku.edu.tw
AI System Lab
Department of Electrical Engineering
National Cheng Kung University

Outline



- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

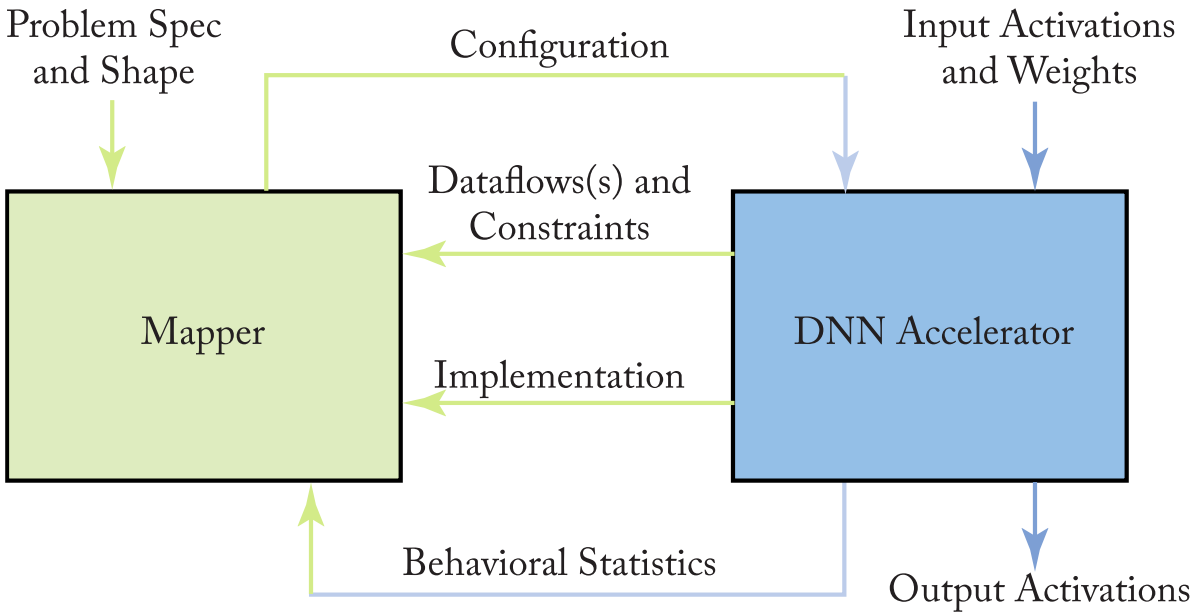
Outline



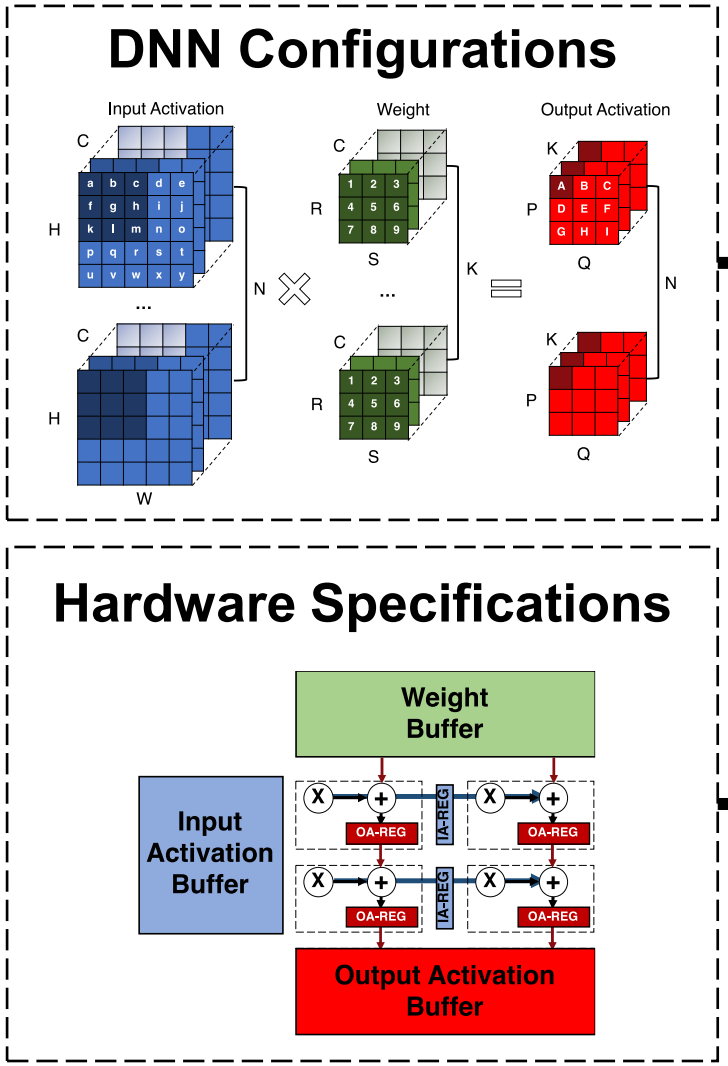
- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

Mapping

- Compiler-like process
  - **Map** the **computation** for a particular workload layer shape **onto a specific DNN accelerator** design
  - Optimize behavior with respect to **energy efficiency** and **performance**



The DNN Mapping Problem



Mapping (Blocking/Tiling/ Compiling)

Execution Order (on the target HW)

Key Steps

- **Determine the dataflow**
  - Targeted DNN accelerator may supports more than one dataflow
  - Major impact on the behavior of an accelerator
  - Define **loop order**
- **Determine the data tile sizes**
  - Tiling is in **both space and time**
  - Define **Loop bounds**
- **Bind the operations to the hardware**
  - **Network-on-Chip**
  - Physical attribute of the hardware can be taken into account
    - Physical proximity of specific PEs/buffers to one another

Map Space

- The set of **all possible mappings** **for a specific workload** shape on to a specific DNN accelerator
- Given the number of degrees of freedom in mapping, **map spaces can be very large**
- In AI processors
  - **Many attributes** of an accelerator's design **are fixed**
    - Number of PEs
    - Number and sizes of buffers
    - Network-on-Chip
  - **Complex constraints** on which mappings are legal
  - Map spaces tend to be **irregular**
- Different mappings
  - Result in different performance and energy efficiency on hardware

Mapper

- Find the **optimal mapping** in the map space
- Express and search the map space
- Evaluate and quantify
  - **Energy efficiency**
  - **Performance**

Outline

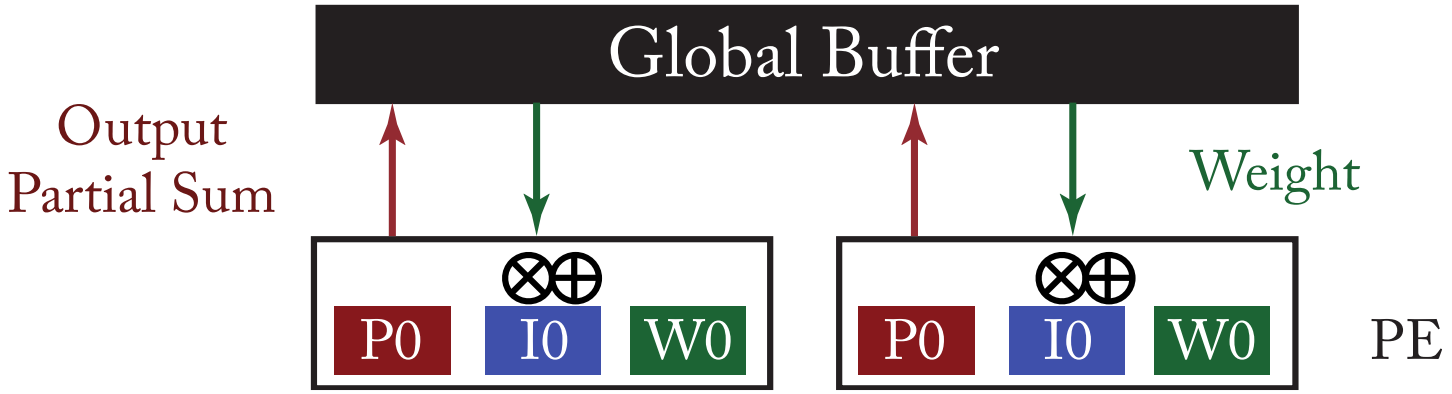
- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

Mapping Workload from Loop Nest



- Picking a supported dataflow
- Picking both the spatial and temporal tiling
  - For operands to the storage
  - For computational components
- **Creating a mapping from the perspective of selecting and parameterizing loop nests.**

Example 101 - Hardware Architecture



- Single input-channel
- Multiple output channels
- Each PE buffer for
  - One input feature map
  - One output feature map
  - One filter weights
- Pre-load input feature map
- Can be configured as output-stationary or weight-stationary

Example 101 - Loop Nest

- W, M, S, Q: workload
  - Define shape of workload
- M2, M1, PE_dataflow
  - Control the behavior of loop nest
- Control the placement and scheduling of activity in both space and time
- Mapping constraints
  - $M \leq M2 \times M1$
  - $M1 \leq 2$

```
i = Array(W) # Input feature map
f = Array(M, S) # Filter weights
o = Array(M, Q) # Output feature map

# Mapping

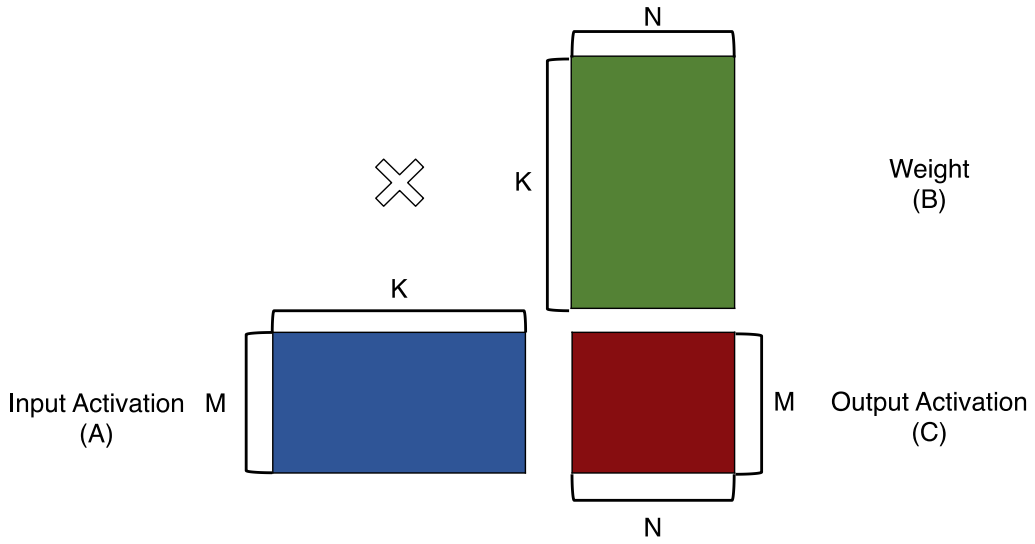
M2 = mapping["M2"]
M1 = mapping["M1"]
PE_dataflow = mapping["PE_dataflow"]

# Level 2 - Global Buffer
for m2 in [0, M2):

  # Level 1 - PE array
  spatial_for m1 in [0, M1):
    m = m2 * M1 + m1

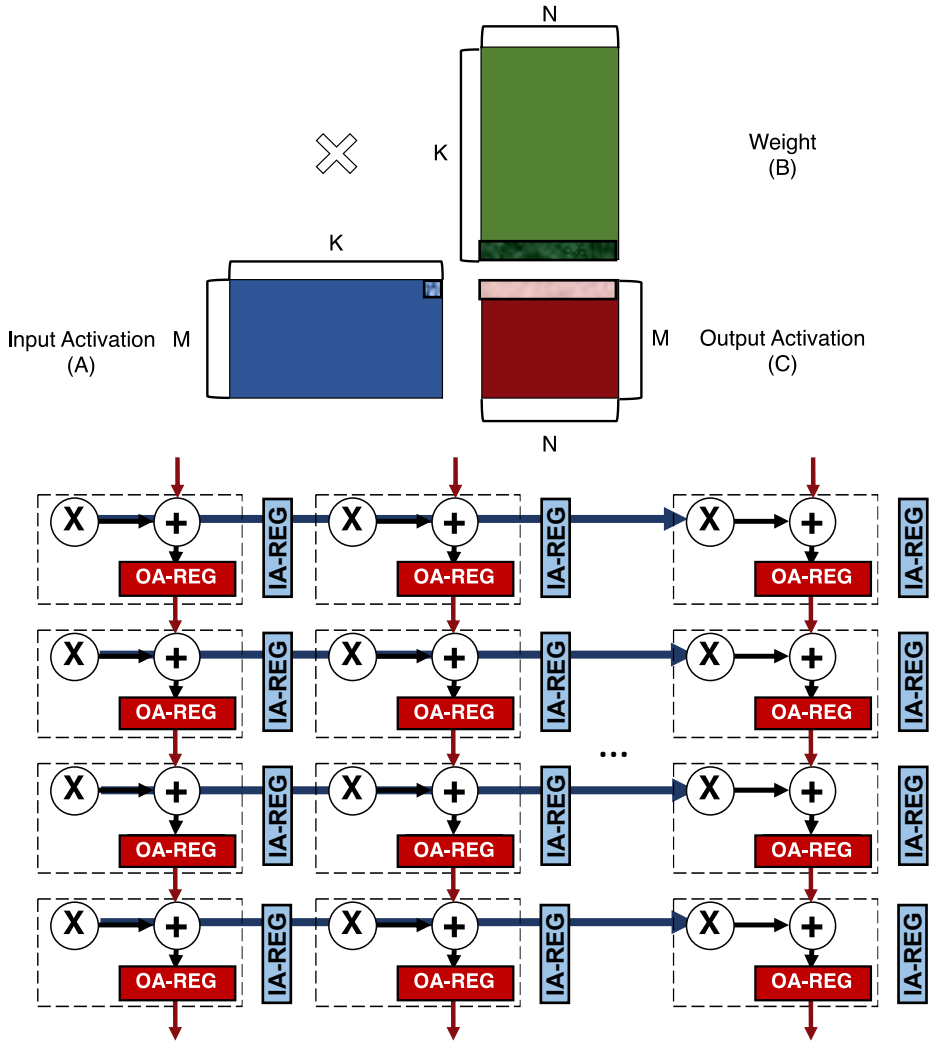
    # Level 0 - PE
    if PE_dataflow == "sq" ):
      for s in [0, S):
        for q in [0, Q):
          w = q + s
          o[m, q] += i[w] * f[m, s]
    elif PE_dataflow == "qs" :
      for q in [0, Q):
        for s in [0, S):
          w = q + s
          o[m, q] += i[w] * f[m, s]
```

Recap: Matrix Multiply




```
for (m=0; m<M; m++) {                    } for each output activation
        for (n=0; n<N; n++) {
                OA[n,m] = 0;
                for (k=0; k<K; k++) {    } convolution window
                        OA[n,m] += IA[m, k] * W[k, n];
                }
        }
}
```

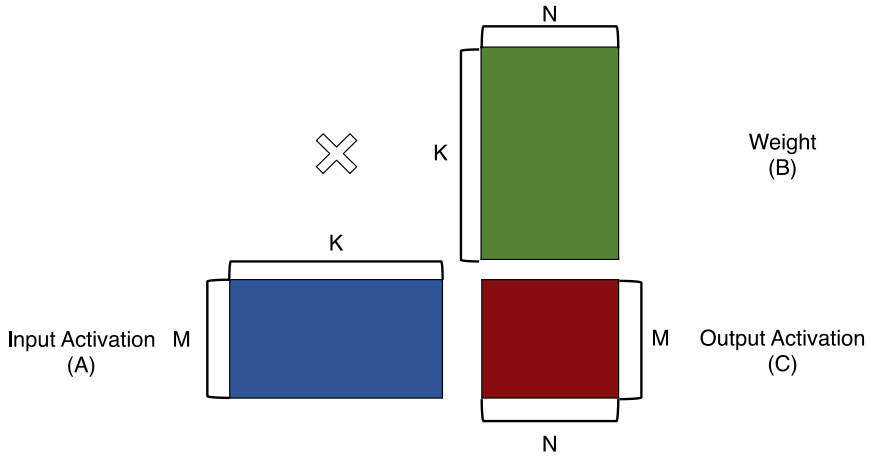
Recap: Loop Nest for TPU

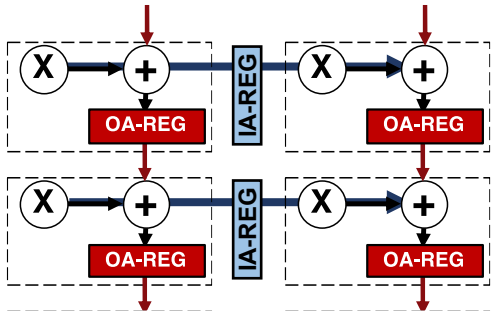



```
for (m=0; m<M; m++) {
        spatial_for (n=0; n<N; n++) {
                OA[n,m] = 0;
                 spatial_for (k=0; k<K; k++) {
                        OA[n,m] +=  IA[m, k] * W[k, n];
                }
                OA[n,m] = Activation(OA[n,m]);
        }
}
```

- What's the mapping constraints for this hardware?

What If The Systolic Array Size < K/N?

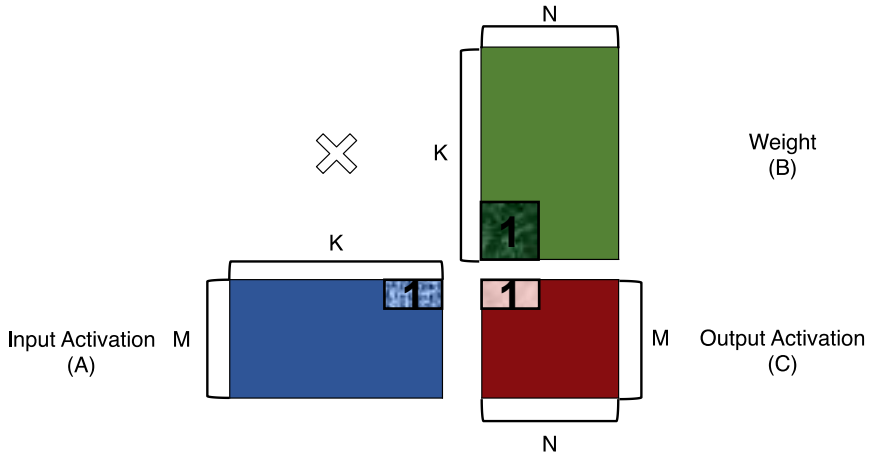


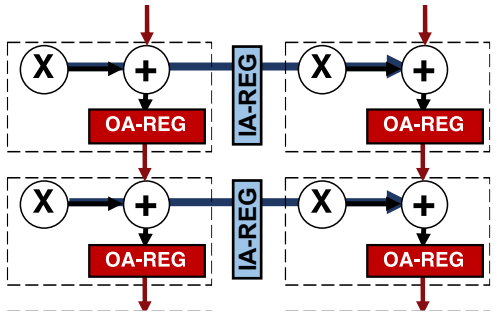


- Hardware Specifications:
  - Systolic array size: 2x2

```
for (m=0; m<M; m++) {
        spatial_for (n=0; n<2; n++) {
                OA[n,m] = 0;
                spatial_for (k=0; k<2; k++) {
                        OA[n,m] +=  IA[m, k] * W[k, n];
                }
                // OA[n,m] is not done yet
        }
}
```

What If The Systolic Array Size < K/N?

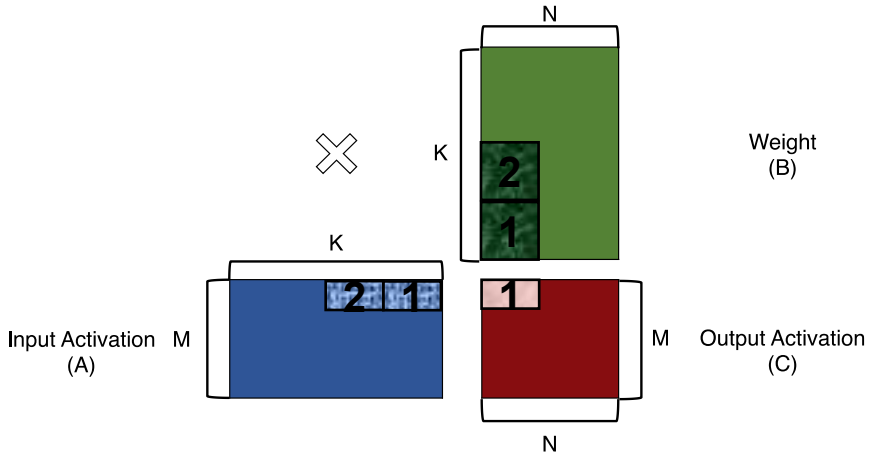


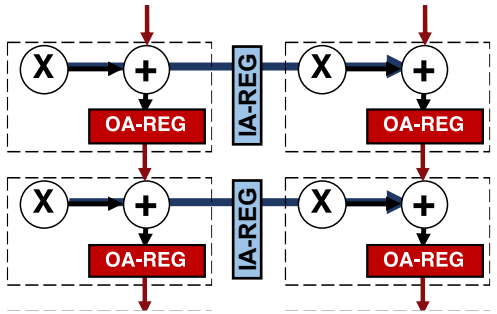


- Hardware Specifications:
  - Systolic array size: 2x2
- Notation:
  - K?/N?: loop bounds

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

What If The Systolic Array Size < K/N?

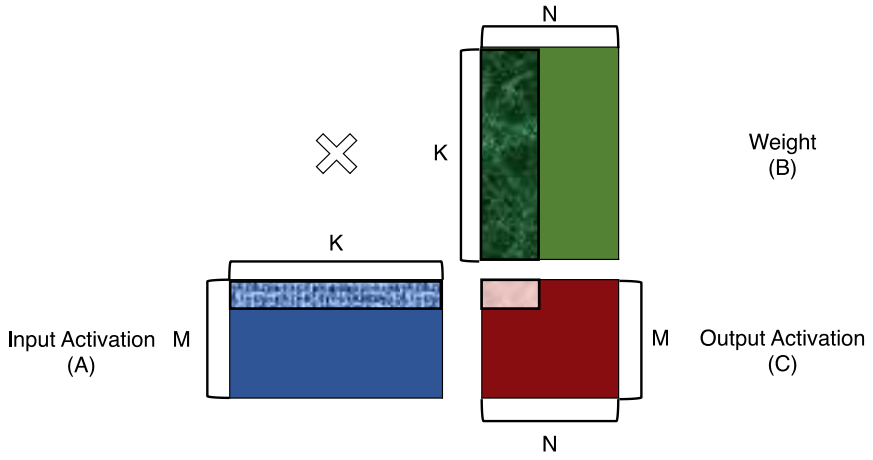


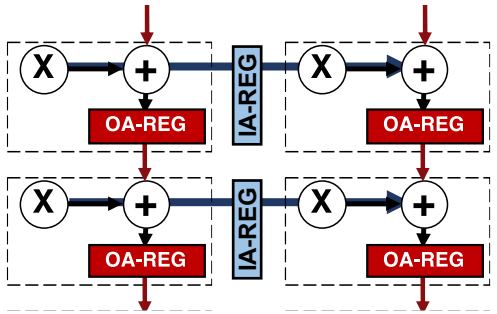


- Hardware Specifications:
  - Systolic array size: 2x2
- Notation:
  - K?/N?: loop bounds

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

What If The Systolic Array Size < K/N?

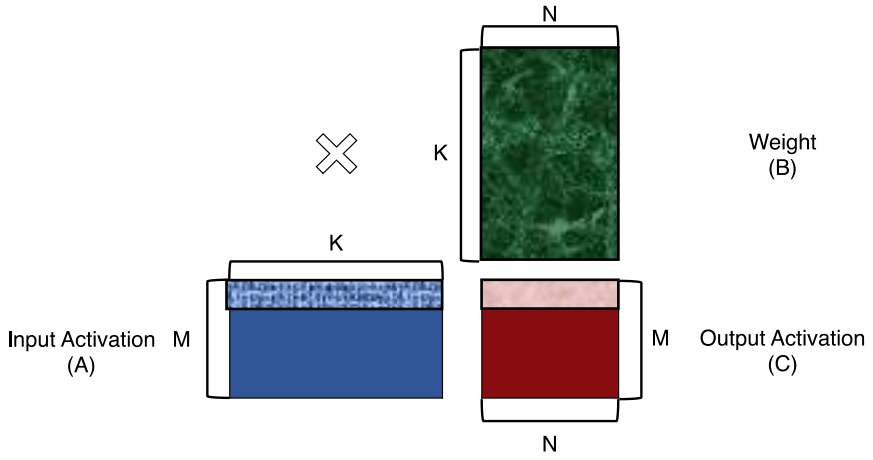


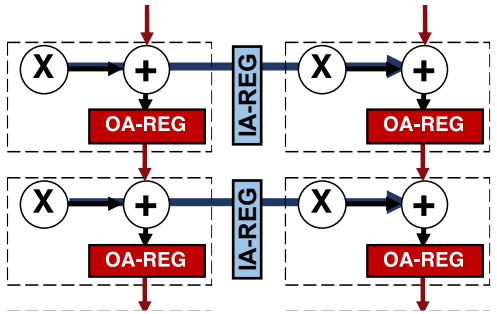


- Hardware Specifications:
  - Systolic array size: 2x2
- Notation:
  - K?/N?: loop bounds

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

What If The Systolic Array Size < K/N?

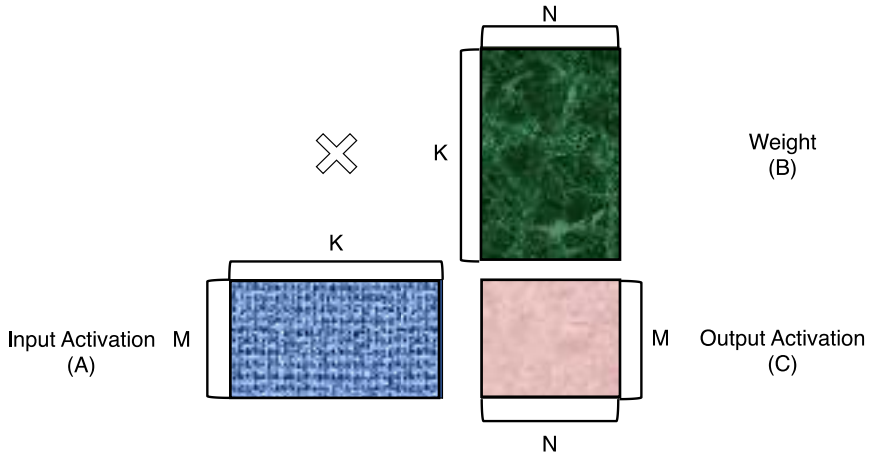


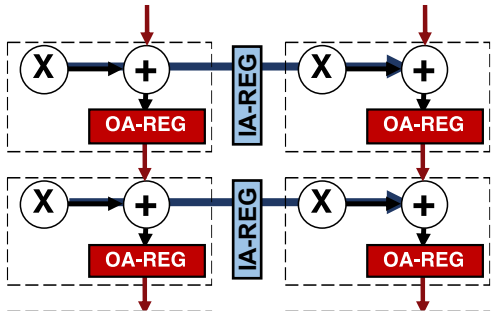


- Hardware Specifications:
  - Systolic array size: 2x2
- Notation:
  - K?/N?: loop bounds

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

What If The Systolic Array Size < K/N?

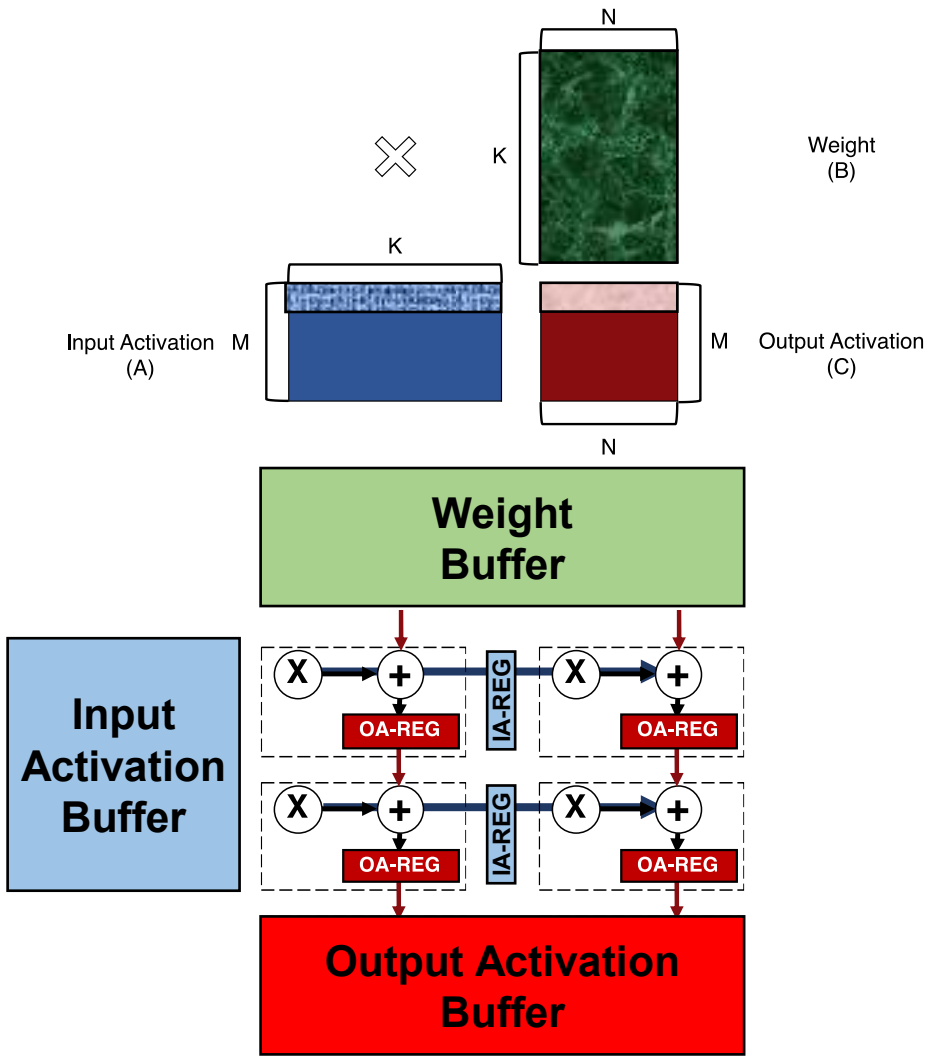




- Hardware Specifications:
  - Systolic array size: 2x2
- Notation:
  - K?/N?: loop bounds

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

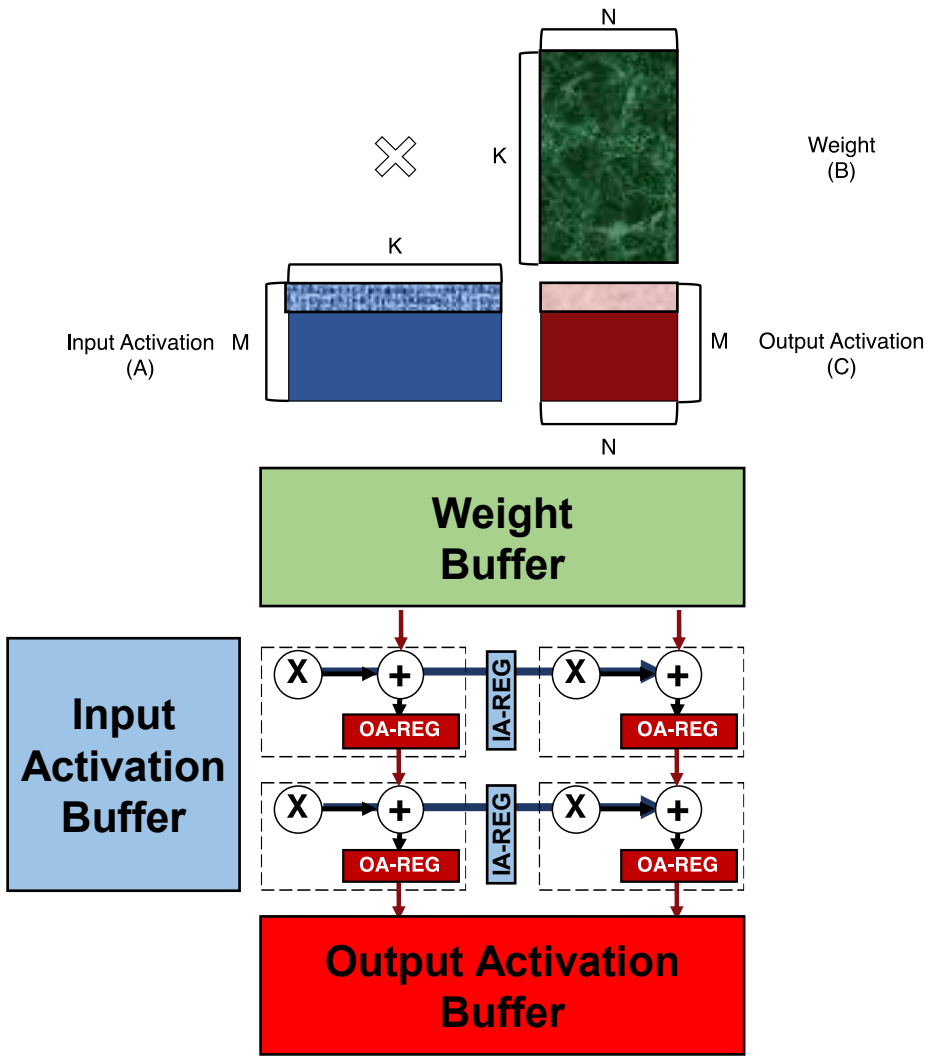
What If Weight Buffer Size < N*K?



- Hardware Specifications:
  - Systolic array size: 2x2
  - Explicit data movement

```
for (m=0; m<M; m++) {
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
}
```

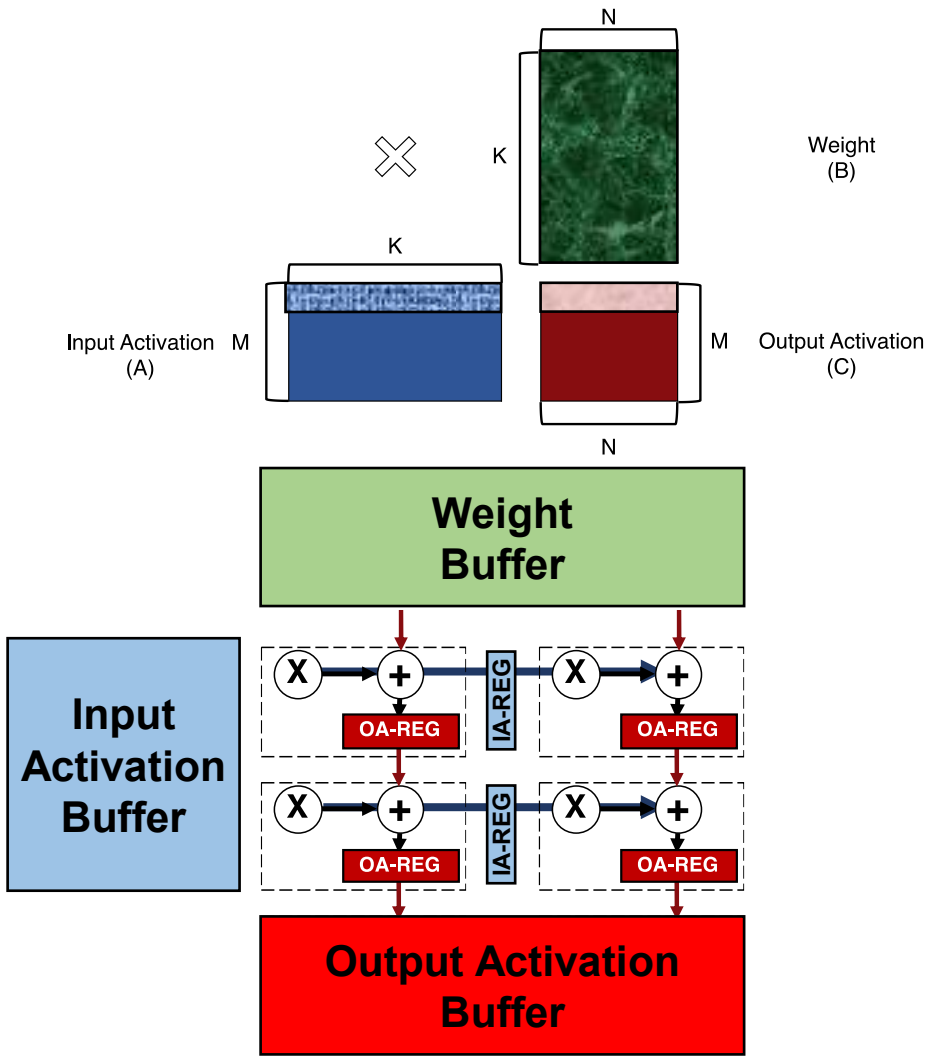
What If Weight Buffer Size < N*K?



- Hardware Specifications:
  - Systolic array size: 2x2
  - Explicit data movement

```
for (m=0; m<M; m++) {
    mvin(W[0:K,0:N]);
    mvin(IA[m:m+1,0:K]);
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
    mvout(OA[0:N,m:m+1]);
}
```

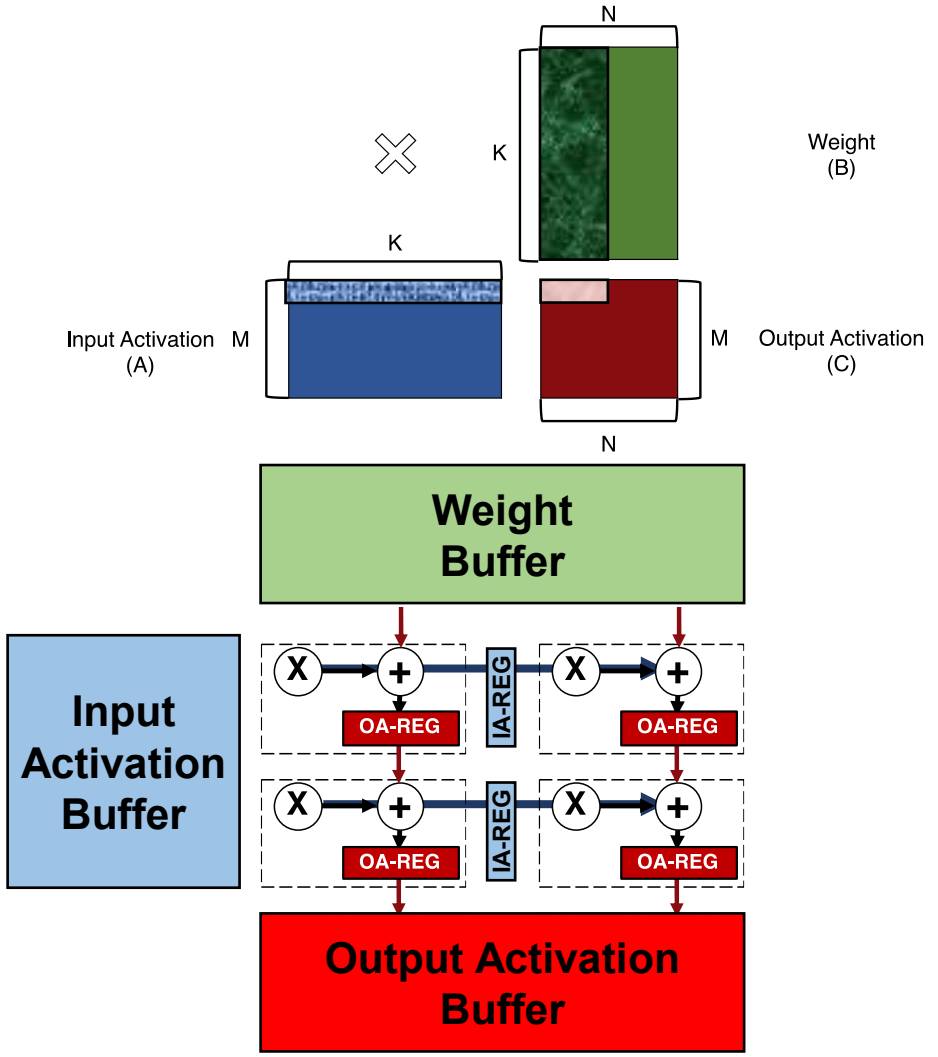
What If Weight Buffer Size < N*K?



- Hardware Specifications:
  - Systolic array size: 2x2
  - Explicit data movement
  - Output/Weight/Input buffer sizes

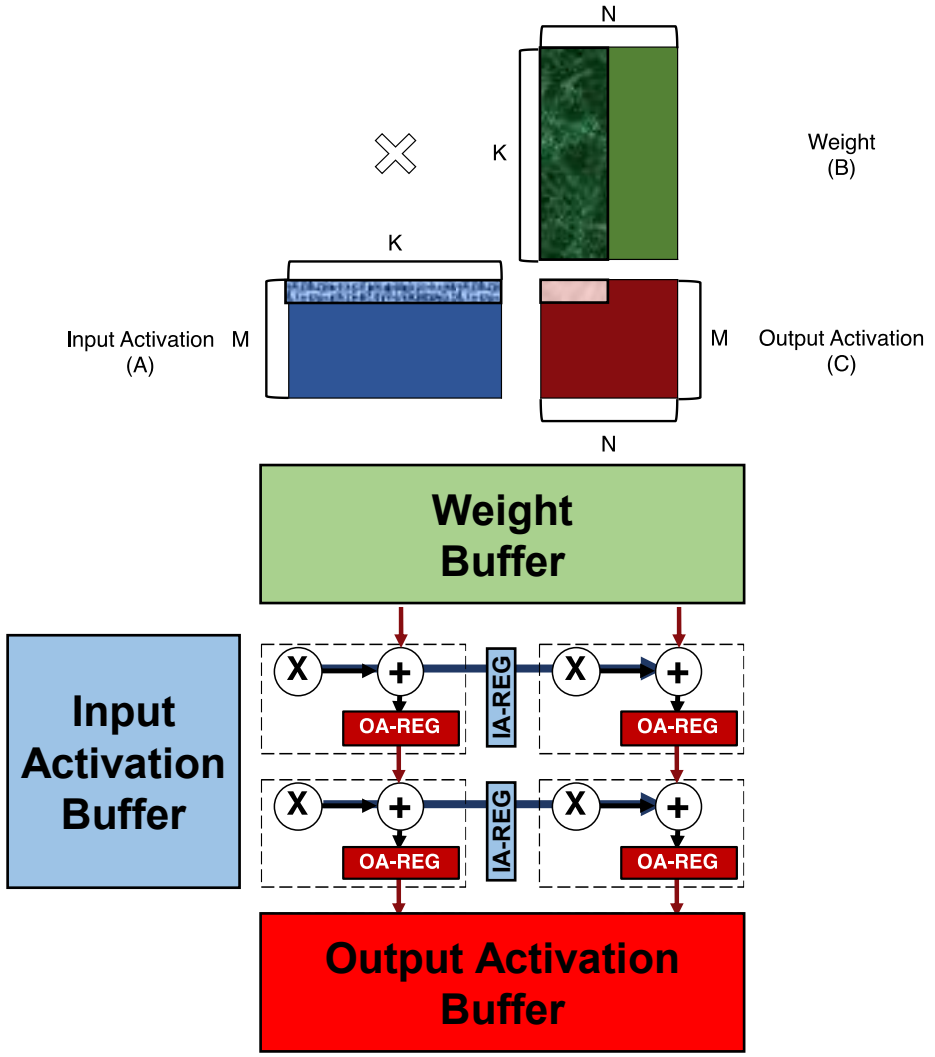
```
for (m=0; m<M; m++) {
    mvin(W[0:K,0:N]);  //W buffer >= NK
    mvin(IA[m:m+1,0:K]); //IA buffer >= 1*K
    for (n1=0; n1<N1; n1++) {
        OA[n1*N0:(n1+1)*N0,m] = 0;
        for (k1=0; k1<K1; k1++) {
            spatial_for (n0=0; n0<N0; n0++) {
                spatial_for (k0=0; k0<K0; k0++) {
                    OA[n1*N0+n0,m] += IA[m, k1*K0+k0]
                                    * W[k1*K0+k0, n1*N0+n0];
                }
            }
        }
    }
    mvout(OA[0:N,m:m+1]); //OA buffer >= 1*N
}
```

What If Weight Buffer Size < N*K?



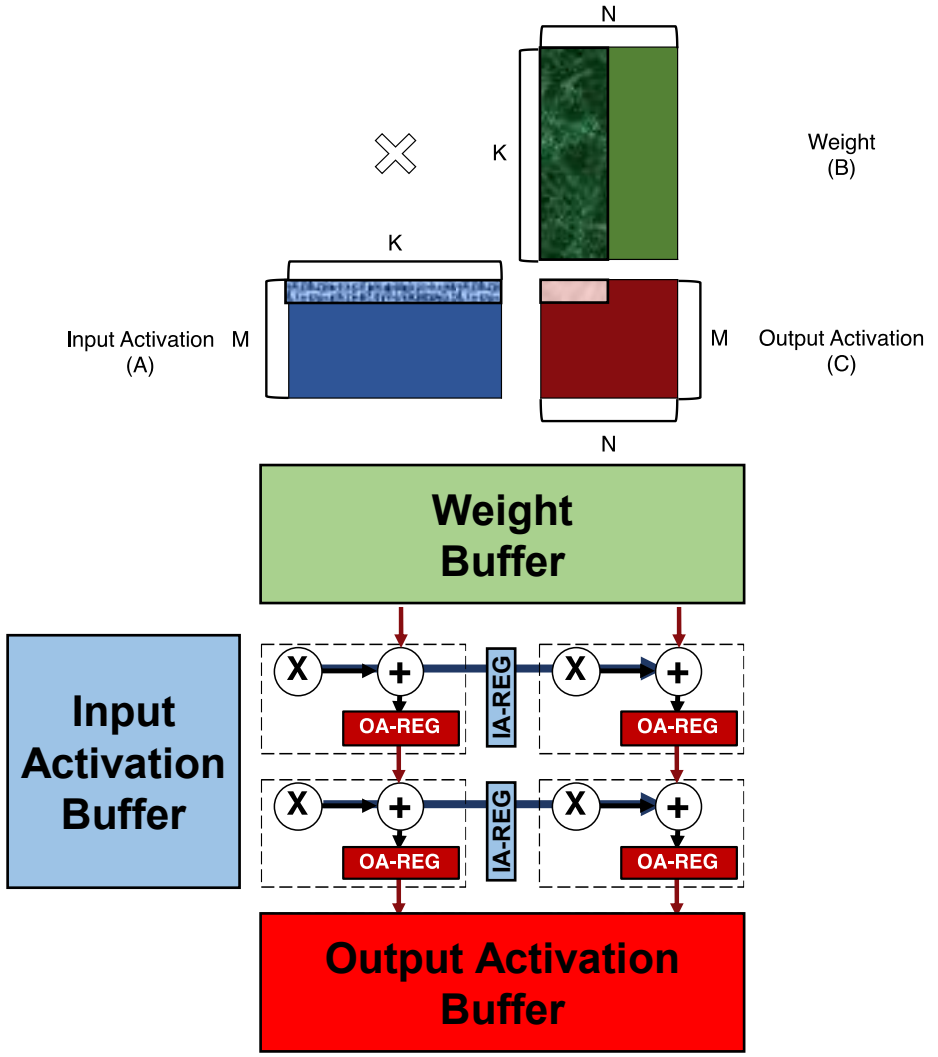

```
for (m=0; m<M; m++) {
    // IA buffer stores: 1*K
    mvin(IA[m:m+1,0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]=0;
        for (n1=0; n1<N1; n1++) {
            for (k1=0; k1<K1; k1++) {
                spatial_for (n0=0; n0<N0; n0++) {
                    spatial_for (k0=0; k0<K0; k0++) {
                        OA[n2*N1*N0+n1*N0+n0,m] +=IA[m, k1*K1+k0]
                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]);
    }
}
```

What If Weight Buffer Size < N*K?



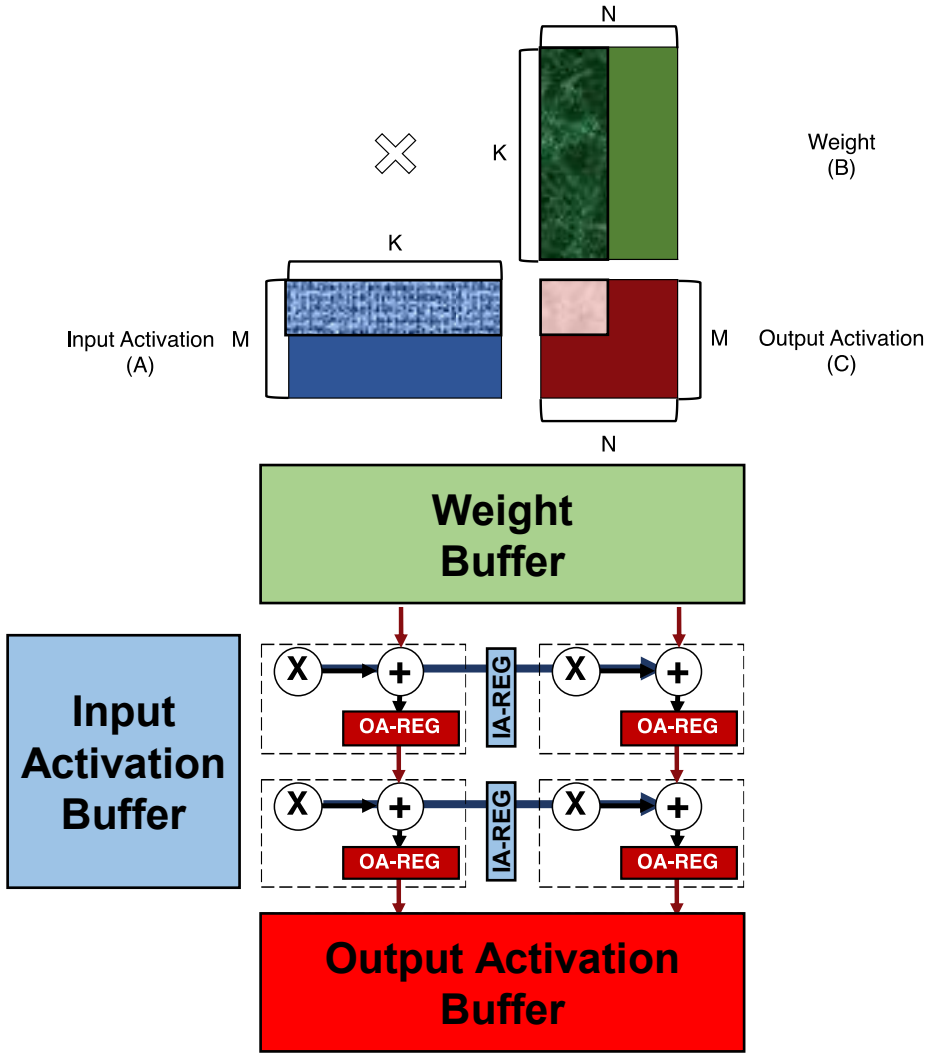

```
for (m=0; m<M; m++) {
    // IA buffer stores: 1*K
    mvin(IA[m:m+1,0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]=0;
        for (n1=0; n1<N1; n1++) {
            for (k1=0; k1<K1; k1++) {
                spatial_for (n0=0; n0<N0; n0++) {
                    spatial_for (k0=0; k0<K0; k0++) {
                        OA[n2*N1*N0+n1*N0+n0,m] +=IA[m, k1*K1+k0]
                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]);
    }
}
```

What If Weight Buffer Size < N*K?



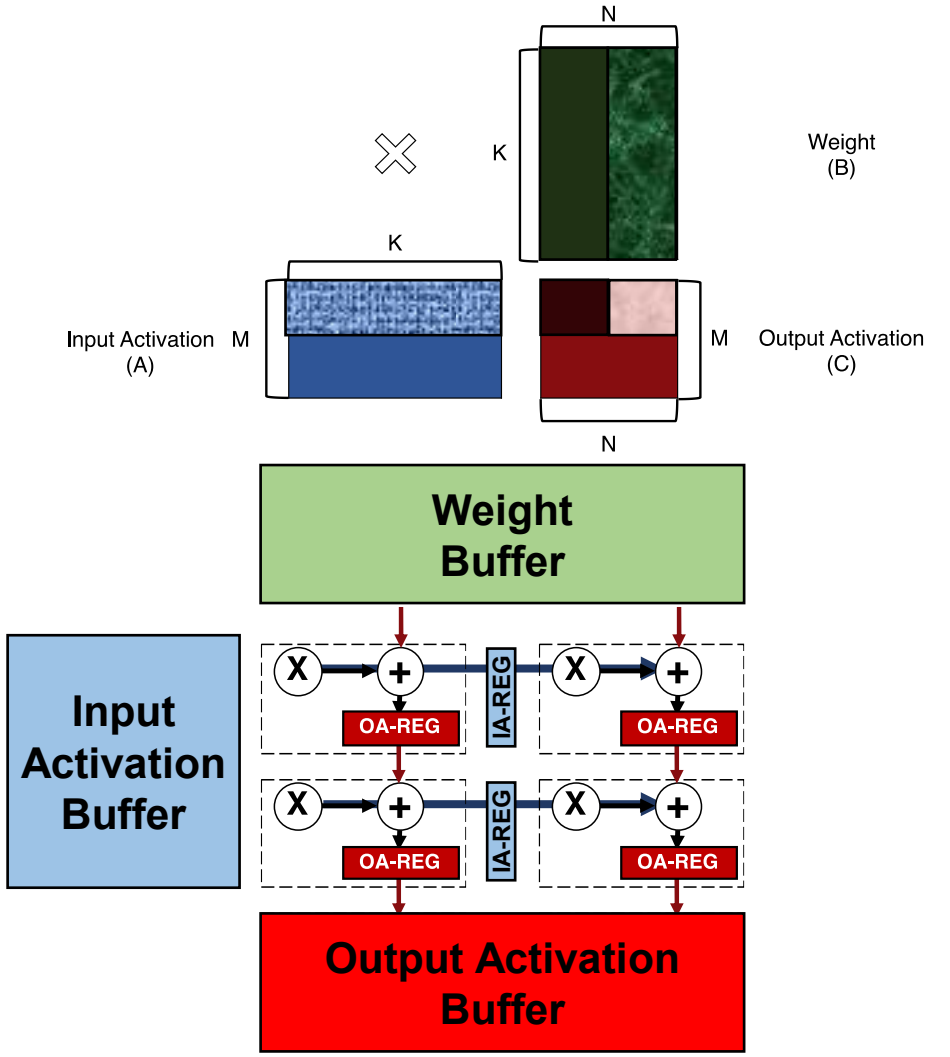

```
for (m=0; m<M; m++) {
    // IA buffer stores: 1*K
    mvin(IA[m:m+1,0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]=0;
        for (n1=0; n1<N1; n1++) {
            for (k1=0; k1<K1; k1++) {
                spatial_for (n0=0; n0<N0; n0++) {
                    spatial_for (k0=0; k0<K0; k0++) {
                        OA[n2*N1*N0+n1*N0+n0,m] +=IA[m, k1*K1+k0]
                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]);
    }
}
```

What If Weight Buffer Size < N*K?



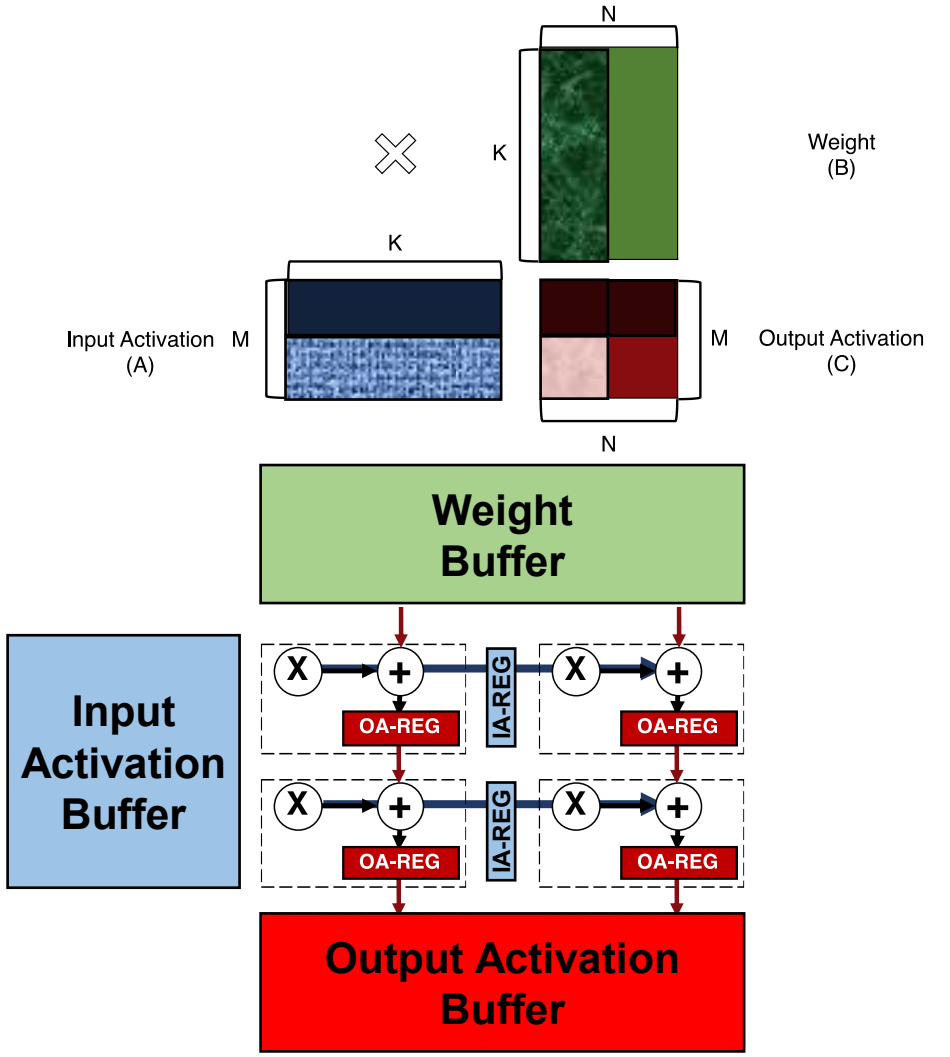

```
for (m2=0; m2<M2; m2++) {
    // IA buffer stores: M1*K
    mvin(IA[m2*M1:(m2+1)*M1,0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]=0;
        for (m1=0; m1<M1; m1++) {
            for (n1=0; n1<N1; n1++) {
                for (k1=0; k1<K1; k1++) {
                    spatial_for (n0=0; n0<N0; n0++) {
                        spatial_for (k0=0;k0<K0; k0++){
                            OA[n2*N1*N0+n1*N0+n0,m2*M1+m1]+=IA[m2*M1+m1, k1*K1+k0]
                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                        }
                    }
                }
            }
        }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
    }
}
```

What If Weight Buffer Size < N*K?



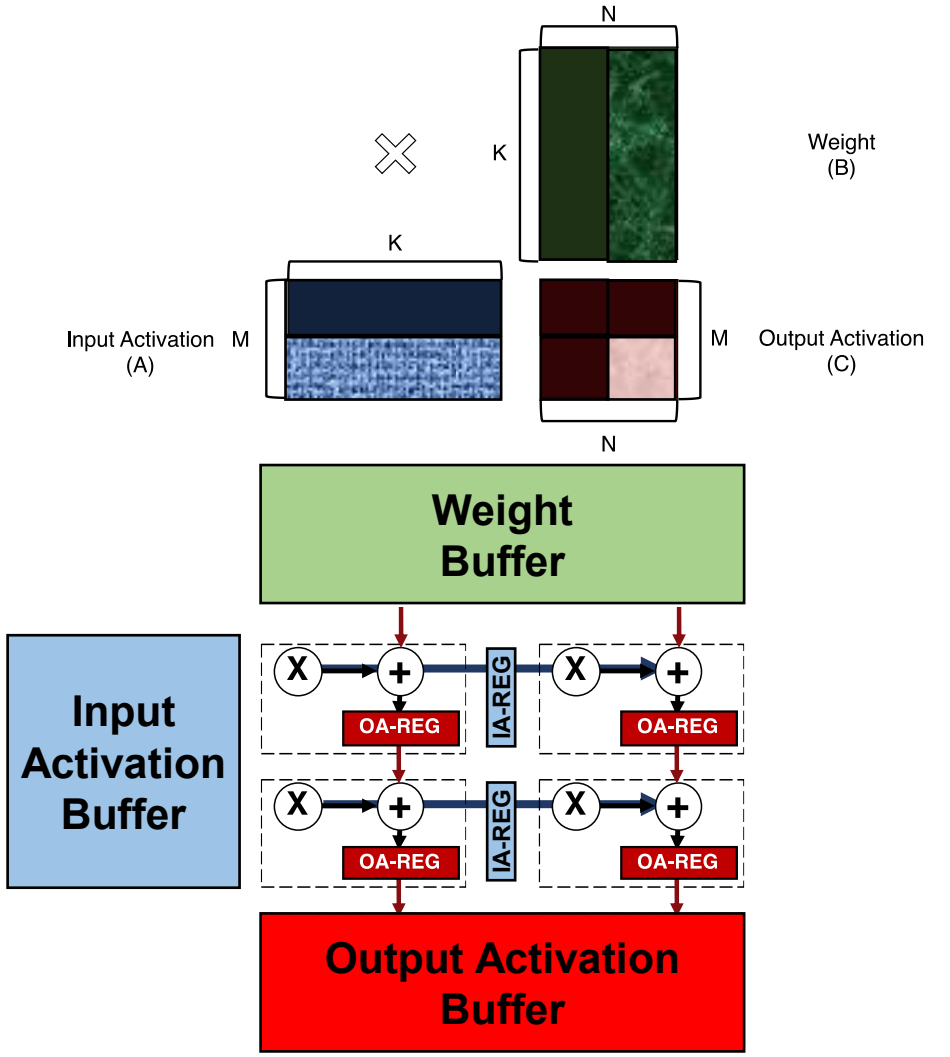

```
for (m2=0; m2<M2; m2++) {
      // IA buffer stores: M1*K
      mvin(IA[m2*M1:(m2+1)*M1,0:K]);
      for (n2=0; n2<N2; n2++) {
            // W buffer stores: N1*N0*K
            mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
            OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]=0;
            for (m1=0; m1<M1; m1++) {
                  for (n1=0; n1<N1; n1++) {
                        for (k1=0; k1<K1; k1++) {
                              spatial_for (n0=0; n0<N0; n0++) {
                                    spatial_for (k0=0;k0<K0; k0++){
                                     OA[n2*N1*N0+n1*N0+n0,m2*M1+m1]+=IA[m2*M1+m1, k1*K1+k0]
                                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                                    }
                              }
                        }
                  }
                                                      }
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
      }
}
```

What If Weight Buffer Size < N*K?



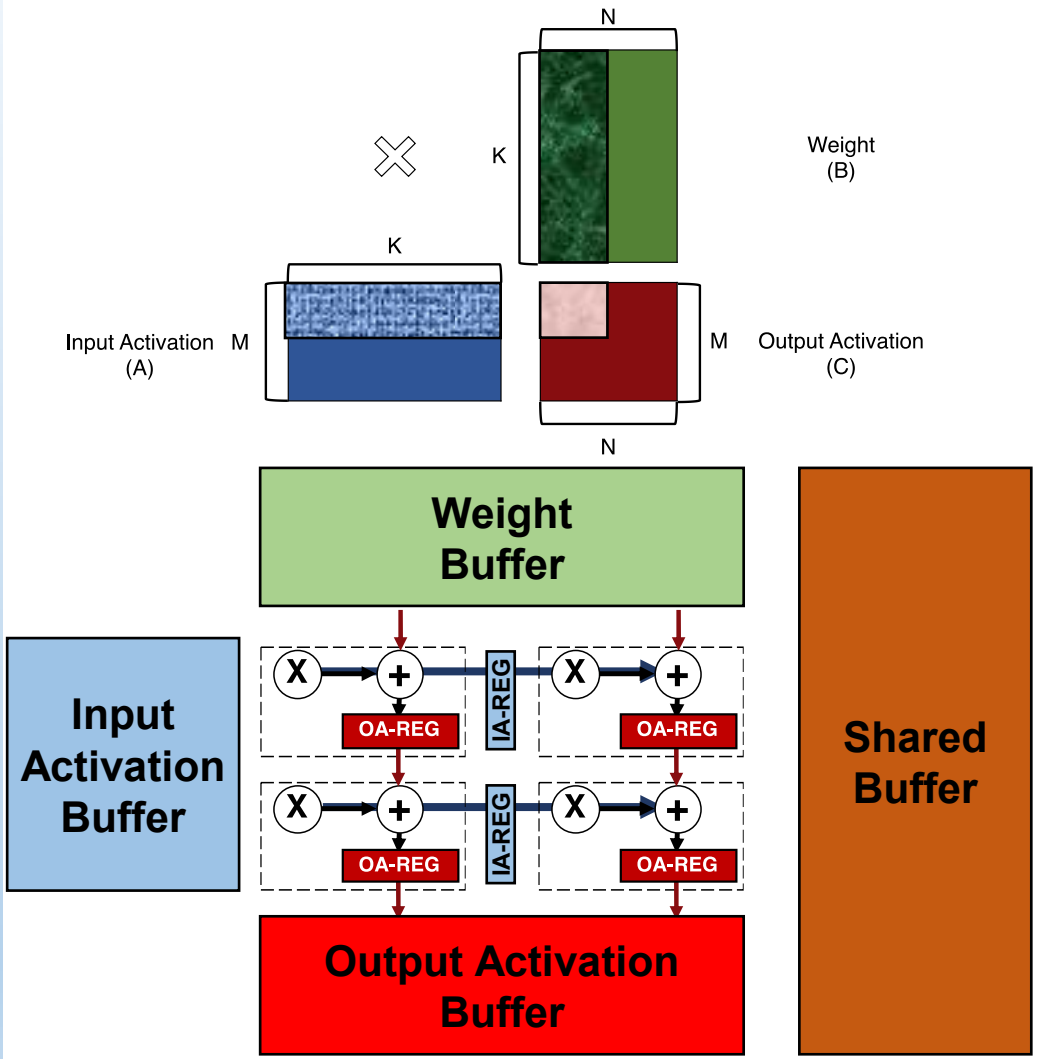

```
for (m2=0; m2<M2; m2++) {
      // IA buffer stores: M1*K
      mvin(IA[m2*M1:(m2+1)*M1,0:K]);
      for (n2=0; n2<N2; n2++) {
            // W buffer stores: N1*N0*K
            mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
            OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]=0;
            for (m1=0; m1<M1; m1++) {
                  for (n1=0; n1<N1; n1++) {
                        for (k1=0; k1<K1; k1++) {
                              spatial_for (n0=0; n0<N0; n0++) {
                                    spatial_for (k0=0;k0<K0; k0++){
                                     OA[n2*N1*N0+n1*N0+n0,m2*M1+m1]+=IA[m2*M1+m1, k1*K1+k0]
                                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                                    }
                              }
                        }
                  }
                                                }
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
      }
}
```

What If Weight Buffer Size < N*K?



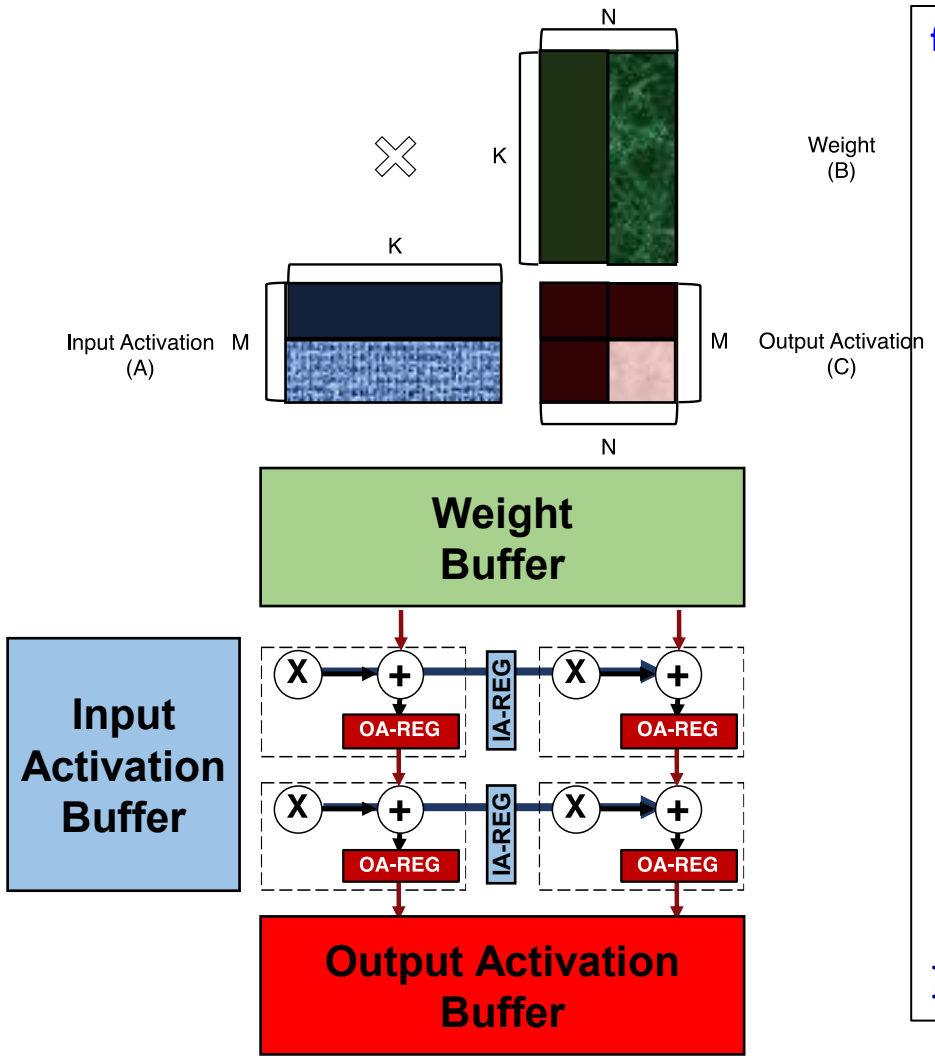

```
for (m2=0; m2<M2; m2++) {
      // IA buffer stores: M1*K
      mvin(IA[m2*M1:(m2+1)*M1,0:K]);
      for (n2=0; n2<N2; n2++) {
            // W buffer stores: N1*N0*K
            mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
            OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]=0;
            for (m1=0; m1<M1; m1++) {
                  for (n1=0; n1<N1; n1++) {
                        for (k1=0; k1<K1; k1++) {
                              spatial_for (n0=0; n0<N0; n0++) {
                                    spatial_for (k0=0;k0<K0; k0++){
                                     OA[n2*N1*N0+n1*N0+n0,m2*M1+m1]+=IA[m2*M1+m1, k1*K1+k0]
                                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                                    }
                              }
                        }
                  }
                                                      }
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
      }
}
```

What If There Is Another Shared Buffer?




```
for (m3=0; m3<M3; m3++) {
    for (n3=0; n3<N3; n3++) {
        // Shared buffer blocking
        for (m2=0; m2<M2; m2++) {
            // IA buffer stores: M1*K
            mvin(IA[…:…]);
            for (n2=0; n2<N2; n2++) {
                // W buffer stores: N1*N0*K
                mvin(W[…:…,…:…]);
                OA[…:…,…:…]=0;
                for (m1=0; m1<M1; m1++) {
                    for (n1=0; n1<N1; n1++) {
                        for (k1=0; k1<K1; k1++) {
                            …
                        }
                    }
                }
            }
            mvout(OA[…:…,…:…]);
        }
    }
}
```

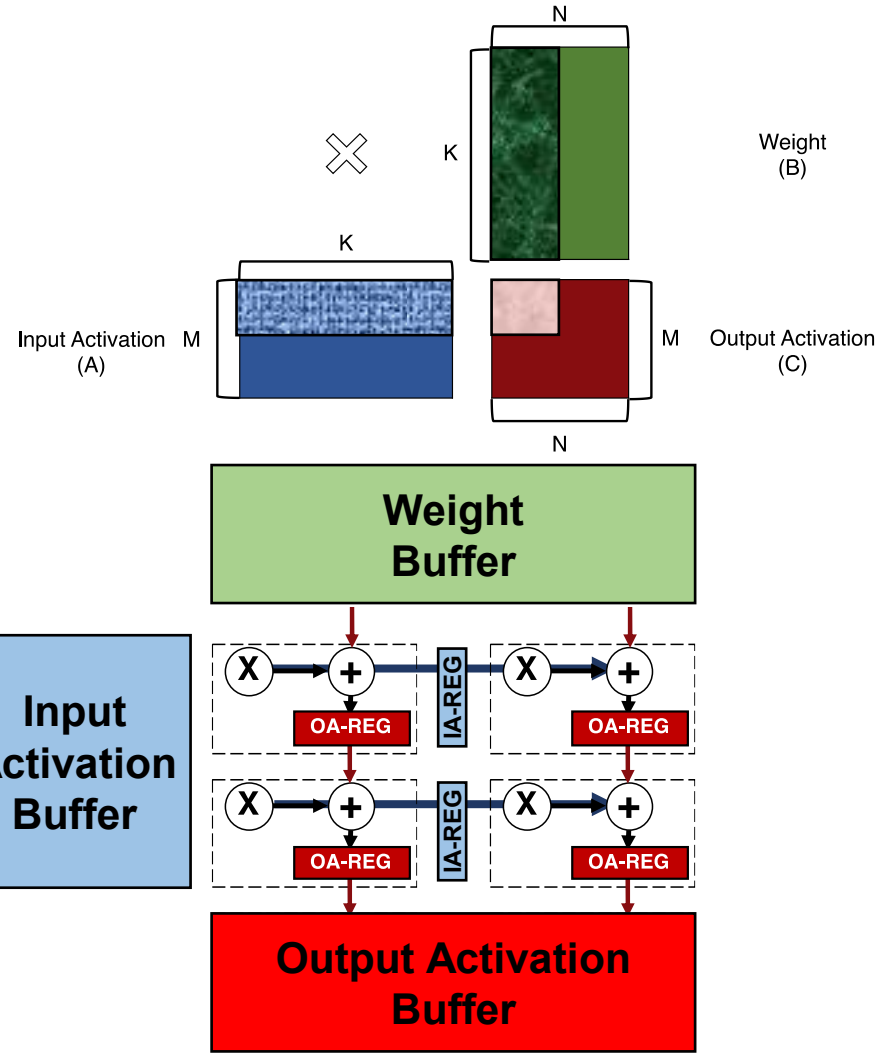
Loop Bounds



$$M1 \times M2 = M$$

```
for (m2=0; m2<M2; m2++) {
    // IA buffer stores: M1*K
    mvin(IA[m2*M1:(m2+1)*M1,0:K]);
    for (n2=0; n2<N2; n2++) {
        // W buffer stores: N1*N0*K
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
        OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]=0;
        for (m1=0; m1<M1; m1++) {
            for (n1=0; n1<N1; n1++) {
                for (k1=0; k1<K1; k1++) {
                    spatial_for (n0=0; n0<N0; n0++) {
                        spatial_for (k0=0;k0<K0; k0++){
                            OA[n2*N1*N0+n1*N0+n0,m2*M1+m1]+=IA[m2*M1+m1, k1*K1+k0]
                                * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];
                        }
                    }
                }
            }
                                                    }
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
    }
}
```

Loop Ordering




```
for (m2=0; m2<M2; m2++) {
      // IA buffer stores: M1*K
      mvin(IA[m2*M1:(m2+1)*M1,0:K]);
      for (n2=0; n2<N2; n2++) {
            // W buffer stores: N1*N0*K
            mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
            compute_matmul(*W, *IA, *OA,
                        N2, N1, N0,
                        M2, M1,
                        K1, K0,
                        m2, n2);
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0
                ,m2*M1:(m2+1)*M1]);
      }
}
```

Loop Ordering

Option 1: Loop_m2 -> Loop_n2

```
for (m2=0; m2<M2; m2++) {
      // IA buffer stores: M1*K
      mvin(IA[m2*M1:(m2+1)*M1,0:K]);
      for (n2=0; n2<N2; n2++) {
            // W buffer stores: N1*N0*K
            mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
            compute_matmul(*W, *IA, *OA,
                        N2, N1, N0,
                        M2, M1,
                        K1, K0,
                        m2, n2);
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
      }
}
```

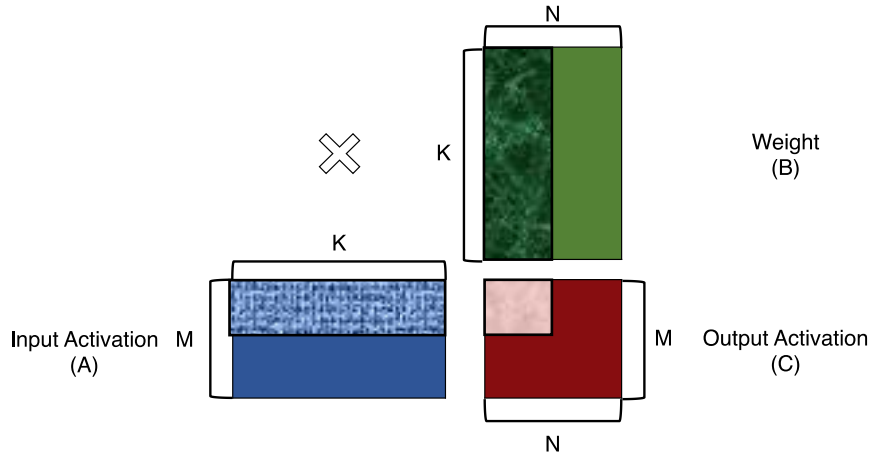
IA movement: M * K
W movement: M2 * N * K

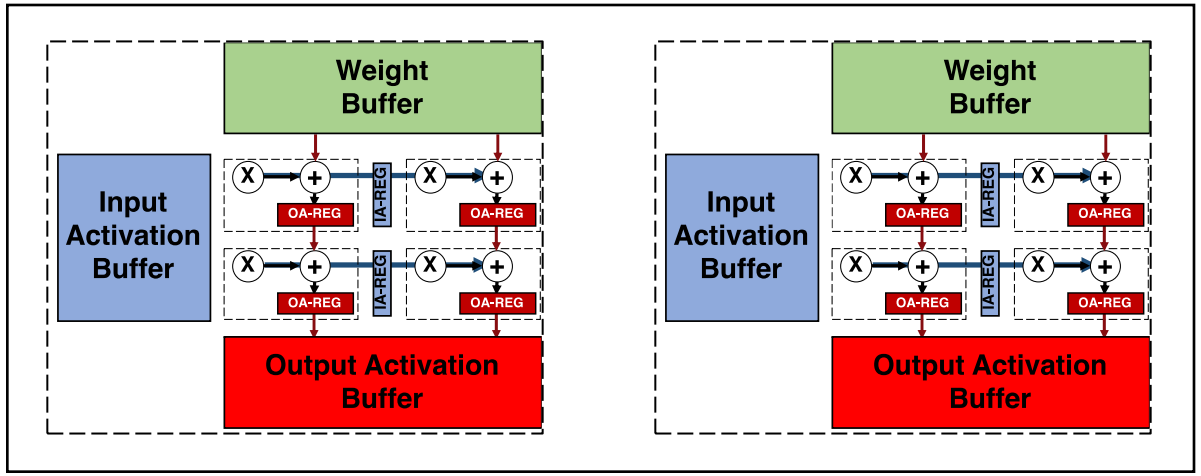
Option 2: Loop_n2 -> Loop_m2

```
for (n2=0; n2<N2; n2++) {
      # W buffer stores: N1*N0*K
      mvin(W[0:K,n2*N1*N0: (n2+1)*N1*N0]);
      for (m2=0; m2<M2; m2++) {
            # IA buffer stores: M1*K
            mvin(IA[m2*M1:(m2+1)*M1,0:K]);
            compute_matmul(*W, *IA, *OA,
                        N2, N1, N0,
                        M2, M1,
                        K1, K0,
                        m2, n2);
            mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
      }
}
```

IA movement: N2 * M* K
W movement: N * K

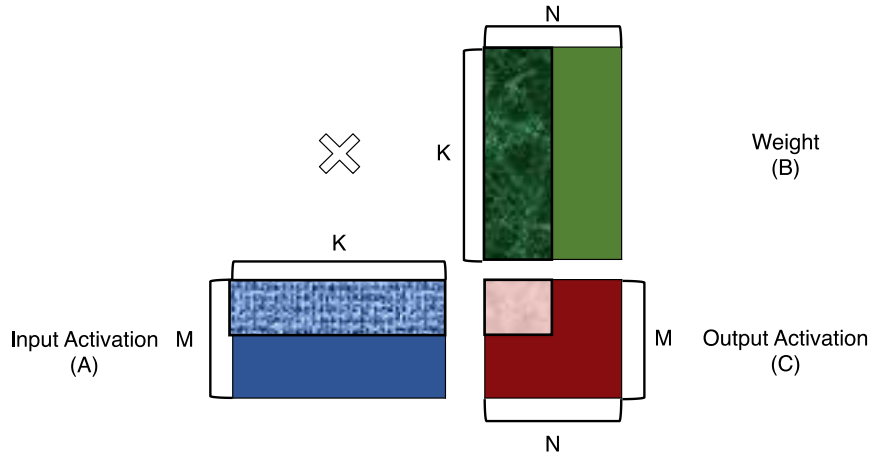
Spatial Choice

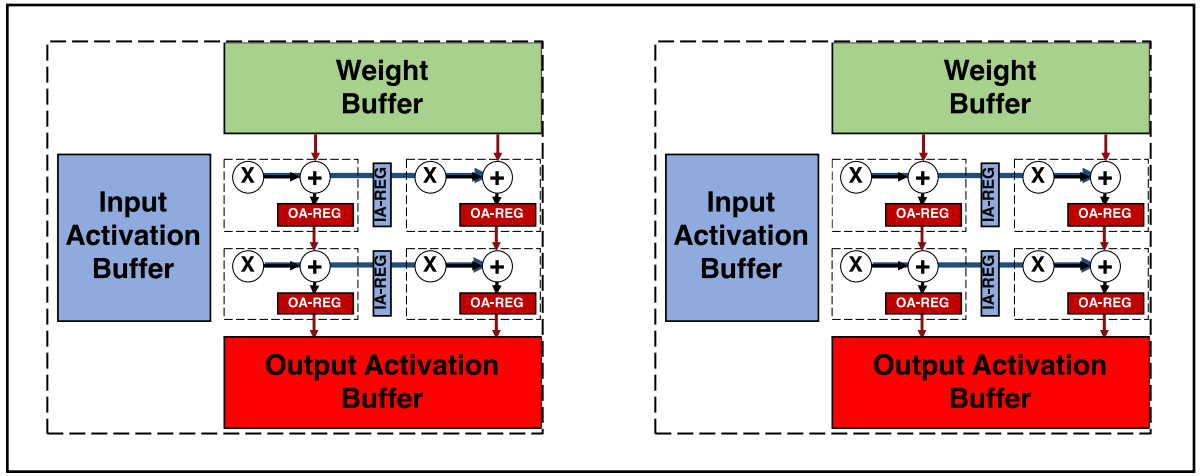





```
spatial_for (n2=0; n2<N2; n2++) {
    # Model Parallelism
    mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);
    for (m2=0; m2<M2; m2++) {
        mvin(IA[m2*M1:(m2+1)*M1,0:K]);
        compute_matmul(*W, *IA, *OA,
                N2, N1, N0,
                M2, M1,
                K1, K0,
                m2, n2);
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
    }
}
```

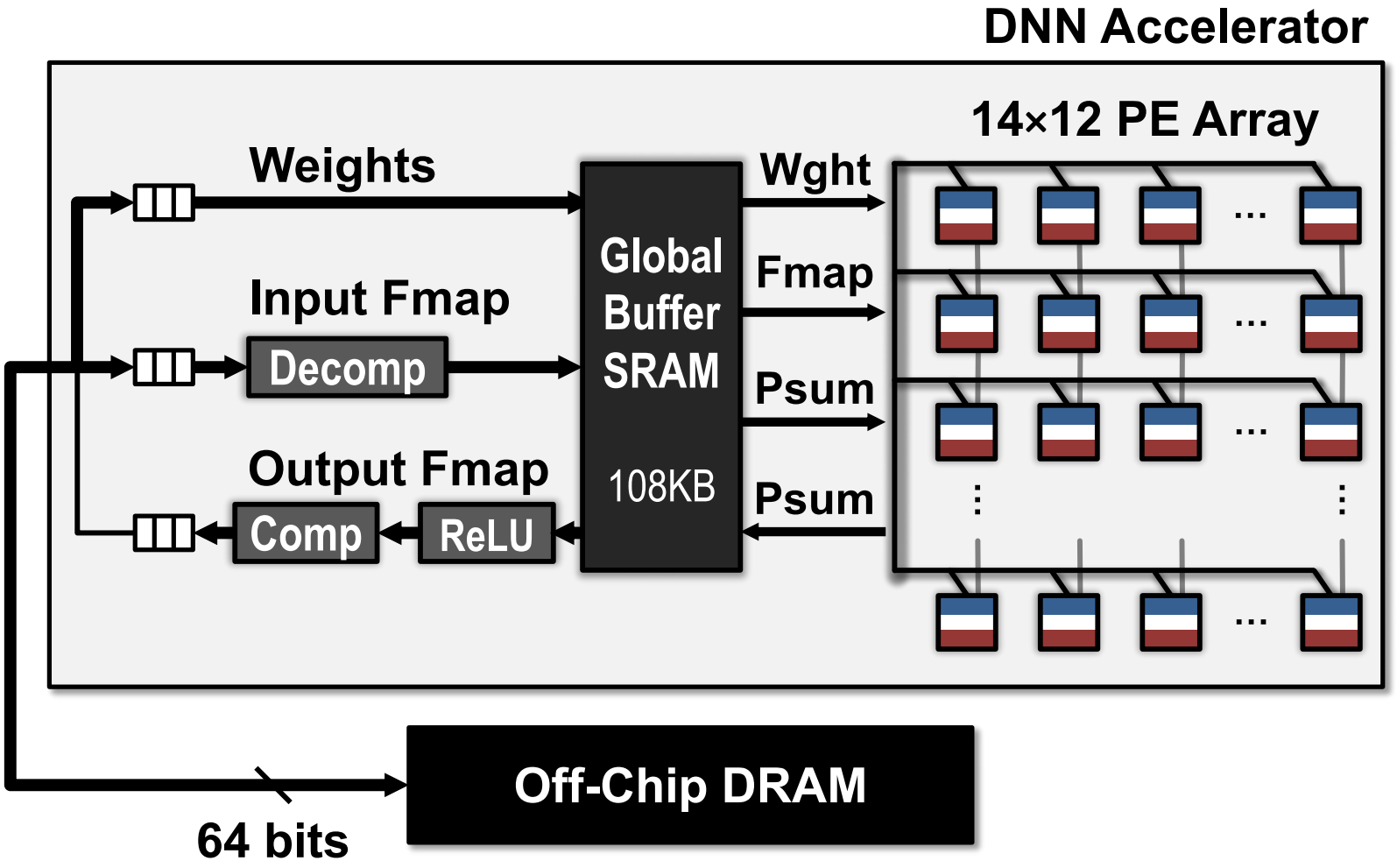
Spatial Choice



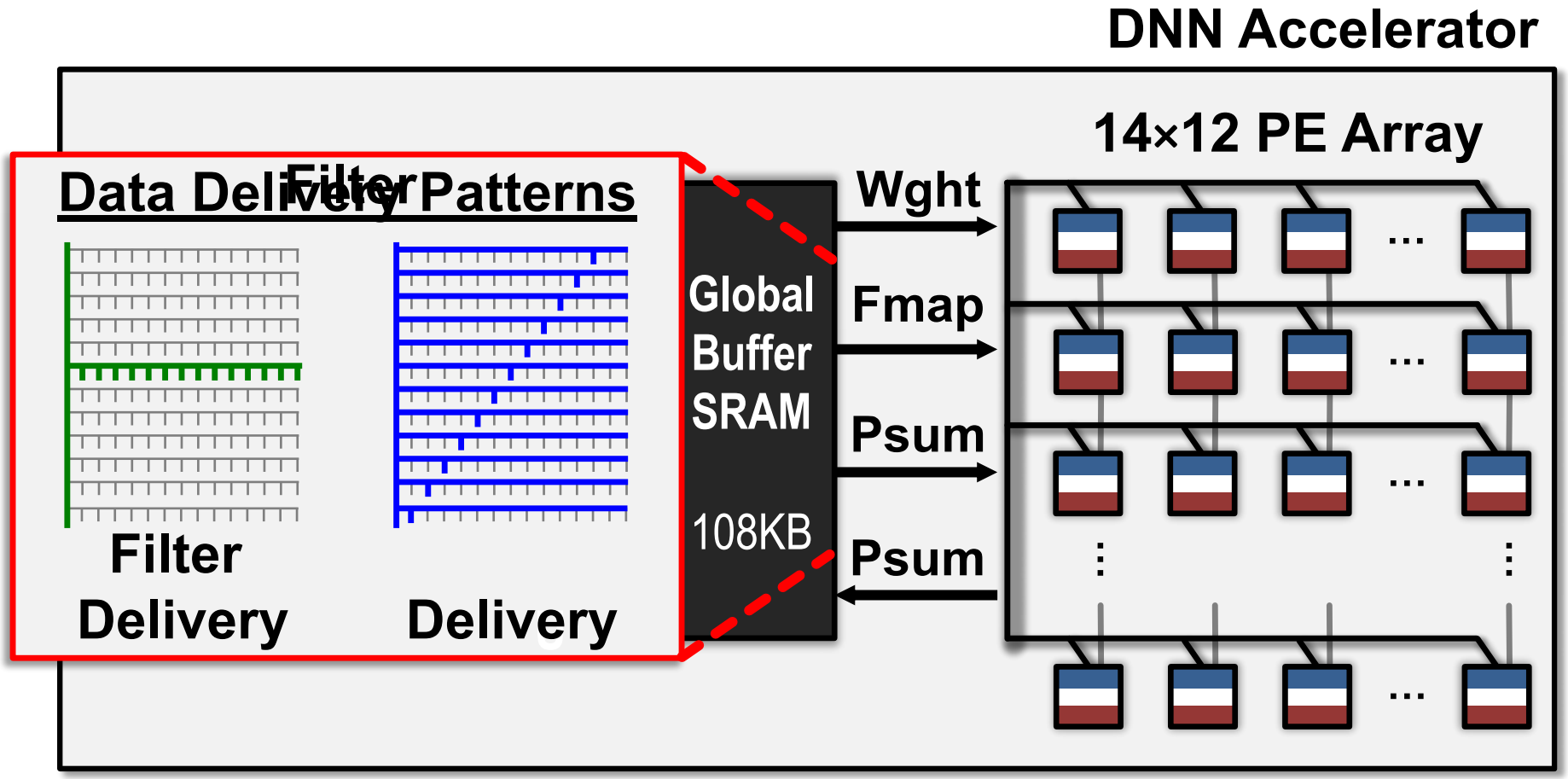



```
spatial_for (m2=0; m2<M2; m2++) {
    # Data Parallelism
    mvin(IA[m2*M1:(m2+1)*M1,0:K]);
    for (n2=0; n2<N2; n2++) {
        mvin(W[0:K,n2*N1*N0(n2+1)*N1*N0]);
        compute_matmul(*W, *IA, *OA,
                N2, N1, N0,
                M2, M1,
                K1, K0,
                m2, n2);
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m2*M1:(m2+1)*M1]);
    }
}
```

Data Delivery Pattern – Hardware Constraint



Data Delivery Pattern – Hardware Constraint



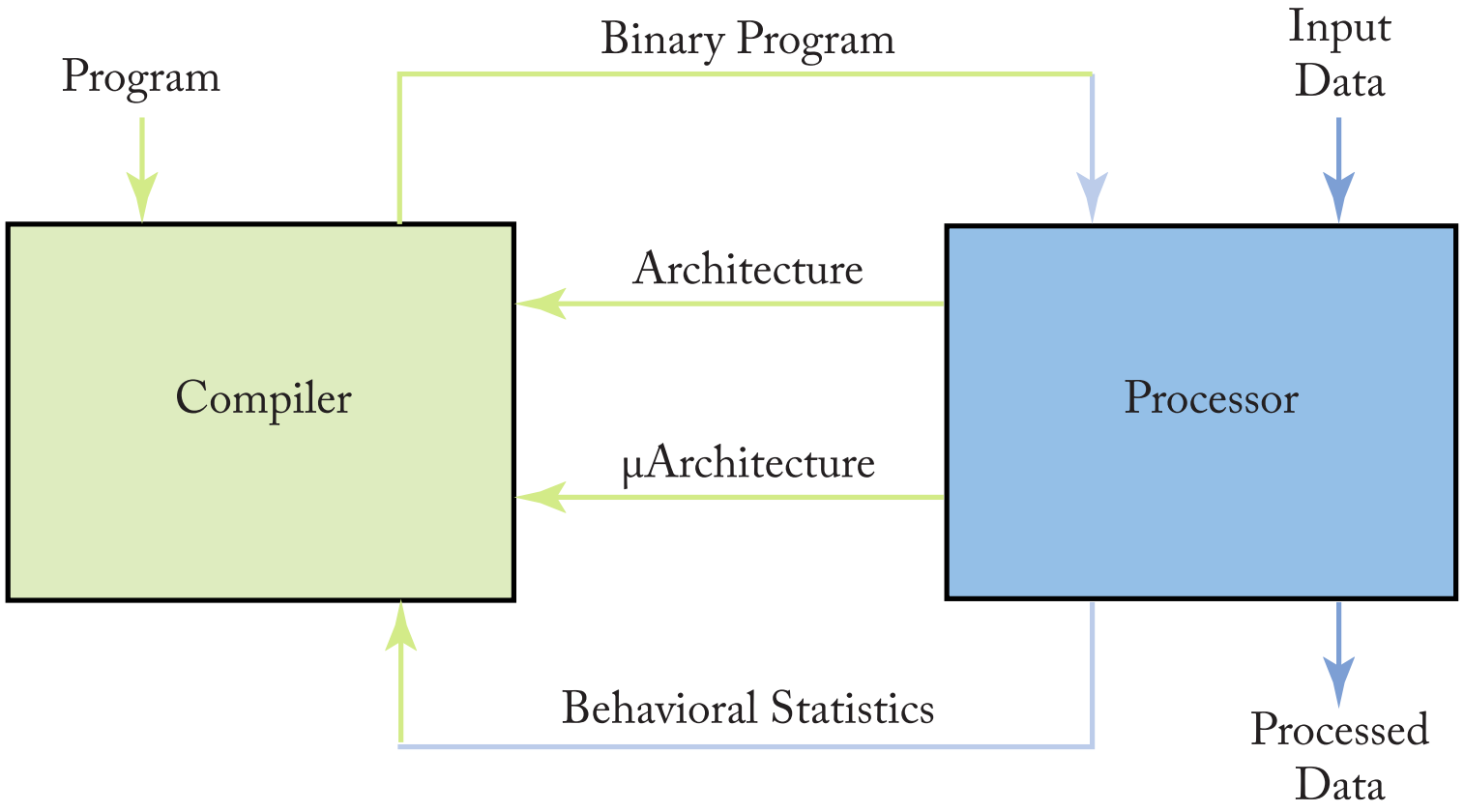
Outline



- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

General Purpose Processor Compilation Flow

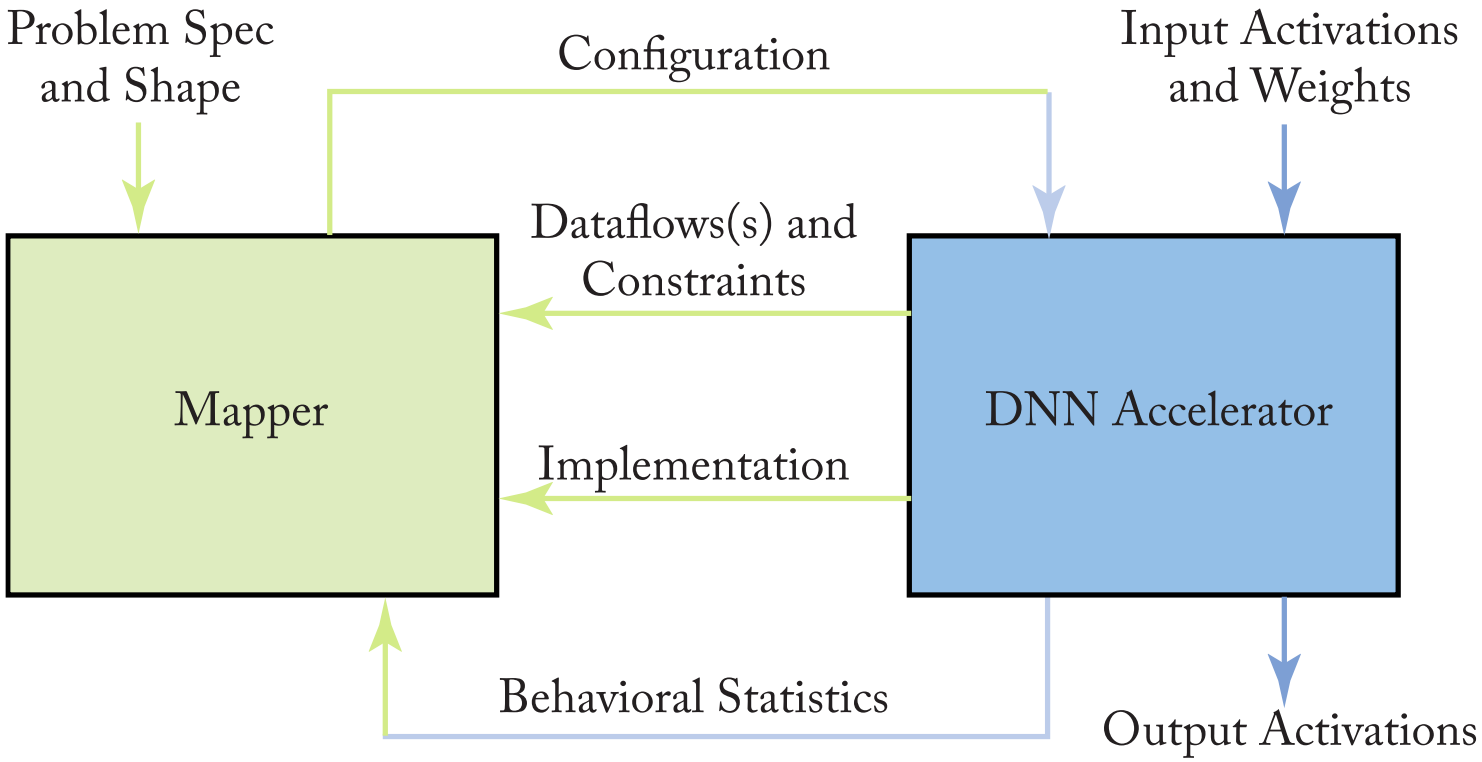
- **Translates** the **program** into **machine-readable binary codes** for execution



Usually optimizes just for performance

DNN Accelerator Mapping Flow

- **Translates** the **desired DNN layer** along with its shape and size into a **hardware compatible mapping** for execution



Typically optimize for performance and/or energy efficiency

Architecture vs. Dataflow

- Analogous
  - Prescribes **what constitutes a properly formed program** for the system
  - Characterizes the **hardware implementation**
  - Defines the many of the **rules** that the **mapper/compiler has to follow**
  - Remains stable or "**upward compatible**"
- Stability including **supported dataflow could diminish for DNN Hardware architecture**
  - **Increased reliance on the compiler/mapper** to mask differences between designs.

Architecture vs. Constraints for AI Processor

- Buffer size
  - Storage sizes in a **general processor** (like cache sizes) are **generally not included in the architecture**
- **Buffer size in AI Processor is similar to register file in CPU**
  - Manifests as inviolable constraints for creating legal binaries.
- **Network-on-Chip**
  - Define **data delivery pattern**

Micro-Architecture and Hardware Implementation

- Detailed information about hardware implementation
  - Latency
  - Energy Cost for storage access
  - NoC traffic at each level of the storage hierarchy
- Analogous
  - **Vary considerably across implementations**
  - Play a vital part in performance and energy efficiency optimization
  - Characteristics is not essential
  - Can be ignored to generate a functional, sub-optimal mappings

Behavior Statistics

- Compiler/mapper can better decide on good optimizations via **iterative refinement using behavioral statistics**
- In many cases, this information is **not available**
  - Compiler/mapper must **make its own projections** of the impact of a choice it makes
- For AI Processor mapper
  - Modeling the design to project metrics of interest for a given mapping

Output

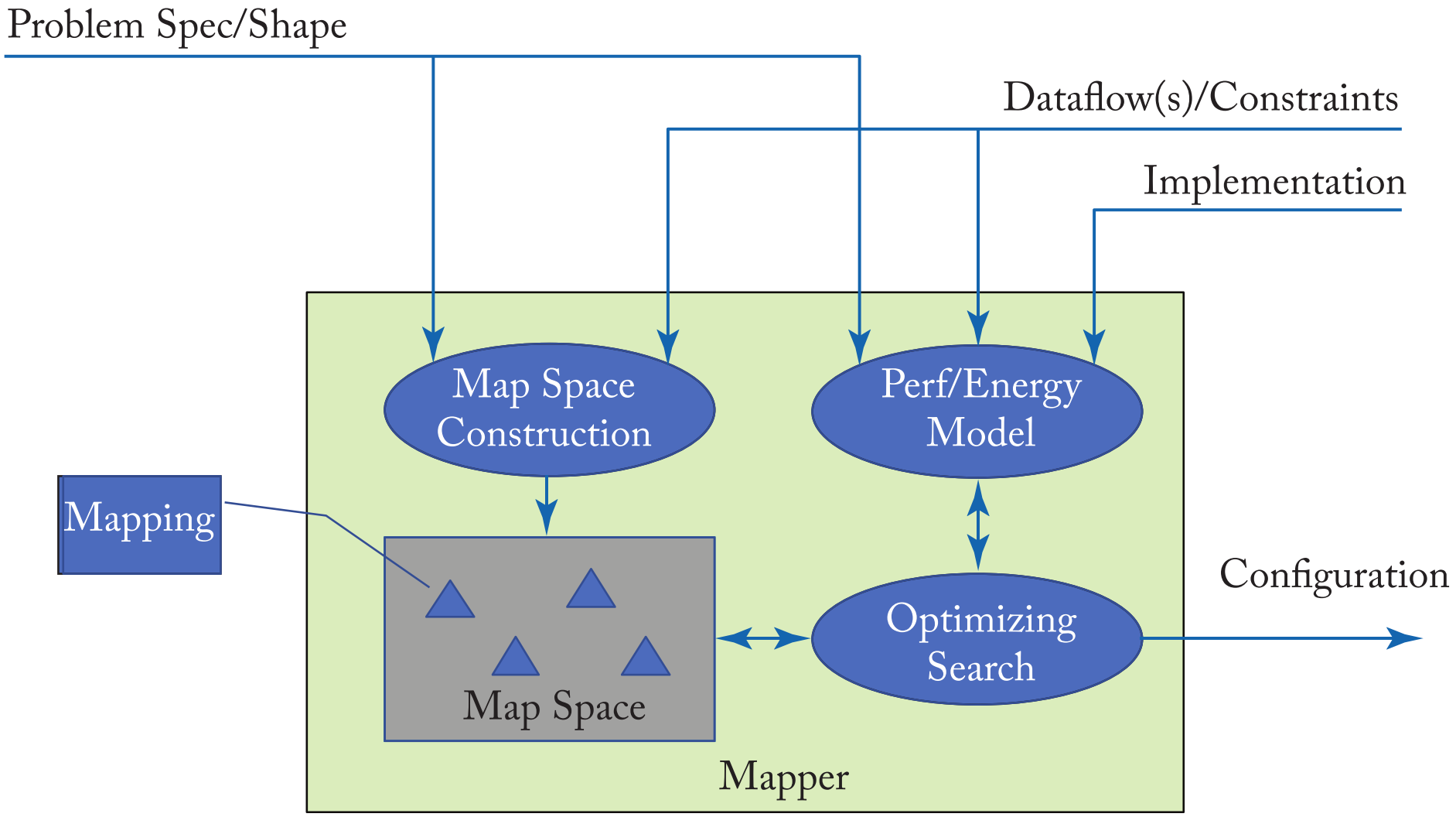
- Compiler
  - Optimized binary program that can run on the processor.
- Mapper of AI Processor
  - Configuration for the AI processor that embodies that optimal mapping
  - Consist of a set of values
    - Will be loaded into configuration registers of AI processor
    - Control the operation of the accelerator
    - Read/write access patterns to all the buffers
    - NoC data transfer patterns
    - Sequence of computations at the PEs

Outline



- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

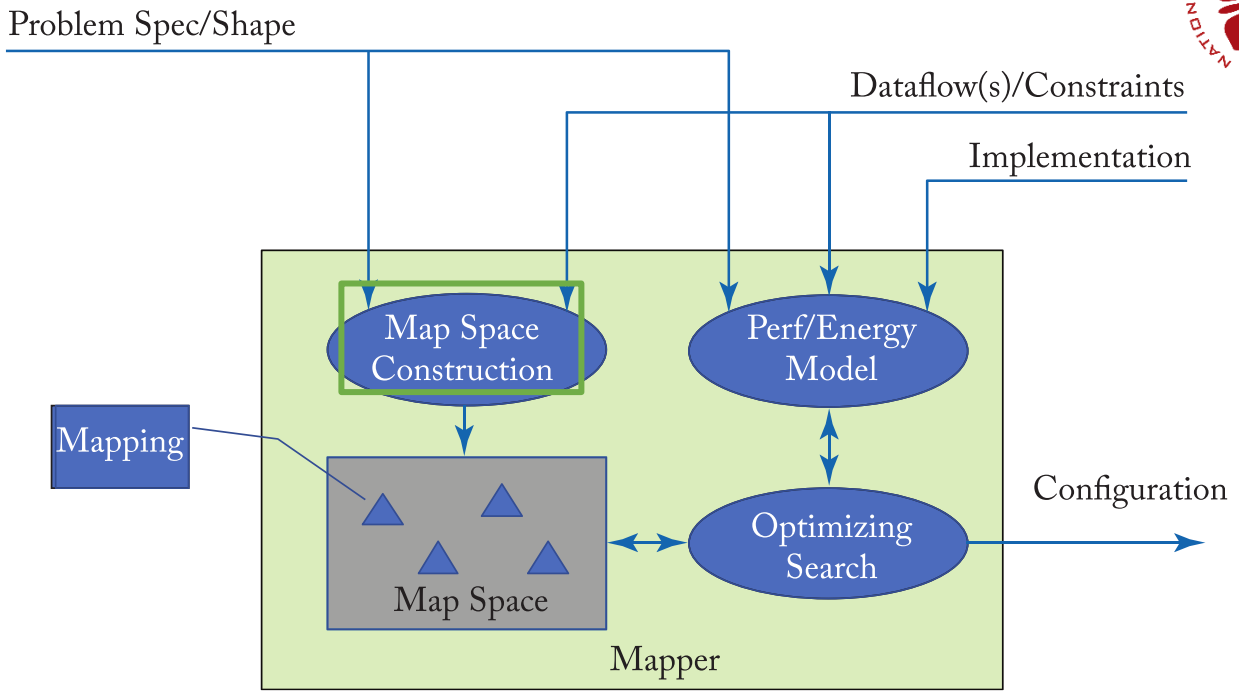
Mapper


Problem Spec/Shape
Dataflow(s)/Constraints
Implementation
Map Space Construction
Perf/Energy Model
Mapping
Map Space
Optimizing Search
Configuration
Mapper

Mapping Steps for AI Processor

1. **Creating a representation of the map space** for the given workload and DNN accelerator
2. **Searching that space for optimal mappings** based on the metrics of interest
   - Performance
   - Energy efficiency
   - Combination
3. **Evaluate the metrics** for mappings proposed by the search
   - Back to 2 until optimal or best trade-off mapping found
4. **Selecting a desired mapping** creating a configuration for the accelerator that will result in an execution with that mapping

Map Space Construction



- Two input
  - DNN problem specification/shape
  - Dataflow/Constraints
- Objective
  - Create map space
- Map space
  - An enumeration of all the possible legal mappings for the given problem shape and architecture
- It is possible to **exhaustively enumerate** the mappings in the map space for simple design
  - **Not generally feasible**

Abstraction for Map Space

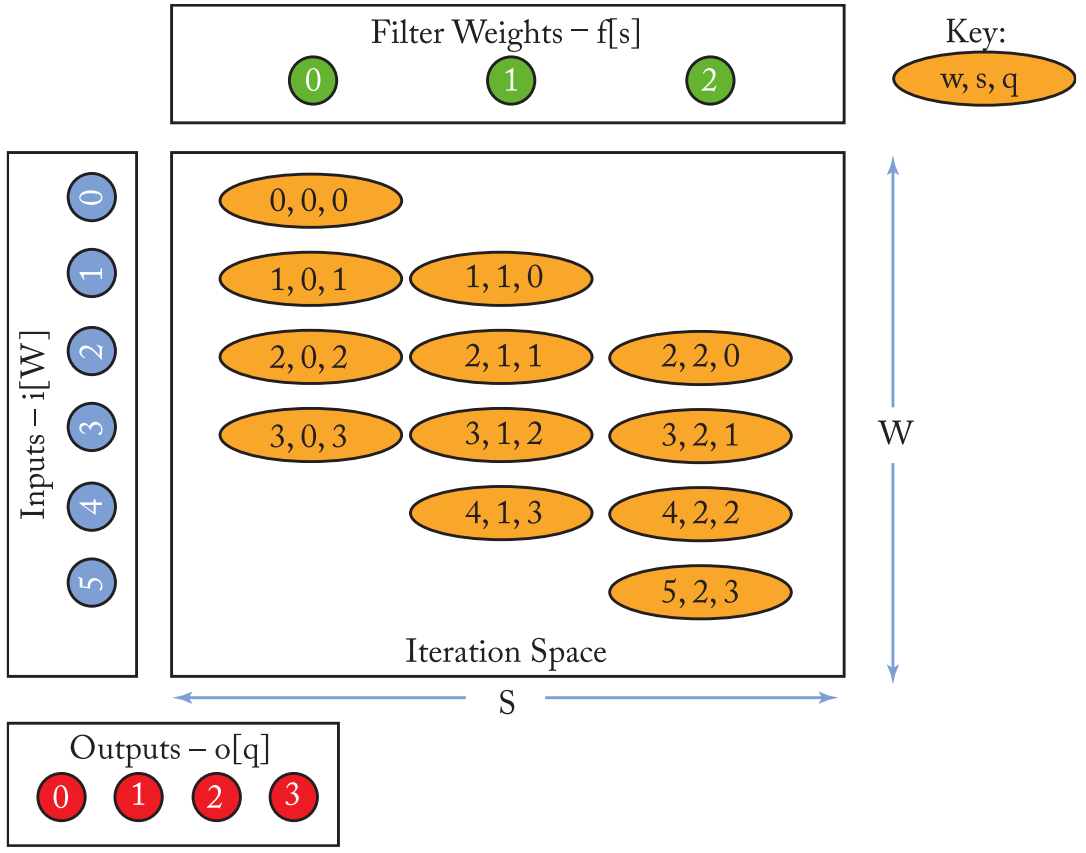
- Iteration space
  - A concept commonly used in the compiler community
- Example for 1-D CNN

Problem specification - a single loop nest

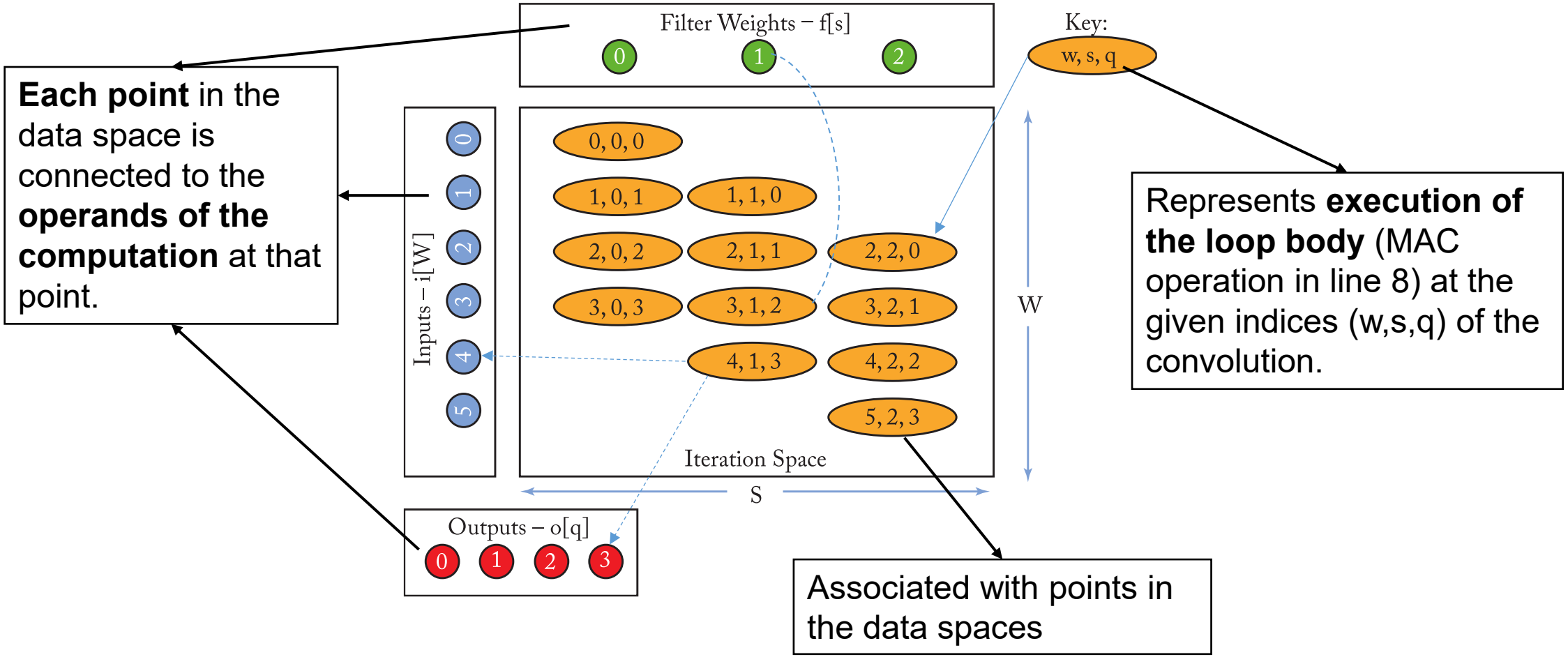
```
i = Array(W) # Input feature map
f = Array(S) # Filter weights
o = Array(Q) # Output feature map

for q in [0, Q):
    for s in [0, S):
        w = q + s
        o[q] += i[w] * f[s]
```

Iteration space - Multi-dimensional space with one point for each execution of the body of the loop nest



Iteration Space



Mapping - Visit Order of Iteration Space



- **A specific mapping** = a specific **visit order** through the iteration space
- **Execution** of the problem requires a **visit to each point in the iteration space**
  - Include Dataflow/Constraints
- For convolution
  - Operations can be performed in any order
    - There are no dependencies
    - Sum of products is commutative
  - Mathematically, could visit the points in the iteration space in any order

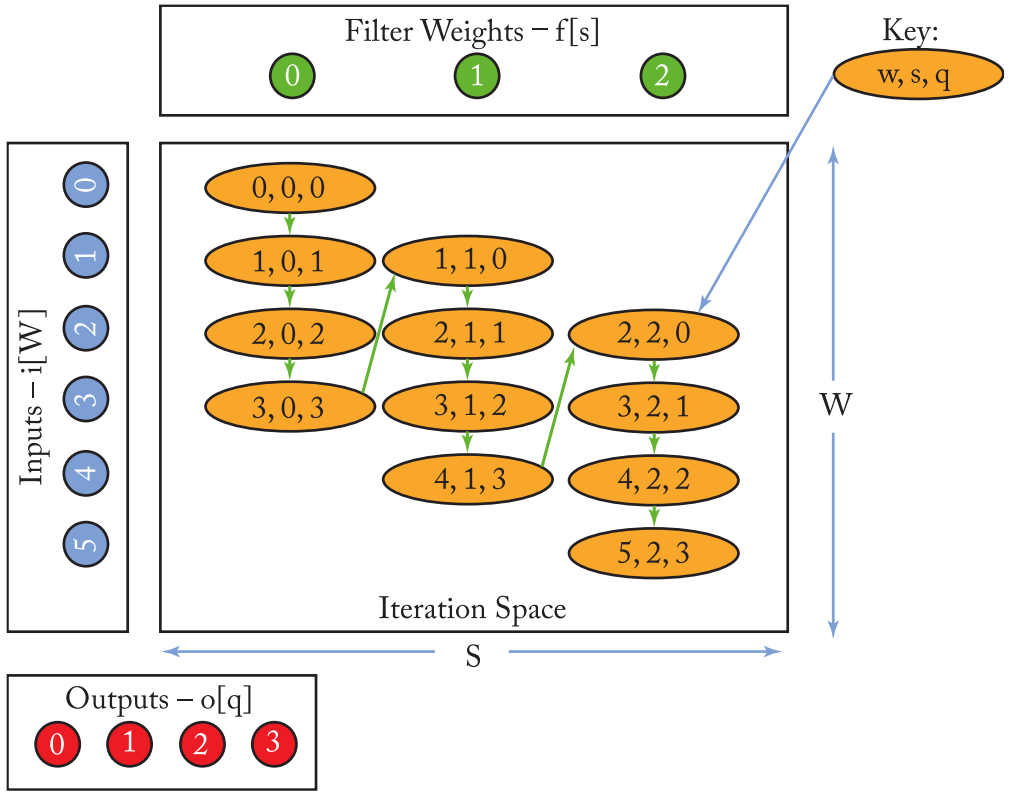
The visit order is not determined by the specification that defined the layer's computation, but by intrinsic characteristics of the desired computation

Weight-Stationary Visit Order(Mapping)

- Many useful traversal orders can be specified by the loop nests

```
i = Array(W) # Input feature map
f = Array(S) # Filter weights
o = Array(Q) # Output feature map

for q in [0, Q):
    for s in [0, S):
        w = q + s
        o[q] += i[w] * f[s]
```

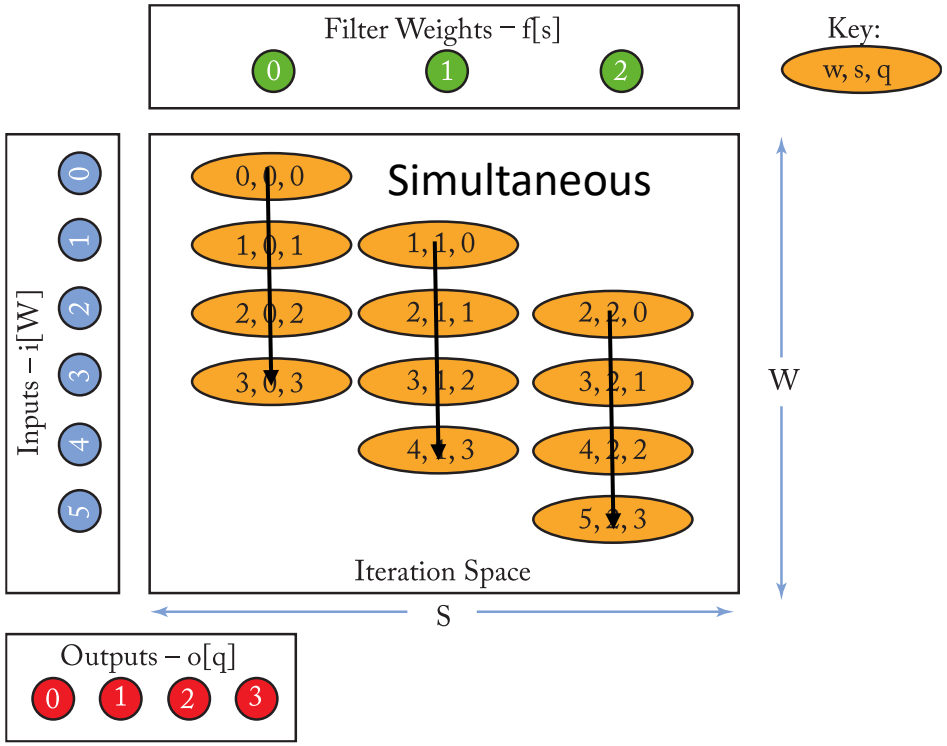


Loop Nest and Visit Order

- Loop nest **roughly defines the visit order**
  - **for** loops result in **traversal order**
  - **spatial_for** loops result in **multiple simultaneous traversals**
- However, in generally, traversal sequences that **do not precisely follow the loop nest pattern**
  - More complex visit order might be created through the use of more complex computations on the for loop variables
  - Create references to the data tensors
  - Result in a more diverse set of reference patterns to the data tensors
- **The benefits and hardware costs of such traversal orders are still largely an open question**

Parallelism in Iteration space

- **Simultaneous traversal paths** through the nodes in the iteration space
- spatial_for
  - Naturally result in such multiple traversals



Imperfect Factorization of Problem Shape Parameter

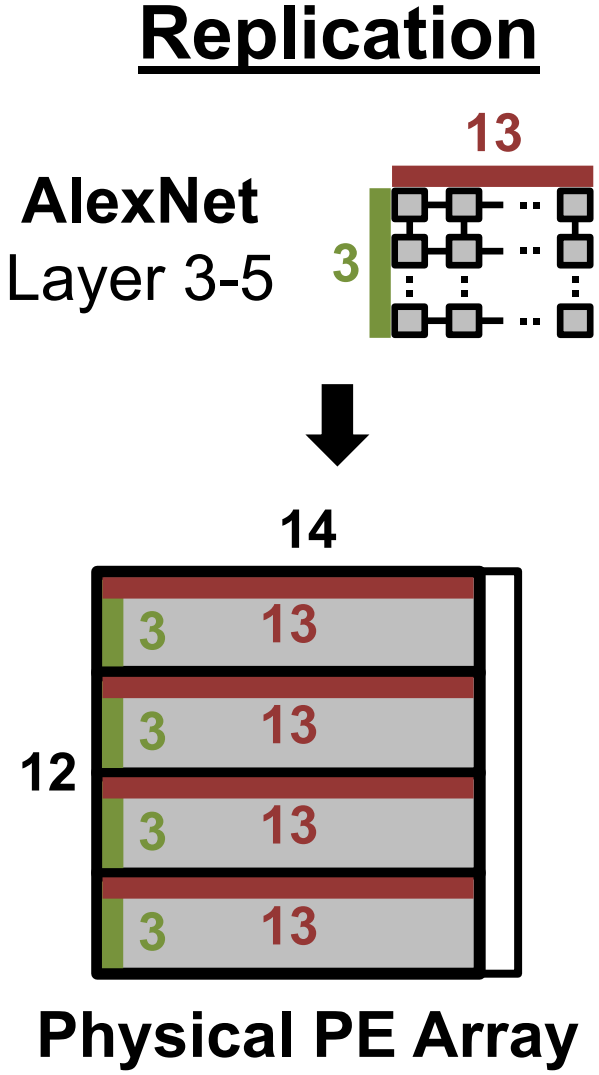


- Number of parallel traversals might not equal the number of PE
- Lengths of the traversals might not be all be equal
- May result in underutilization of hardware resources
  - Lower performance
- Important job for Mapper
  - **Decide when an imperfect factorization is the right choice to optimize the mapping**
    - Functionality
    - Despite lower utilization

Replication Mapping Pattern

- Number of parallel traversals might not equal the number of PE
  - Replication Mapping Pattern

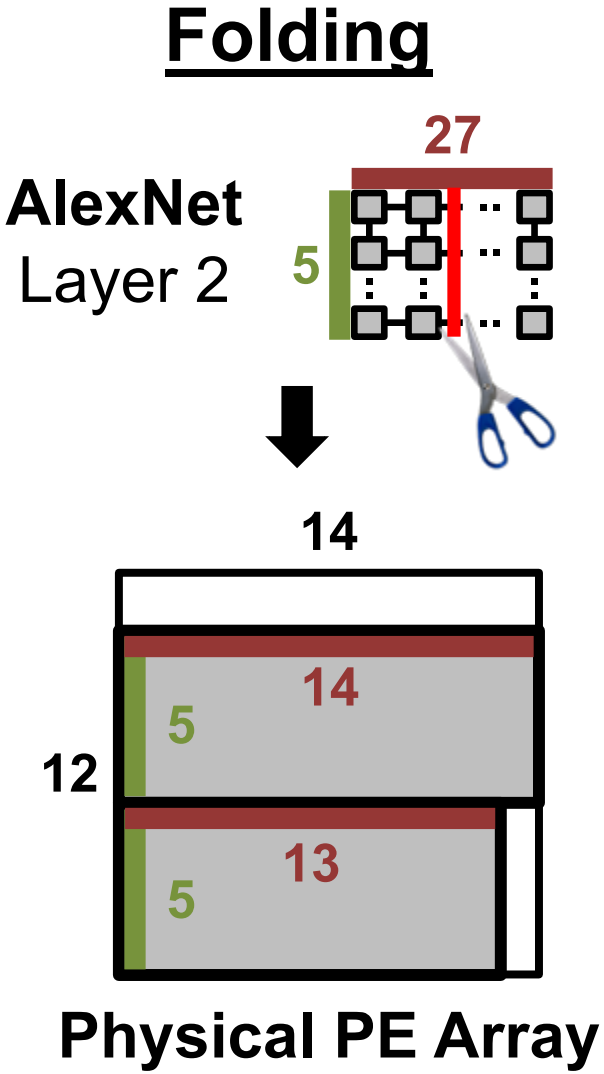
- Input shape 13x13x384
- Kernel shape 3x3x192



Folding Mapping Pattern

- Lengths of the traversals might not be all be equal
  - Folding Mapping Pattern

- Input shape 27x27x96
- Kernel shape 5x5x48



A More Complex Iteration Space

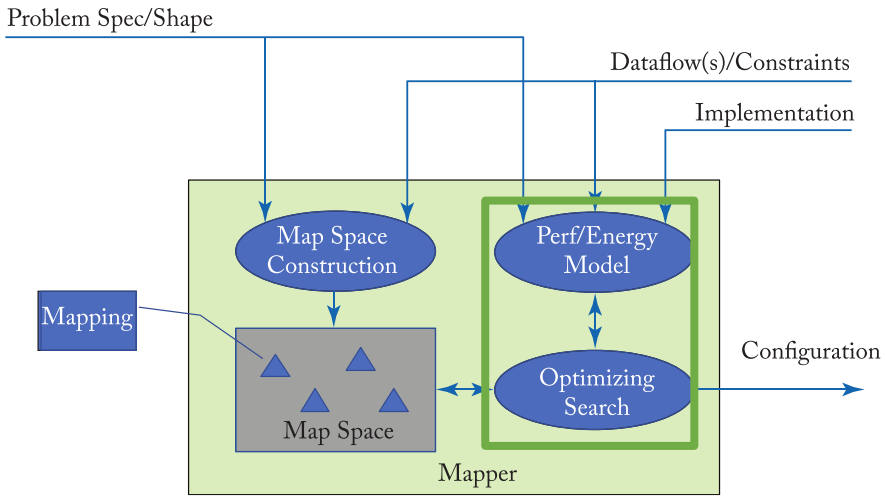


- AI Processor with **Hierarchy of storage levels** and **interconnect network**
  - Result in **hierarchical structure with iteration space points**
  - Corresponding to the data movement actions and computation throughout the hierarchy
- Considered in **combination with the existence of simultaneous traversal paths**
  - Introducing opportunities for **inter-layer** and **intra-layer** data transfers
  - Execution of those distinct paths to be *skewed* in time

Consideration of these issues is an area of current research!

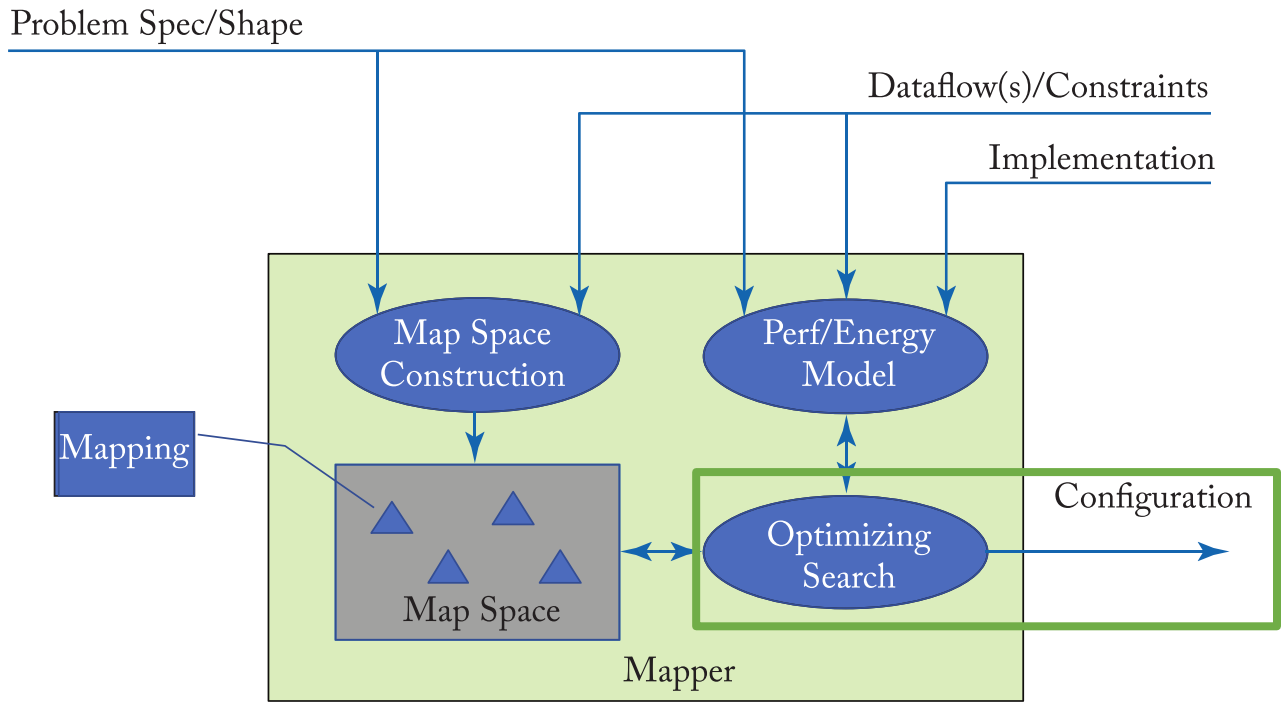
Mapper Search

- **Picking a mapping from the map space**
- Determining **how well that mapping** meets the optimization objectives
  - Typically conducted by a **performance and/or energy model**
  - Require
    - DNN problem specification/shape
    - Dataflow(s)/constraints of the DNN accelerator
    - Micro-architectural implementation details
  - **Project the performance and/or energy consumption for that mapping**
  - Cycle-level simulation or simple analytic models
    - Accelergy - architecture-level energy estimation methodology
    - Eyexam - analytic model for energy projections



Configuration Generation

- After finding an optimal mapping, **convert mappiong into a configuration**



Outline



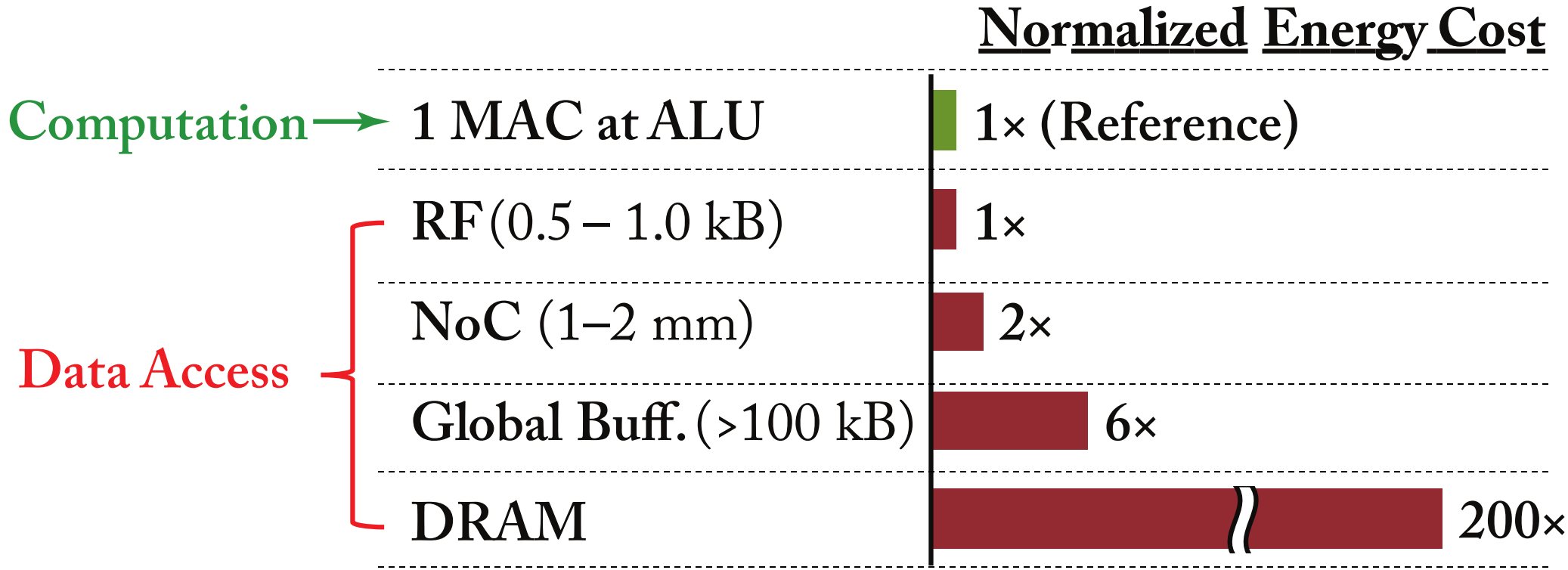
- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

Analytical Model for Energy

- A model capable of **projecting the energy consumption/performance** of a particular mapping **on an AI Processor**
  - Include data Movement Hierarchy and analysis methodology
- Factors for MAC operation energy cost
  - Input data reuse
  - Partial sum accumulation scheduling
  - Fetching data from different storage elements
    - Different energy costs
- Energy-efficient dataflow
  - Perform most data accesses using the data movement paths with lower energy cost
- Search for the **optimal mapping** for a dataflow that results in the **highest energy efficiency** for a given DNN layer shape

Data Movement Hierarchy

- Assuming four levels of storage hierarchy

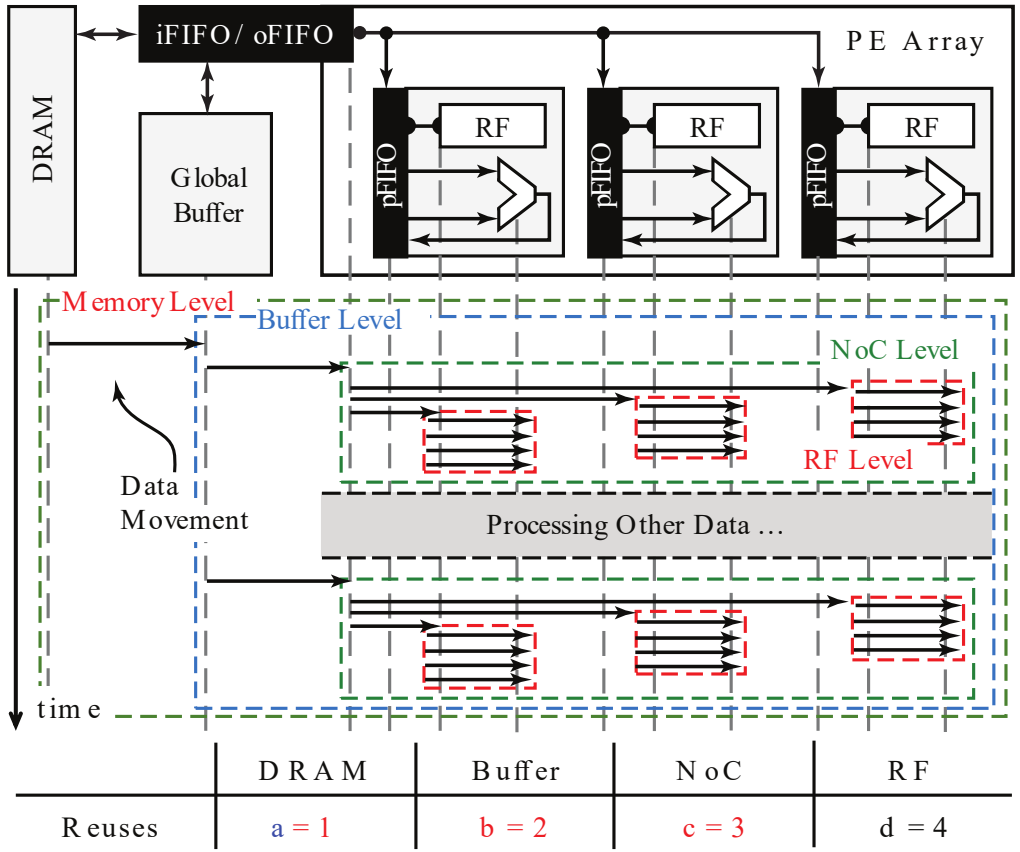


Analysis Methodology

1. Counting the number of accesses at each level for
   - Input data access energy cost
     - Including filter weights and input activations
   - Partial sum accumulation energy cost
2. Weighting the accesses at each level
3. Combining the results from input data and partial sums

Input Data Access Energy Cost

$$a \times EC(DRAM) + ab \times EC(global\ buffer) + abc \times EC(array) + abcd \times EC(RF)$$



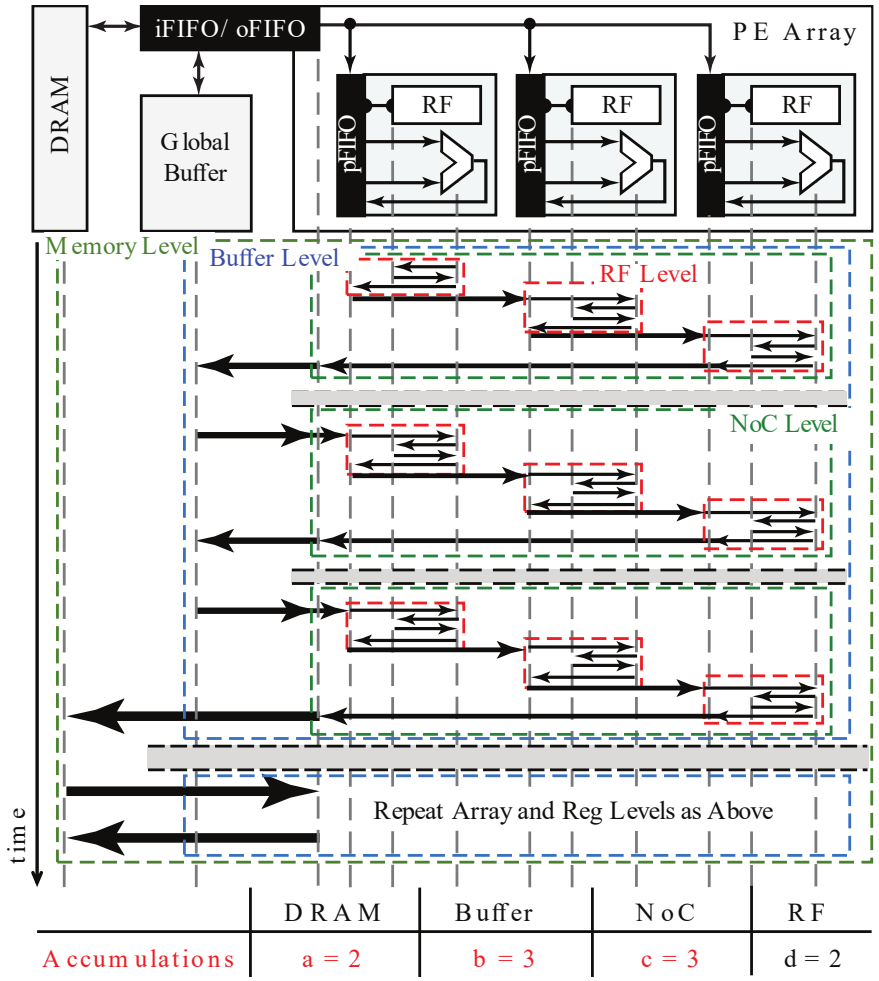
$EC(\cdot)$: *Energy Cost*

total number of reuse = 24

Reuse at each level - the number of times each data value is read from this level to its lower-cost levels during its lifetime

Partial Sum Accumulation Energy Cost

$$(2a-1)\times EC(DRAM)+2a(b-1)\times EC(global\ buffer)+2ab(c-1)\times EC(array)+2abc(d-1)\times EC(RF)$$



$EC(\cdot)$: *Energy Cost*

"2" accounts for both reads and writes

bias term is ignored

total number of accumulations = 36

Obtaining The Reuse Parameters

- 1-D Convolution example

```
Input Fmaps:   i[W]
Filter Weights: f[S]
Output Fmaps:  o[Q]
for (q2=0; q2<Q2; q2++) {
    for(s2=0; s2<S2; s2++) {
        spatial_for (q1=0; q1<Q1; q1++) {
            spatial_for (s1=0; s1<S1; s1++) {
                for(q0=0; q0<Q0; q0++) {
                    for(s0=0; s0<S0; s0++) {
                        o[q2*Q1*Q0+q1*Q0+q0] +=
                            i[q2*Q1*Q0+q1*Q0+q0 + s2*S1*S0+s1*S0+s0] ×
                            f[s2*S1*S0+s1*S0+s0];
                    }
                }
            }
        }
    }
}
```

Data Type	Reuse Parameters			
	a	b	c	d
Input activation	$S/(S0 \times S1 \times S2)$	$S2$	$S1$	$S0$
Weight	$Q/(Q0 \times Q1 \times Q2)$	$Q2$	$Q1$	$Q0$
Partial sums	$S/(S0 \times S1 \times S2)$	$S2$	$S1$	$S0$

Pitfalls

- Comparing different designs by the overall performance
  - Affected by many non-architectural factors
    - System setup
    - Technology differences

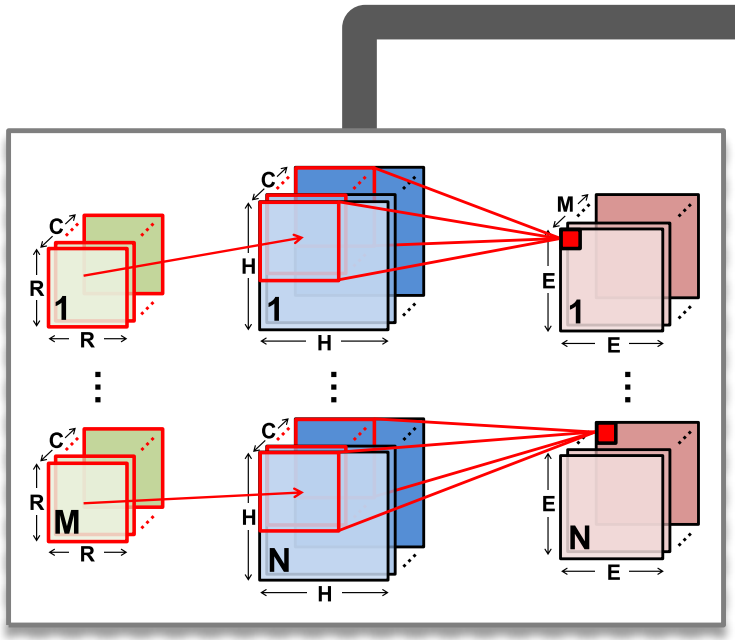
Outline



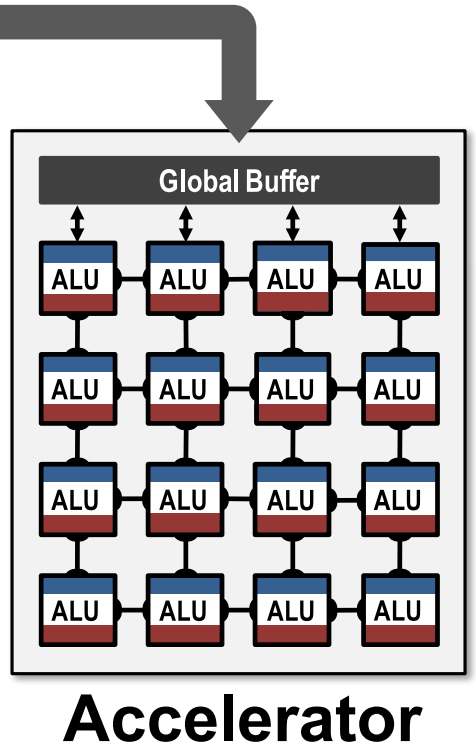
- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

Hardware Performance Analysis

- How to quickly estimate the performance of a DNN accelerator across various DNN workloads?

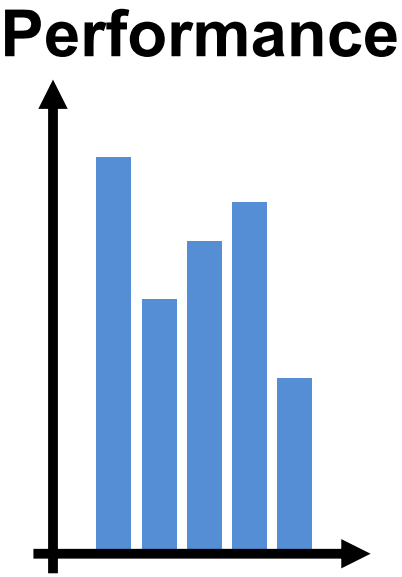


DNN Workloads



Accelerator





Evaluating Performance

- Main factors that affect performance
  - The number of active PEs
    - Due to the mapping as constrained by the dataflow
  - The utilization of active PEs
    - Percentage of active cycles for the PE
    - Based on whether the NoC has sufficient bandwidth to deliver data to PEs to keep them active

$$utilization\ of\ the\ PE\ array = number\ of\ active\ PEs \times utilization\ of\ active\ PEs$$

Performance Analysis of 1D CONV



- Start with infinite processing parallelism, storage capacity and data bandwidth
- Each Step, certain constraints will be added to reflect changes in the assumptions on the architecture or workload

```
Input Fmaps:   i[W]
Filter Weights: f[S]
Output Fmaps:  o[Q]
for (q2=0; q2<Q2; q2++) {
    for(s2=0; s2<S2; s2++) {
        spatial_for (q1=0; q1<Q1; q1++) {
            spatial_for (s1=0; s1<S1; s1++) {
                for(q0=0; q0<Q0; q0++) {
                    for(s0=0; s0<S0; s0++) {
                        o[q2*Q1*Q0+q1*Q0+q0] +=  i[q2*Q1*Q0+q1*Q0+q0 + s2*S1*S0+s1*S0+s0] × f[s2*S1*S0+s1*S0+s0];
                    }
                }
            }
        }
    }
}
```

Step 1 - Layer Shape and Size

- **Look at the impact of the workload constraint**
- Specifically the layer shape
  - S, W, Q
- Assuming unbounded values for S1 and Q1
  - Since no architectural constraints
  - S1=S, Q1=Q, Q2=Q0=1, S2=S0=1
  - All spatial processing
  - No temporal processing
- Performance upper bound is determined by the **finite size of the workload**
  - $Q \times S$ – number of MACs in this layer

Step 2 - Dataflow

- Define the dataflow
- **Examine the impact of this architectural constraint**
- Configure the example loop nest into a weight-stationary dataflow
  - Q1=1, Q0=Q, S1=S, S0=1
  - Each PE stores one weight, reused Q0 times
  - # of PE = # of weights
  - Restrict the absolute maximum amount of reuse for weights at the PE
    - Forced serialization of Q0=Q
- Performance upper bound determined by **maximum parallelism of the dataflow**
  - $Q \times S \rightarrow S$

Step 3 - Number of PEs

- Define a finite number of PEs
- **Look at the impact of this architectural constraint**
- Constrain $S1 \leq \#\ of\ PEs$
- Two cases that actual performance is less than the peak performance
  - **Spatial mapping fragmentation**
    - $S\ < (\#\ of\ PEs)$
    - Some PEs are completely idle throughout the entire period of processing
  - **Temporal mapping fragmentation**
    - $S > (\#\ of\ PEs), and\ (S\ mod (\#\ of\ PEs)) \neq 0$
    - E.g. S=7, S1=4, # of PEs = 4
      - Takes two cycle, one of 4PEs will be 50% active

Step 4 - Physical Dimensions of PE Array

- **Consider the physical dimensions of the PE array**
  - Arranging 12 PEs as 3x4, 2x6, or 4x3, etc.
- Spatial partitioning is constrained per dimension
  - Cause additional performance loss
- This architectural constraint further **reduces the number of active PEs**.

- E.g.
  - Assuming Q1 mapped to width of the 2-D array, S1 is mapped to the height of the 2-D array
  - $If\ Q1 < (width\ of\ the\ 2-D\ array)\ or\ S1 < (height\ of\ the\ 2-D\ array)$
    - Not all PEs will be utilized

Step 5 - Storage Capacity

- **Consider the impact of making the buffer storage finite**
- Architectural constraint **reduce the number of active PEs**

- E.g.
  - For the WS dataflow example, if the allocated storage for partial sums in the GLB is limited
    - Limits the number of weights that can be processed in parallel
  - Constrain how many partial sums can be stored in the GLB
    - restrict Q1 and S1

Step 6 - Data Bandwidth

- **Consider the impact of a finite bandwidth for delivering data across the different levels of the loop nest**
  - i.e. memory hierarchy
  - The amount of data that needs to be transferred for MACs limited by the bandwidth
  - Only addresses average bandwidth
- E.g.
  - Bandwidth of the RF in the PE
    - Dictates the increment speed of s0 and q0
  - Bandwidth of the NoC and GLB
    - Dictates the rate of change of s1 and q1
  - Off-chip bandwidth
    - Dictates the rate of change of s2 and q2

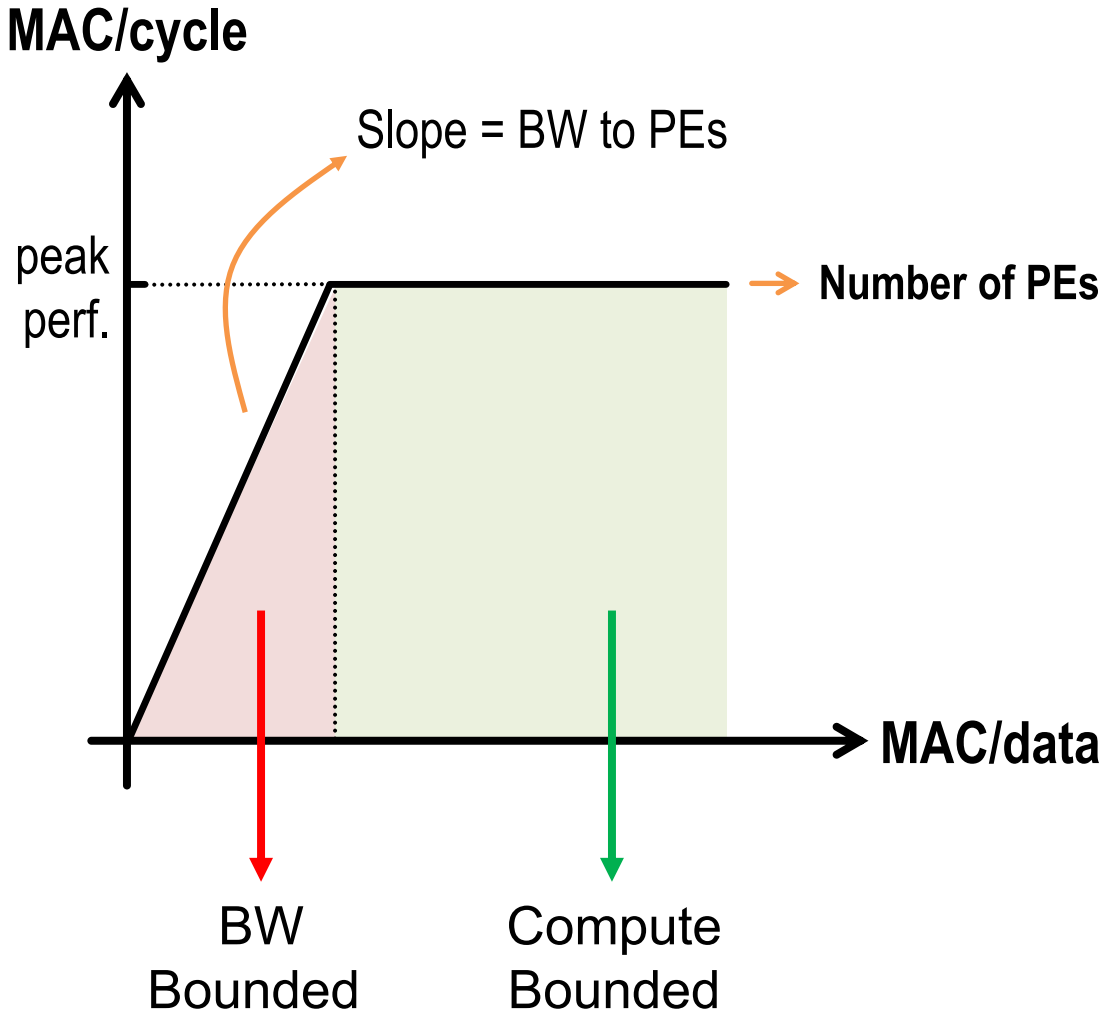
Applying Roofline Model

- Three separate rooflines for the three data types
  - Help to identify the performance bottleneck
  - Since independent NoCs are used for each data type
  - Performance upper-bound will be the **worst case** of the three rooflines
- The **leveled part** of the roofline is at the **performance bound from step 5**
- The **slanted part of** the roofline should only consider the **bandwidth to the active PEs** for each data type
  - The bandwidth should factor in the clock rate differences between processing and data delivery, since performance is measured in MACs/cycle
- The **operational intensity** of a data type is the same as its amount of **data reuse in the PE array**
  - Include both temporal reuse with the RF and the spatial reuse across PEs
  - Measured in MACs per data value (MAC/data) to normalize the differences in bit-width

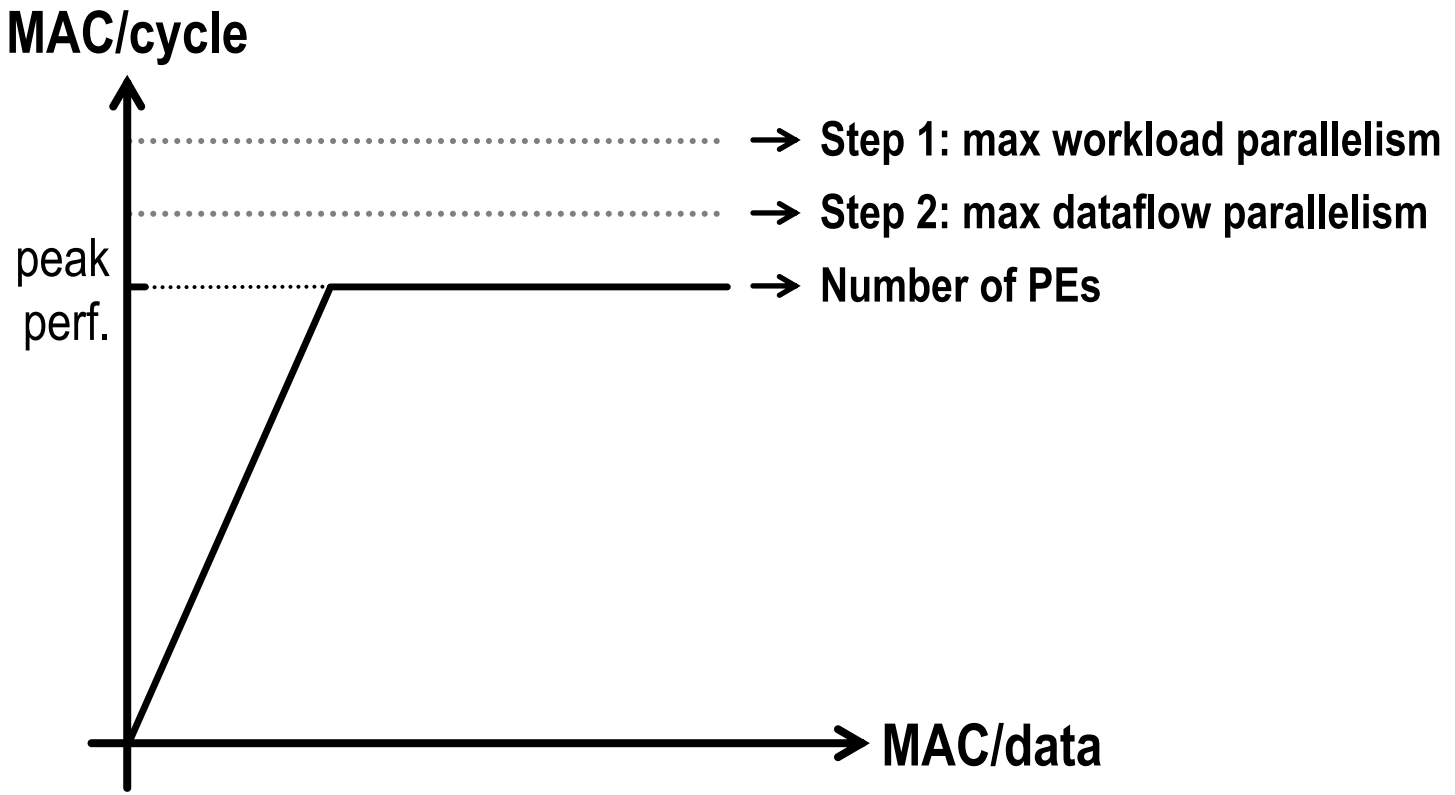
Step 7 - Varying Data Access Patterns

- **Consider the impact of bandwidth varying across time due to the dynamically changing data access patterns**
- Causes the **performance point to fall off the roofline**
- There exist many common solutions to address this issue
  - Double buffering
  - Increased bus width for the NoC
  - Etc.
- E.g.
  - For weight-stationary dataflow
    - High bandwidth for weights NoC during ramp up
      - Load the weights into the RF of the Pes
    - Low bandwidth for weights NoC during steady state
      - Weights are reused within the PE
    - Performance upper bound will be affected by
      - Ratio of time spent in ramp up versus steady state
      - Ratio of the bandwidth demand versus available bandwidth

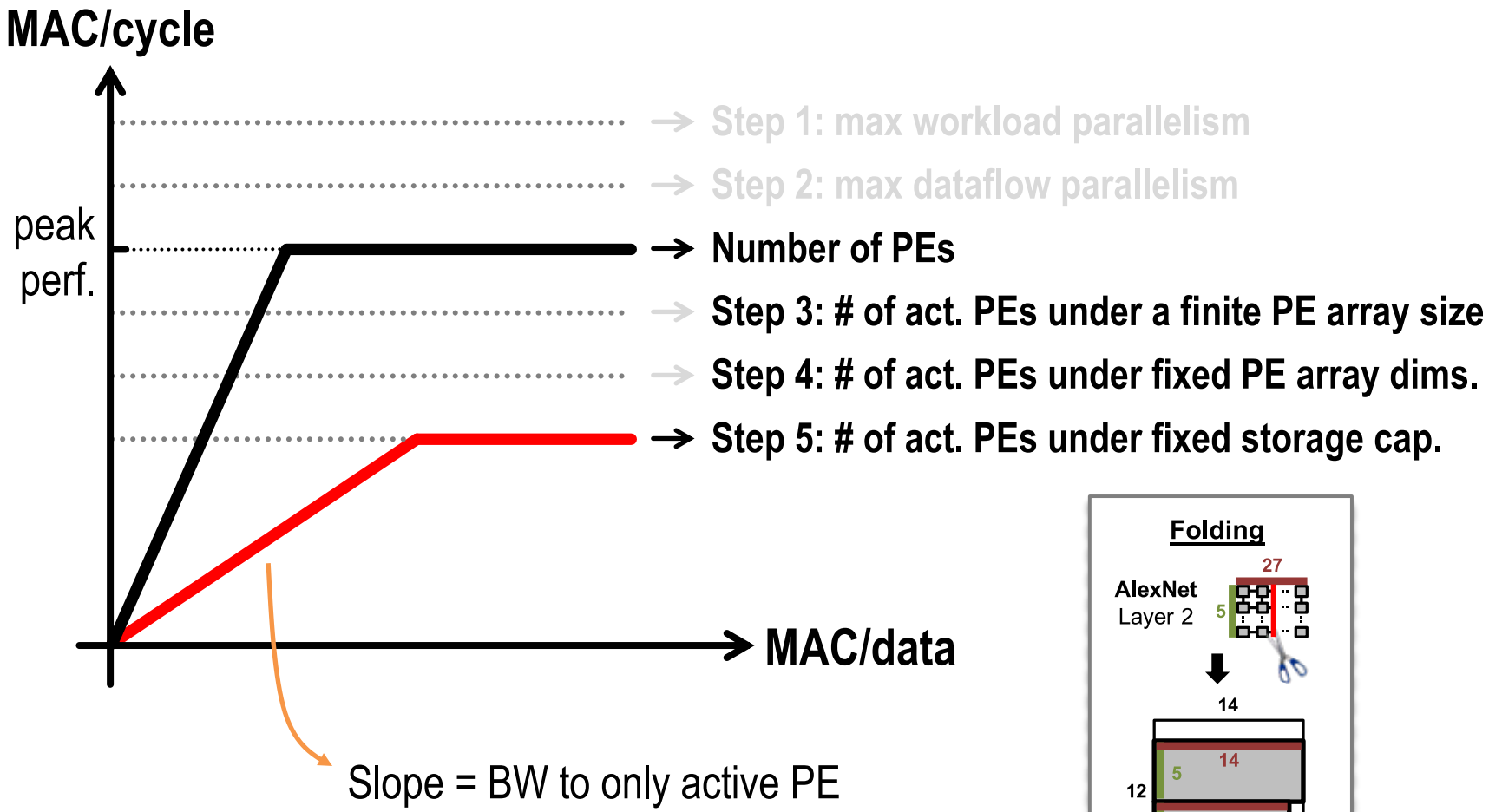
Roofline Model

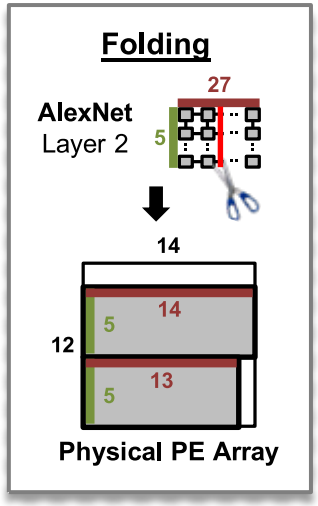


Roofline Model

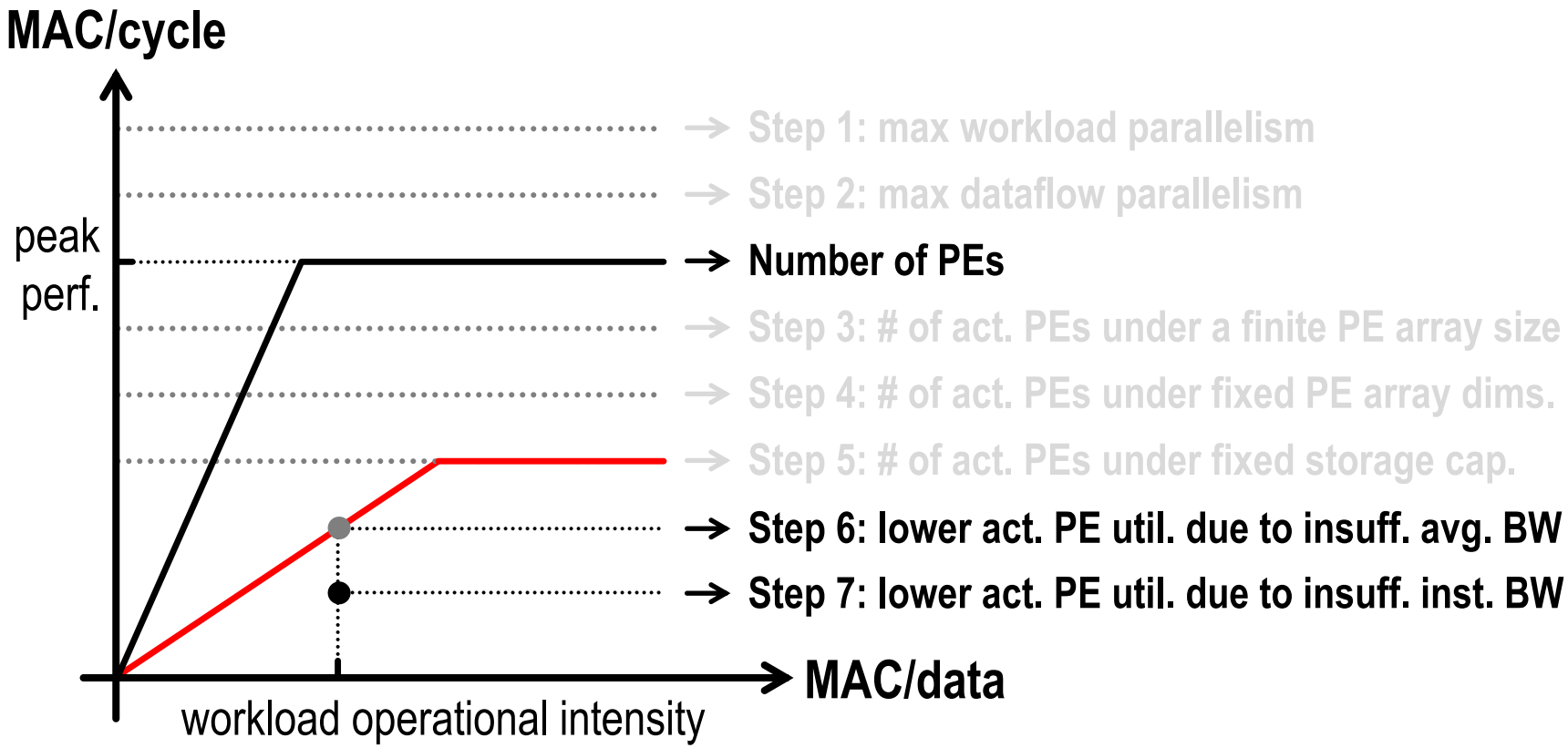


Roofline Model

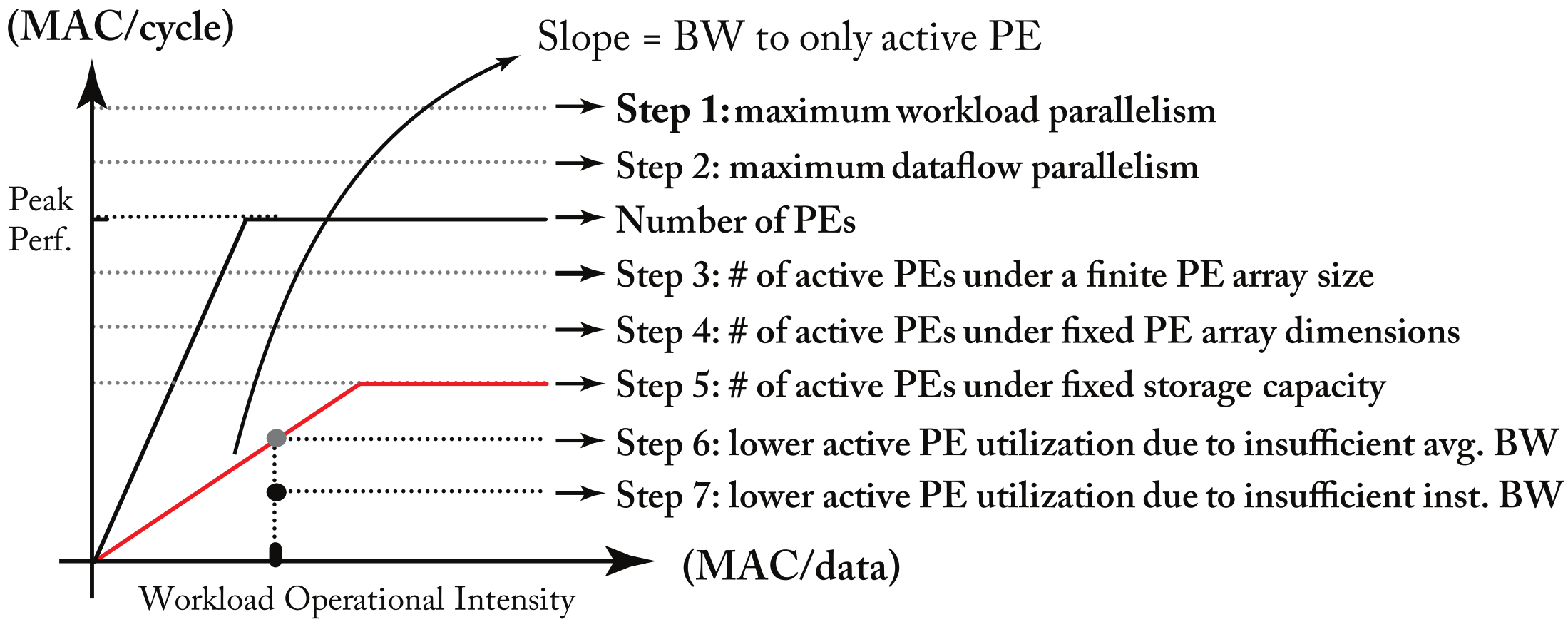




Roofline Model



Impact of Steps on Roofline



Summary

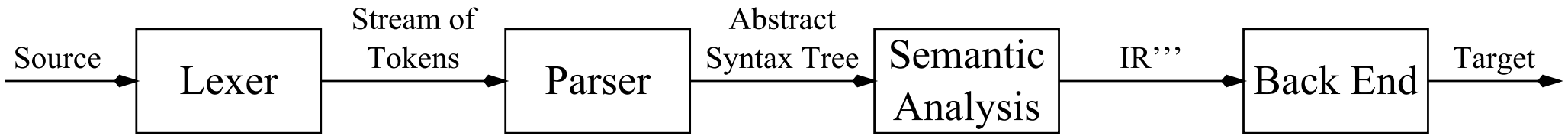
Step	Constraint	Type	New Performance Bound	Reason for Performance Loss
1	Layer size and shape	Workload	Max workload parallelism	Finite workload size
2	Dataflow loop nest	Architectural	Max dataflow parallelism	Restricted dataflow mapping space by defined by loop nest
3	Number of PEs	Architectural	Max PE parallelism	Additional restriction to mapping space due to shape fragmentation
4	Physical dimensions of PEs array	Architectural	Number of active PEs	Additional restriction to mapping space due to shape fragmentation for each dimension
5	Fixed storage capacity	Architectural	Number of active PEs	Additional restriction to mapping space due to storage of intermediate data (depends on dataflow)
6	Fixed data bandwidth	Microarchitectural	Max data bandwidth to active PEs	Insufficient average bandwidth to active PEs
7	Varying data access patterns	Microarchitectural	Actual measure performance	Insufficient instant bandwidth to active PEs

Outline



- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Optimizations
- Tools for Operation Mapping

Intermediate Representation (IR)



- **Intermediate Representation (IR)**
  - An **abstract machine language**
  - Expresses operations of target machine
  - **Not specific to any particular machine**
  - Independent of source language
- IR code generation not necessary:
  - Semantic analysis phase can generate real assembly code directly.
  - Hinders portability and modularity

Intermediate Representation (IR)



- Suppose we wish to build compilers for n source languages and m target machines.

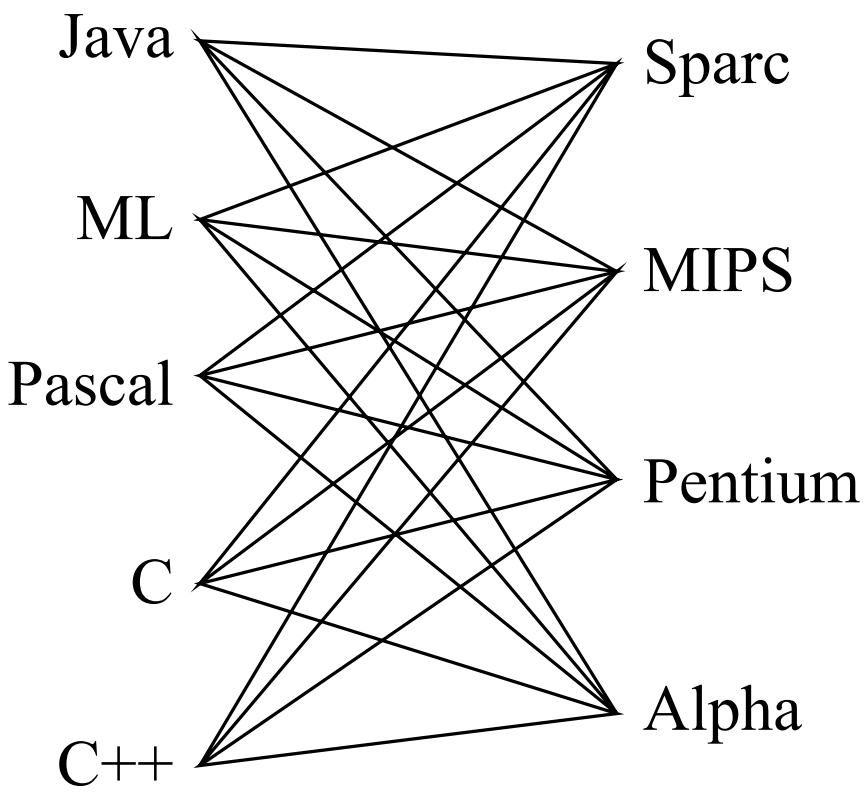
Case 1: no IR

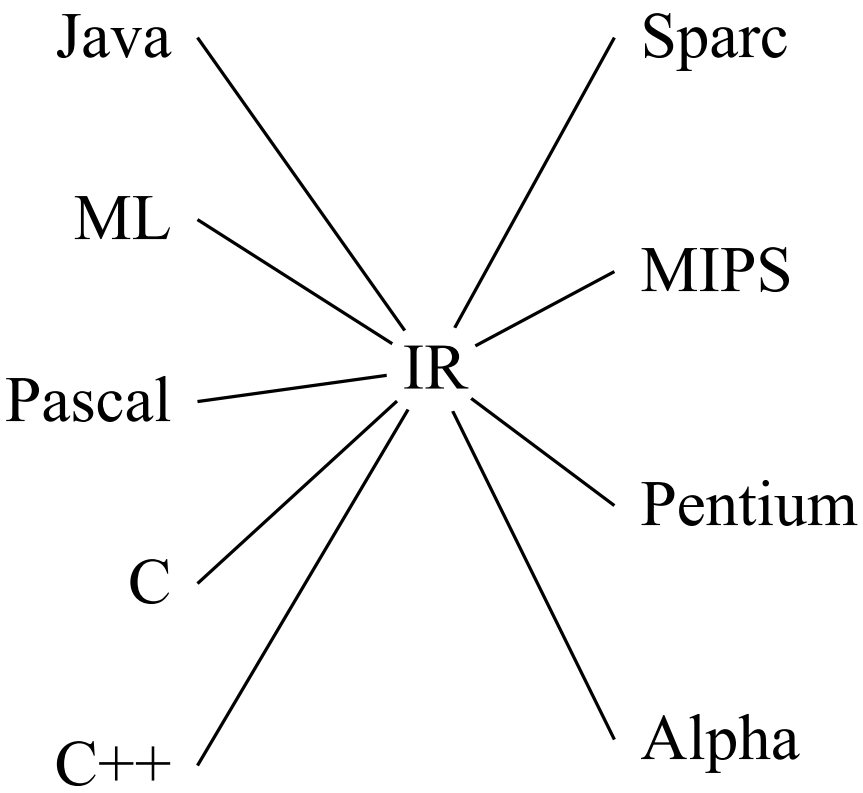
- Need separate compiler for each source language/target machine combination.
- A total of n ∗ m compilers necessary.
- Front-end becomes cluttered with machine specific details, back-end becomes cluttered with source language specific details.

Case 2: IR present

- Need just n front-ends, m back ends.

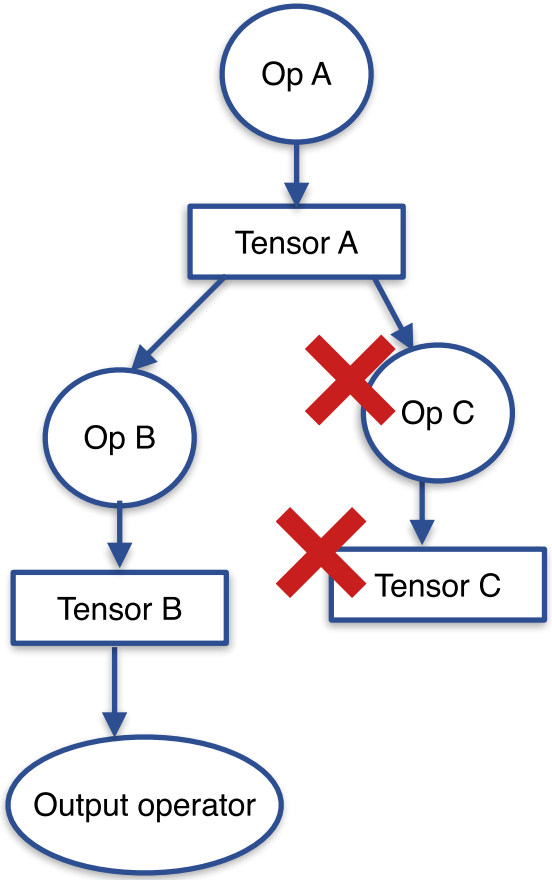
Intermediate Representation (IR)





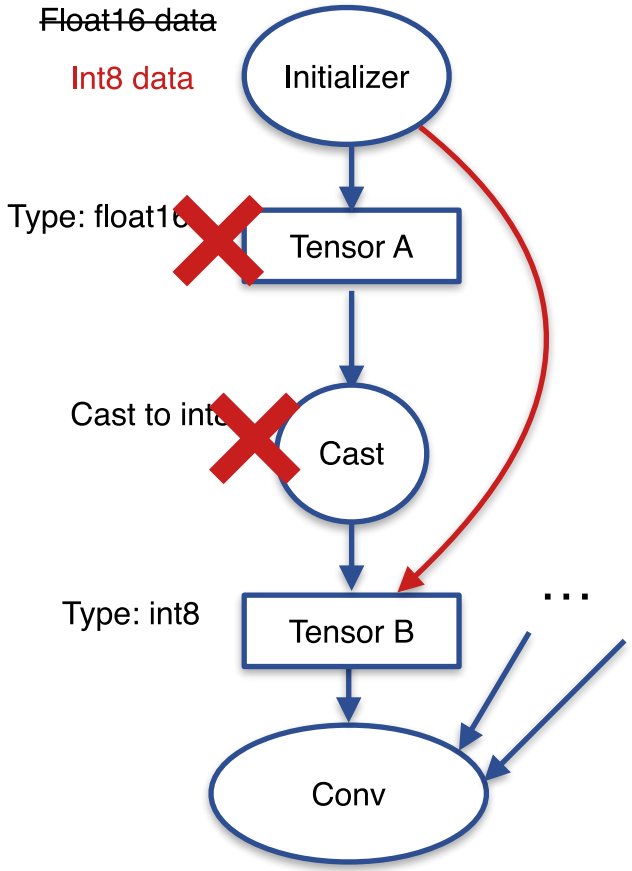
Graph Level Optimizations

- Manipulation on IR graphs
- Simplifying IR graphs
  - Node removal
    - Remove nodes whose outputs are not used at all



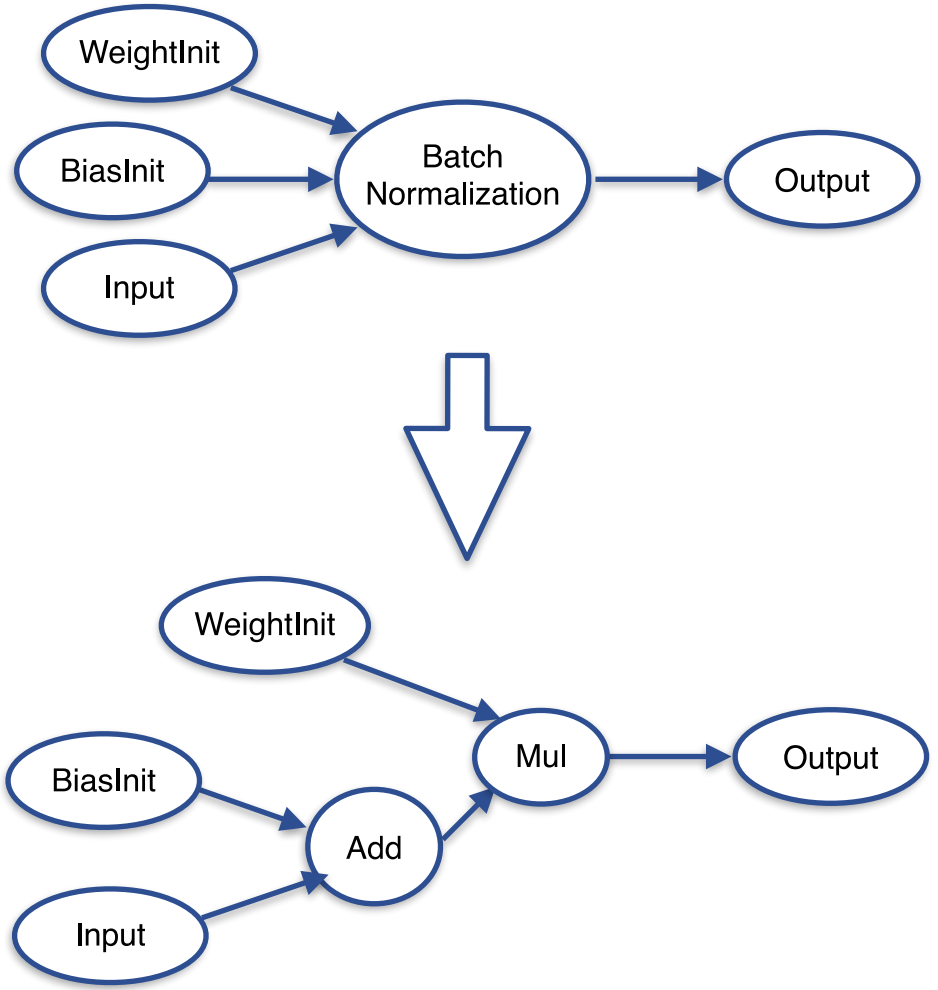
Graph Level Optimizations

- Manipulation on IR graphs
- Simplifying IR graphs
  - Layer fusion
    - Fuse Cast operators to preceding initializers



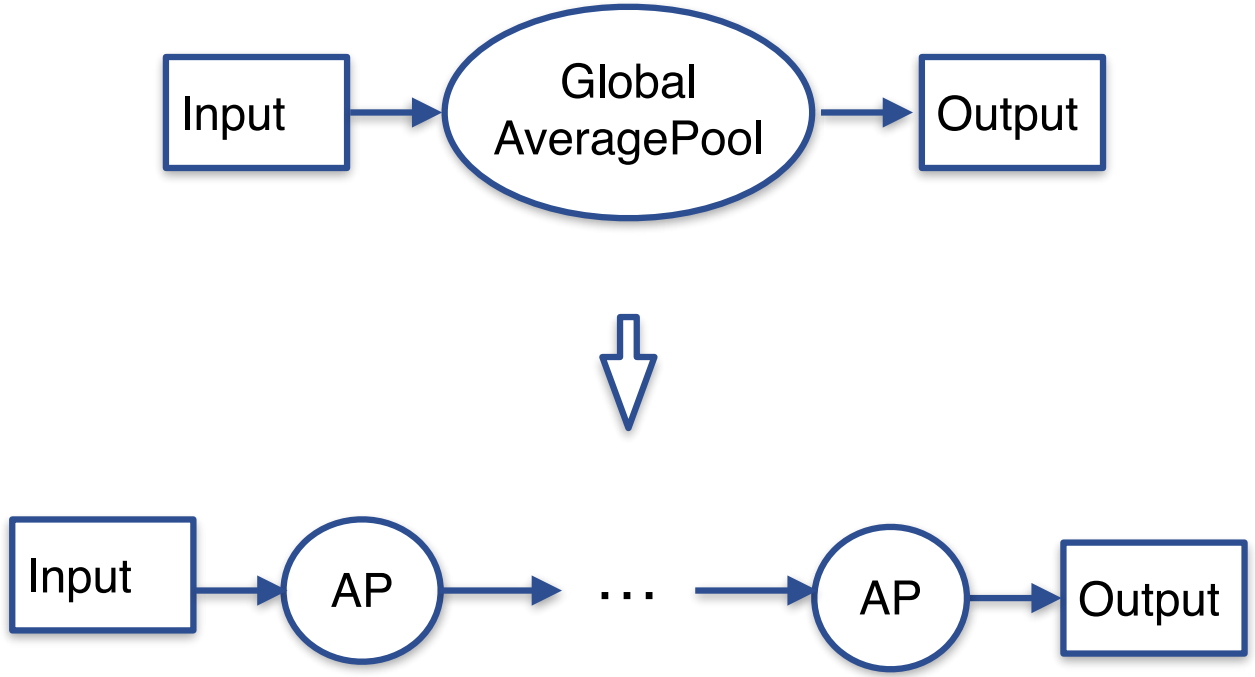
Graph Level Optimizations

- Manipulation on IR graphs
- Simplifying IR graphs
  - Operator refactoring
    - Convert BN to an Add and a Mul



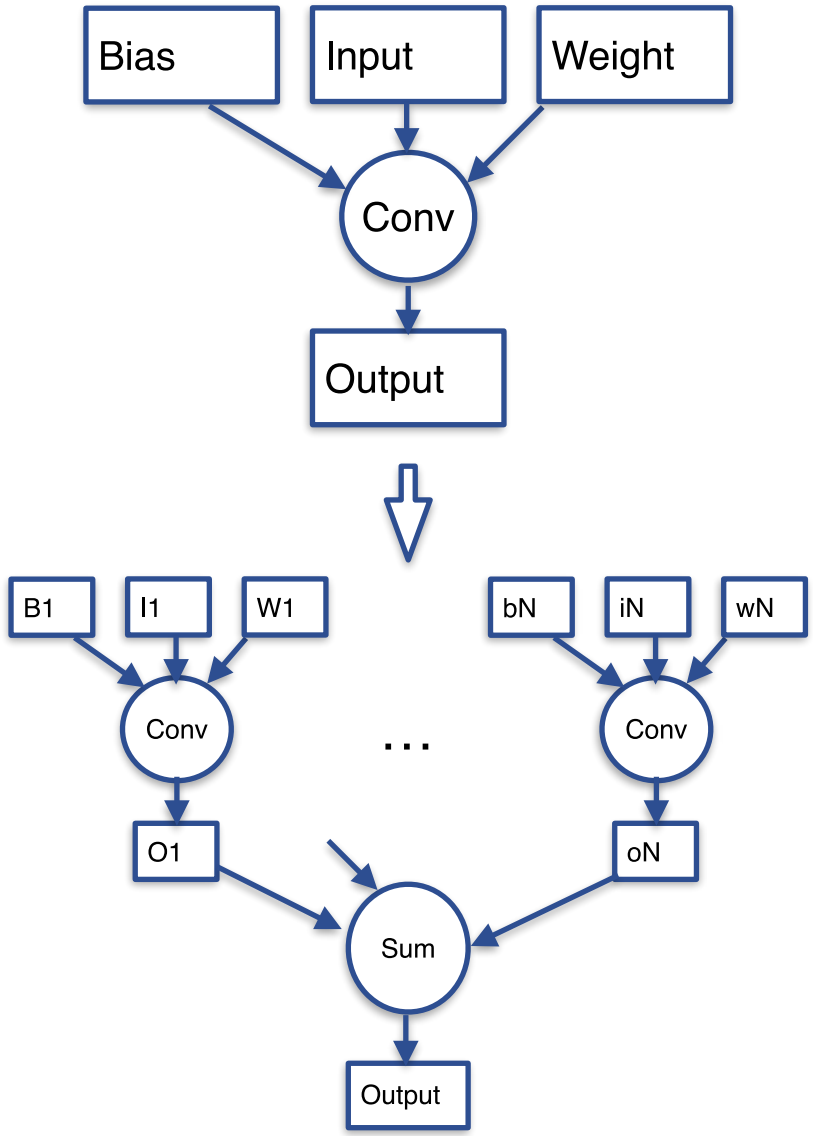
Graph Level Optimizations

- Manipulation on IR graphs
- Legalization: Convert unsupported operators into supported operators
  - NVDLA does not support GlobalAveragePooling

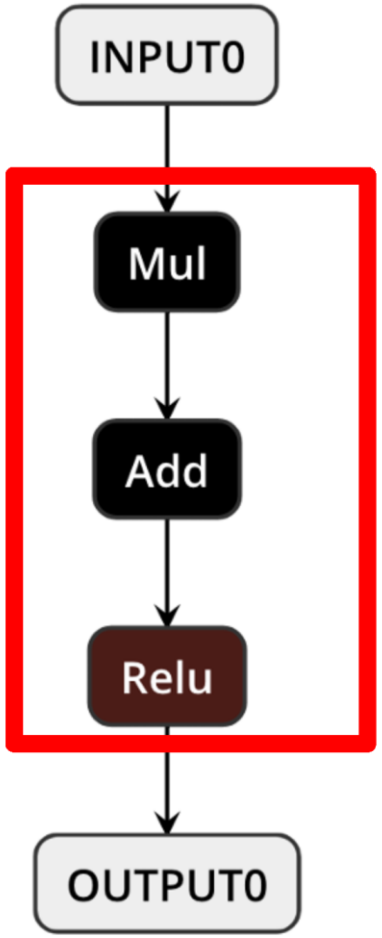


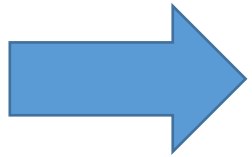
Graph Level Optimizations

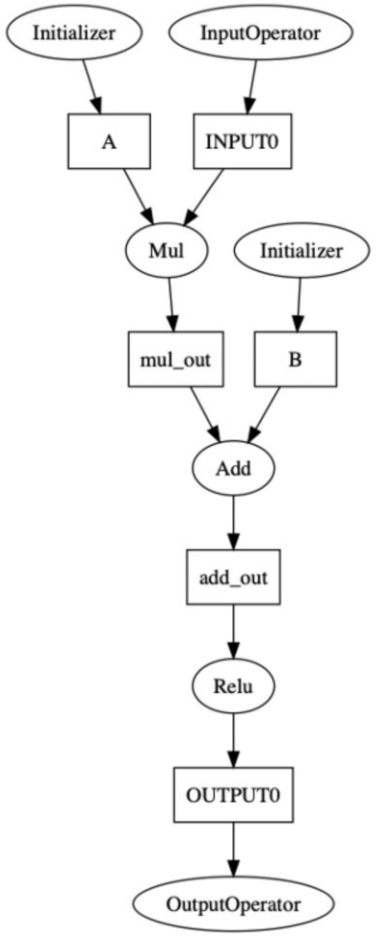
- Manipulation on IR graphs
- Meet hardware constraints
  - NVDLA's convolution buffer size is limited



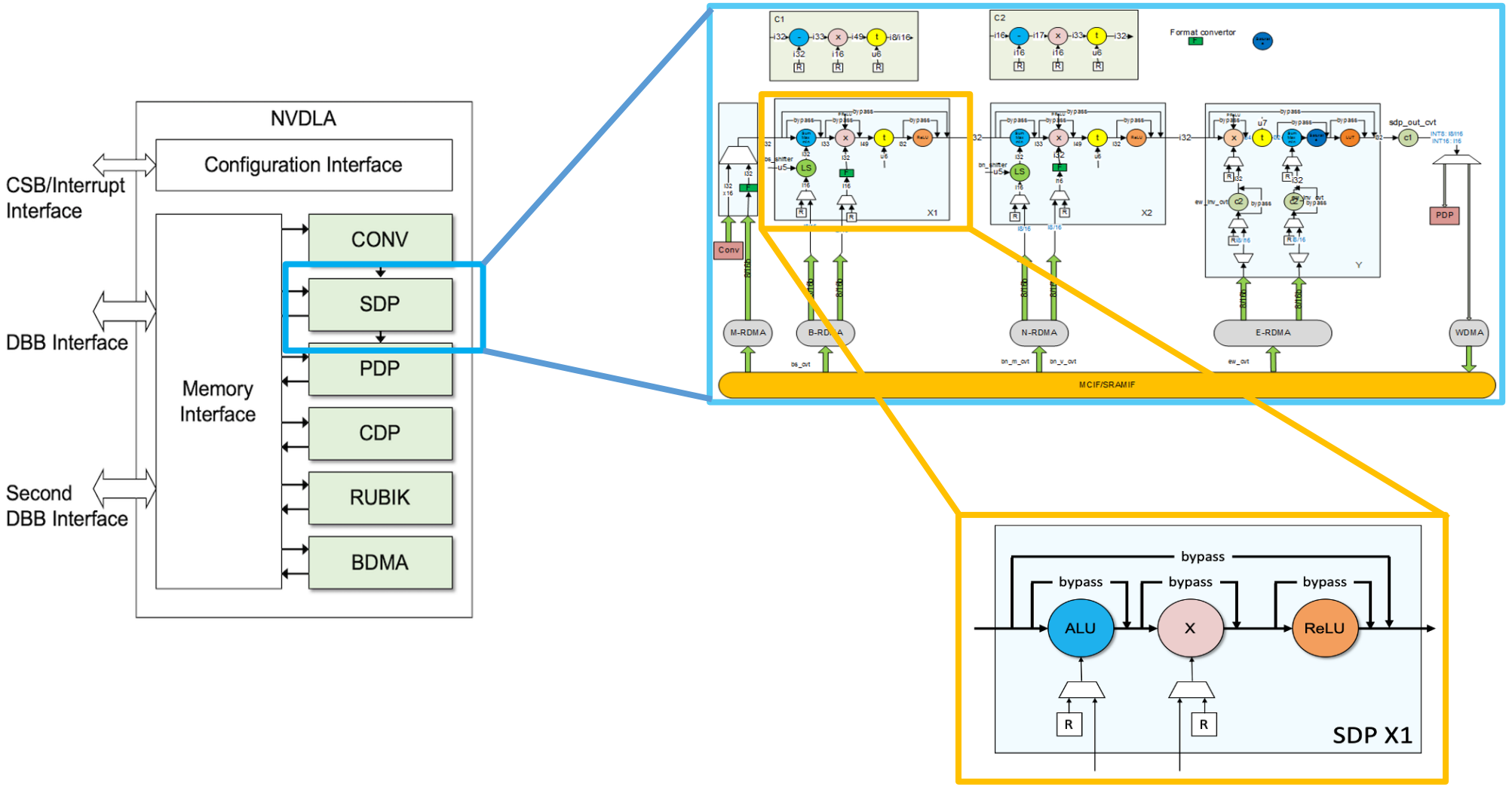
Mapping IR Graph to NVDLA Hardware







Mapping IR Graph to NVDLA Hardware

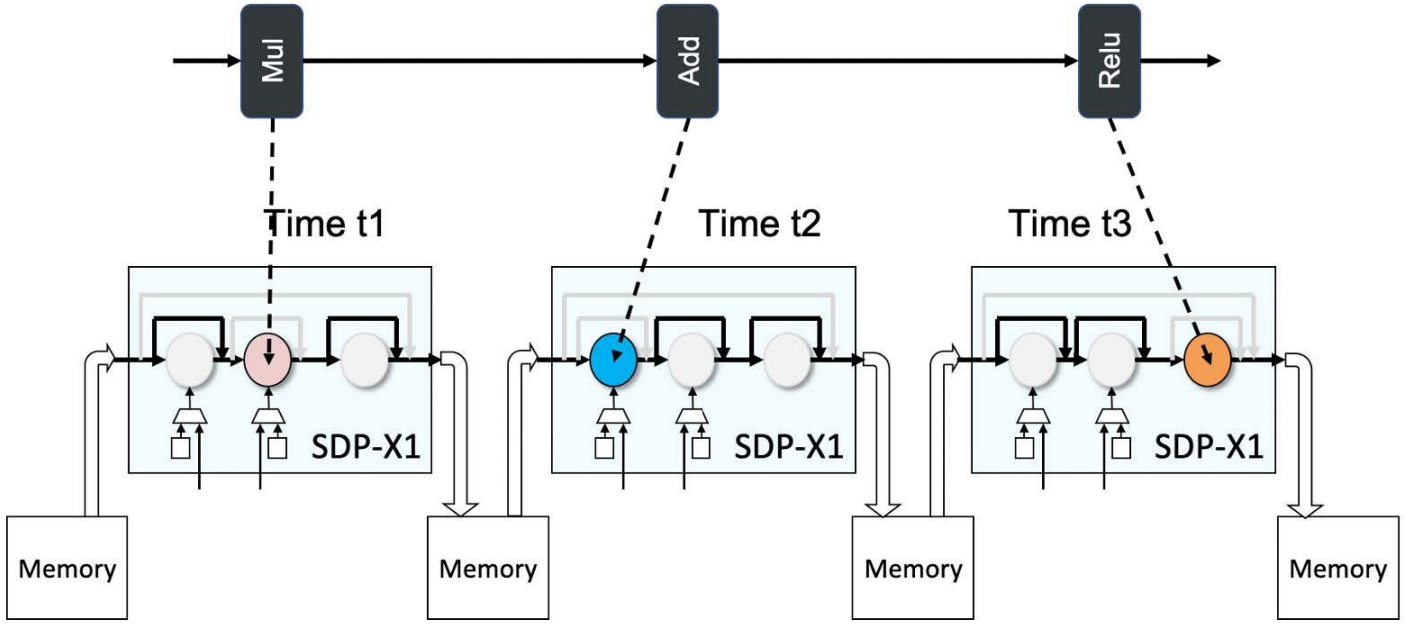


Mapping Options

- If a model contains a series of Mul-Add-ReLU, how to map operators into NVDLA hardware

Option 1:

- Execution time = T
- Memory request = tensor_size *4

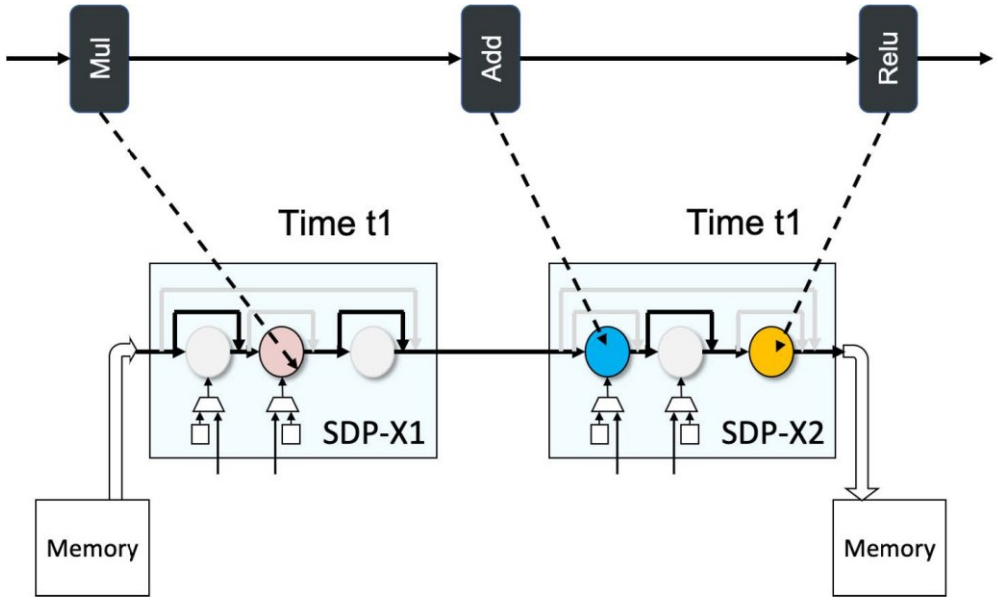


Mapping Options

- If a model contains a series of Mul-Add-ReLU, how to map operators into NVDLA hardware

Option 2:

- Execution time ~= 2T/3
- Memory request = tensor_size *2

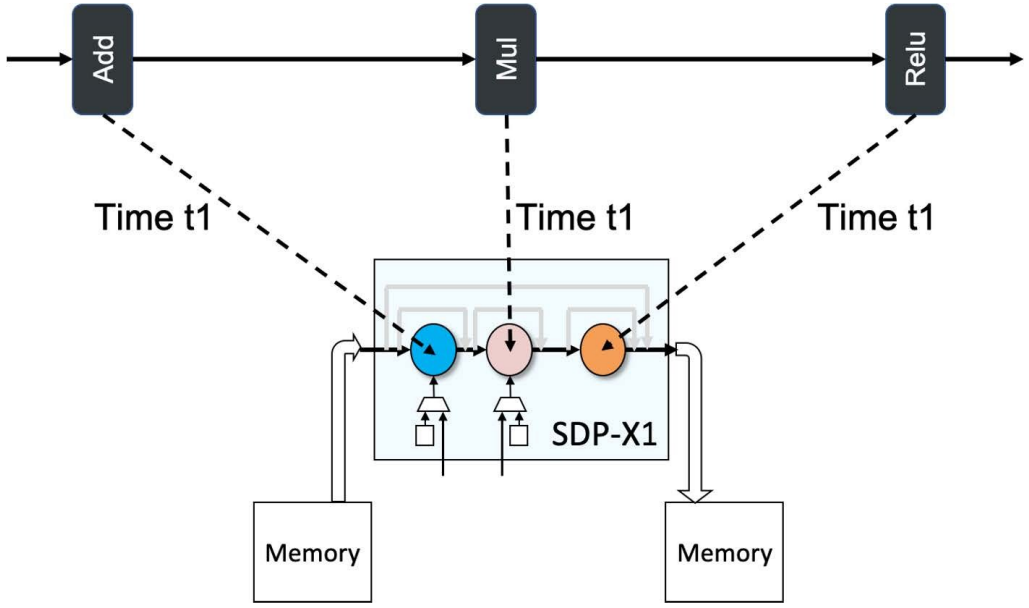


Mapping Options

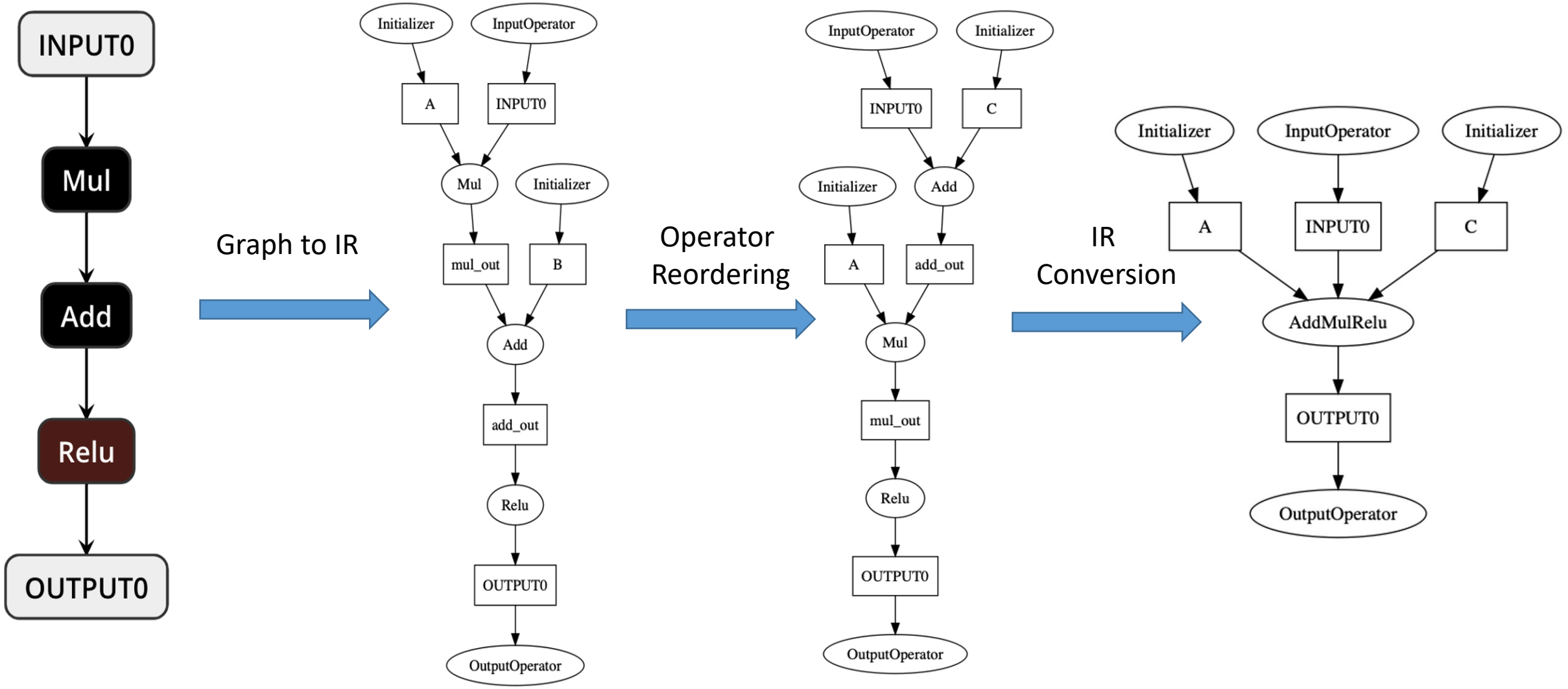
- If a model contains a series of Mul-Add-ReLU, how to map operators into NVDLA hardware

Option 3:

- Execution time ~= T/3
- Memory request = tensor_size *2



Hardware-Dependent Optimization For More Efficient Mapping

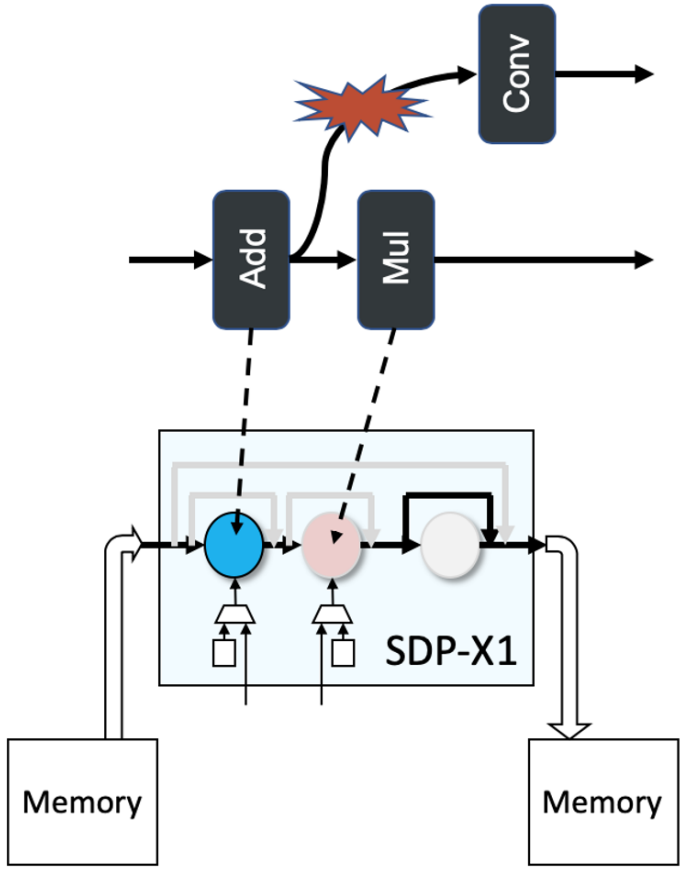


Example of Operator Reordering

- There are two pre-conditions to meet in order to have a valid conversion in this example

$$Y = (X \times \alpha) + \beta \ \rightarrow Y = (x + \gamma) \times \alpha, \gamma = \frac{\beta}{\alpha}$$

1. The output of the **Add** operator can only have one consumer
2. The values of **alpha** and **beta** need to be determined at compile time



Outline

- Overview
- Mapping and Loop Nests
- Mapper and Compilers
- Mapper Organization
- Evaluating Energy
- Evaluating Performance
- Tools for Operation Mapping

Tools For Map Space Exploration

- Specifically for AI Processor
  - Eyeriss mapper
  - MAESTRO
  - R-Stream
  - Timeloop
  - TVM
- Not originally intended to target DNN accelerators but can be adapted
  - Halide
  - Tiramisu

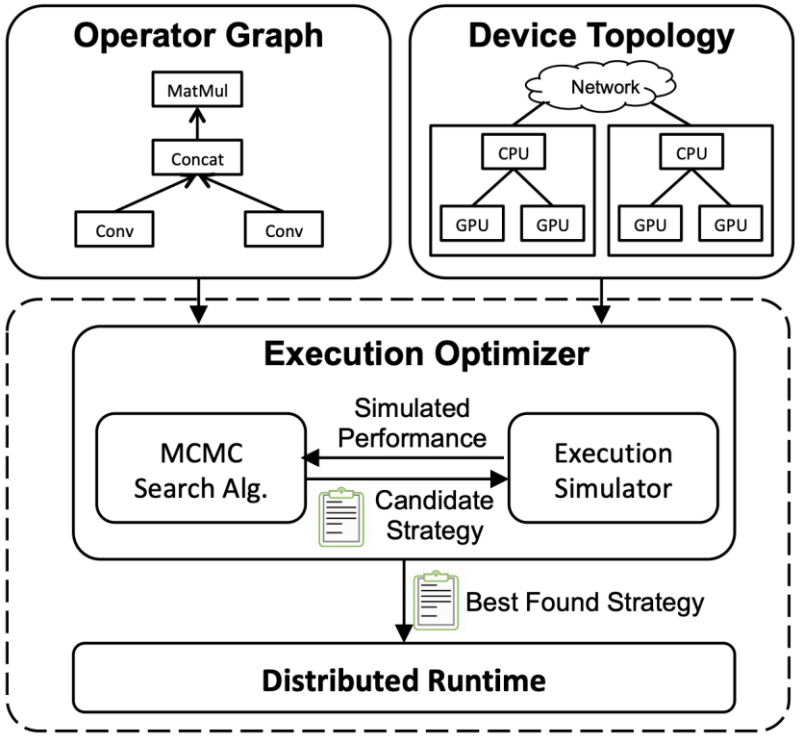
Characterization of Mapping Tools



- **The range of architectures** they target
- **The range of computations** they can map
  - Just DNN layers or more general computation
  - Dictated by the framework they use to describe the map space
- The **heuristic** they use to **search the map space**
- The technique they use to **predict the performance and/or energy** for a given mapping

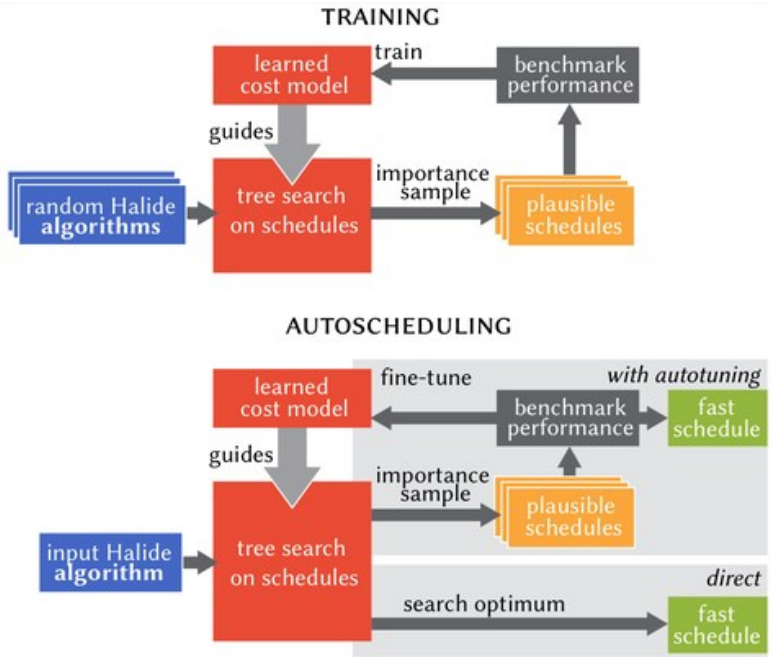
FlexFlow

- The optimizer uses a MCMC search algorithm to **explore the space of possible parallelization strategies** and **iteratively proposes candidate strategies** that are evaluated by an execution simulator. ”



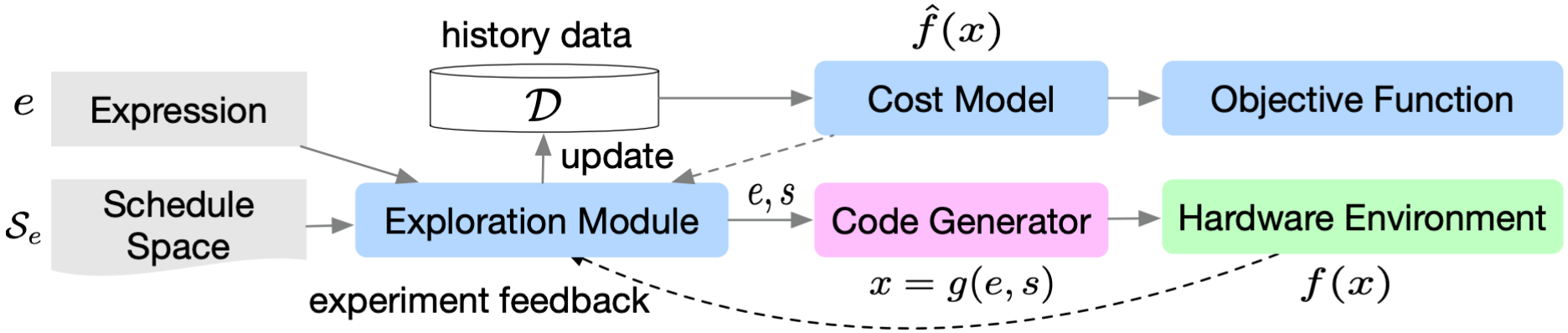
Halide

- “We generate schedules for Halide programs using **tree search over the space of schedules guided by a learned cost model** and optional autotuning. The cost model is trained by benchmarking thousands of randomly-generated Halide programs and schedules. The resulting code significantly outperforms prior work and human experts.”

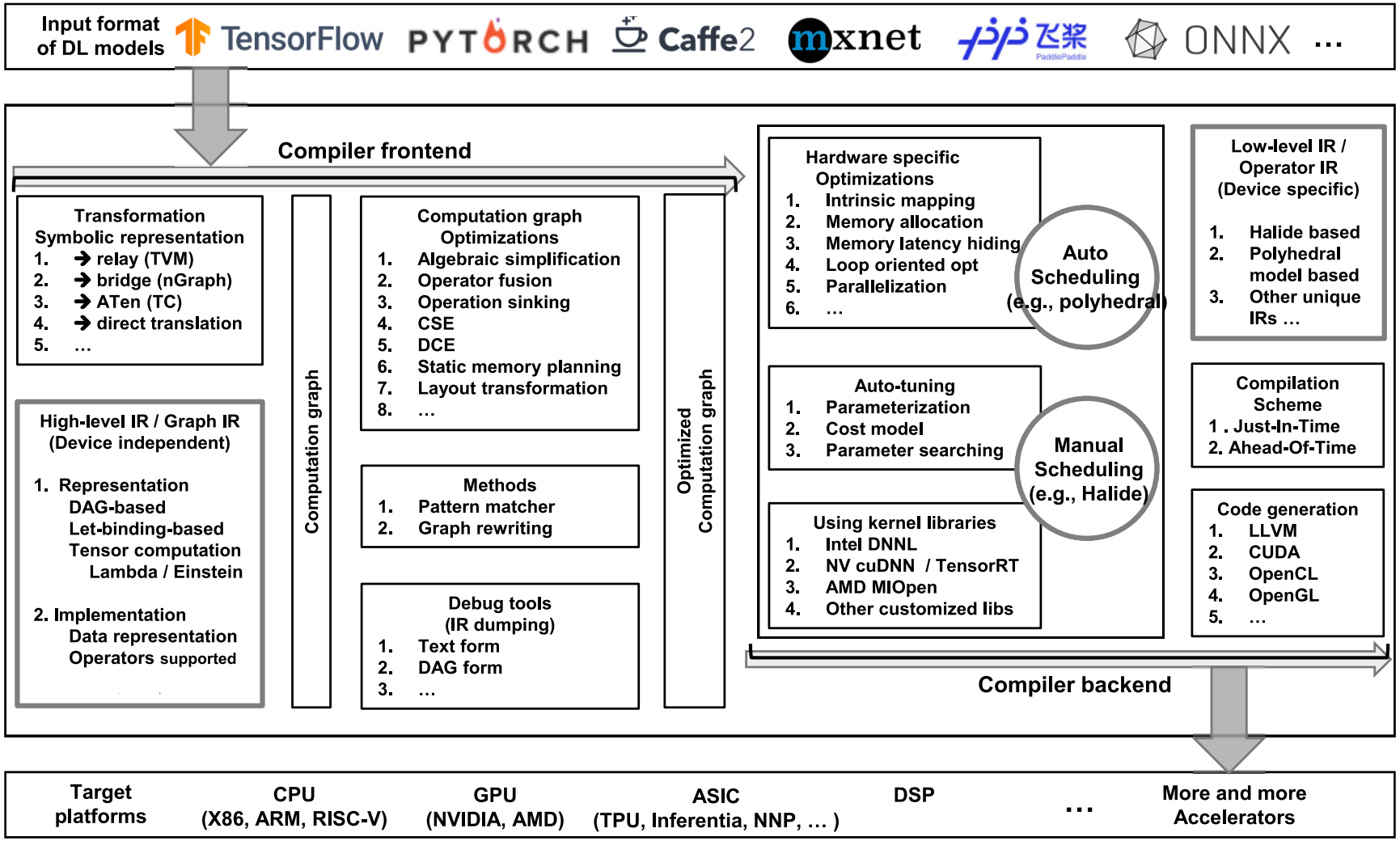


TVM

- "We learn domain-specific **statistical cost models** **to guide the search of tensor operator implementations** over billions of possible program variants. We further accelerate the search using effective model transfer across workloads."

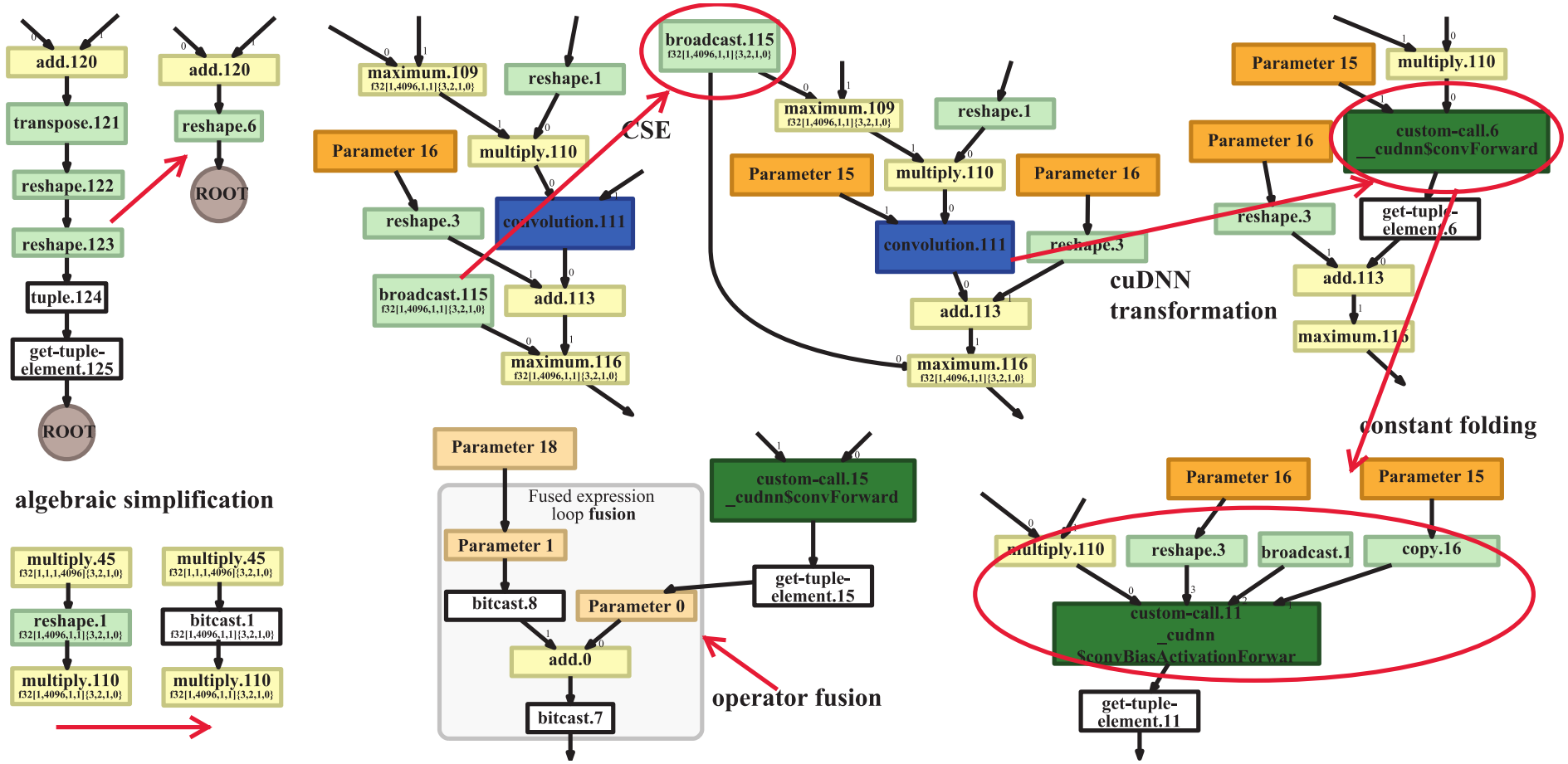


Commonly Adopted Design Architecture for AI Compilers



Computation Graph Optimization

- Example from Tensorflow XLA

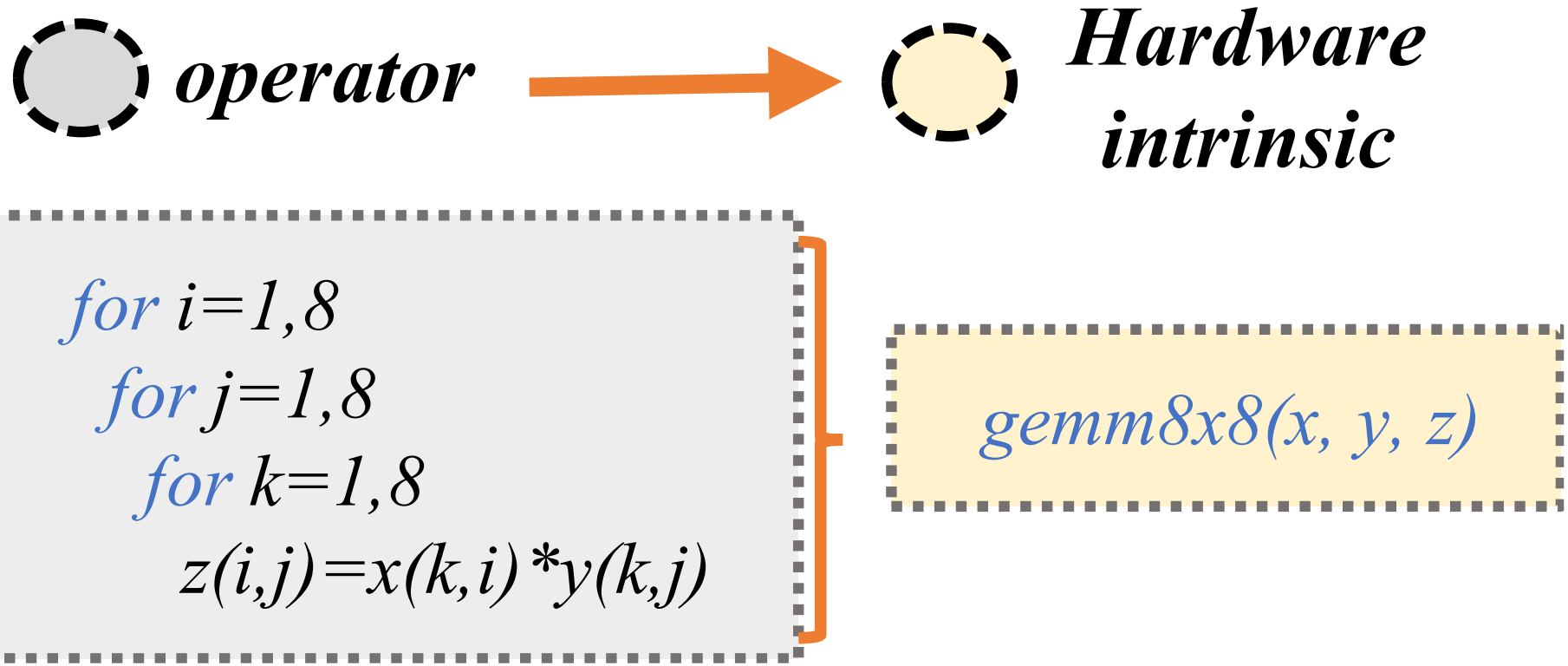


Hardware-specific Optimization

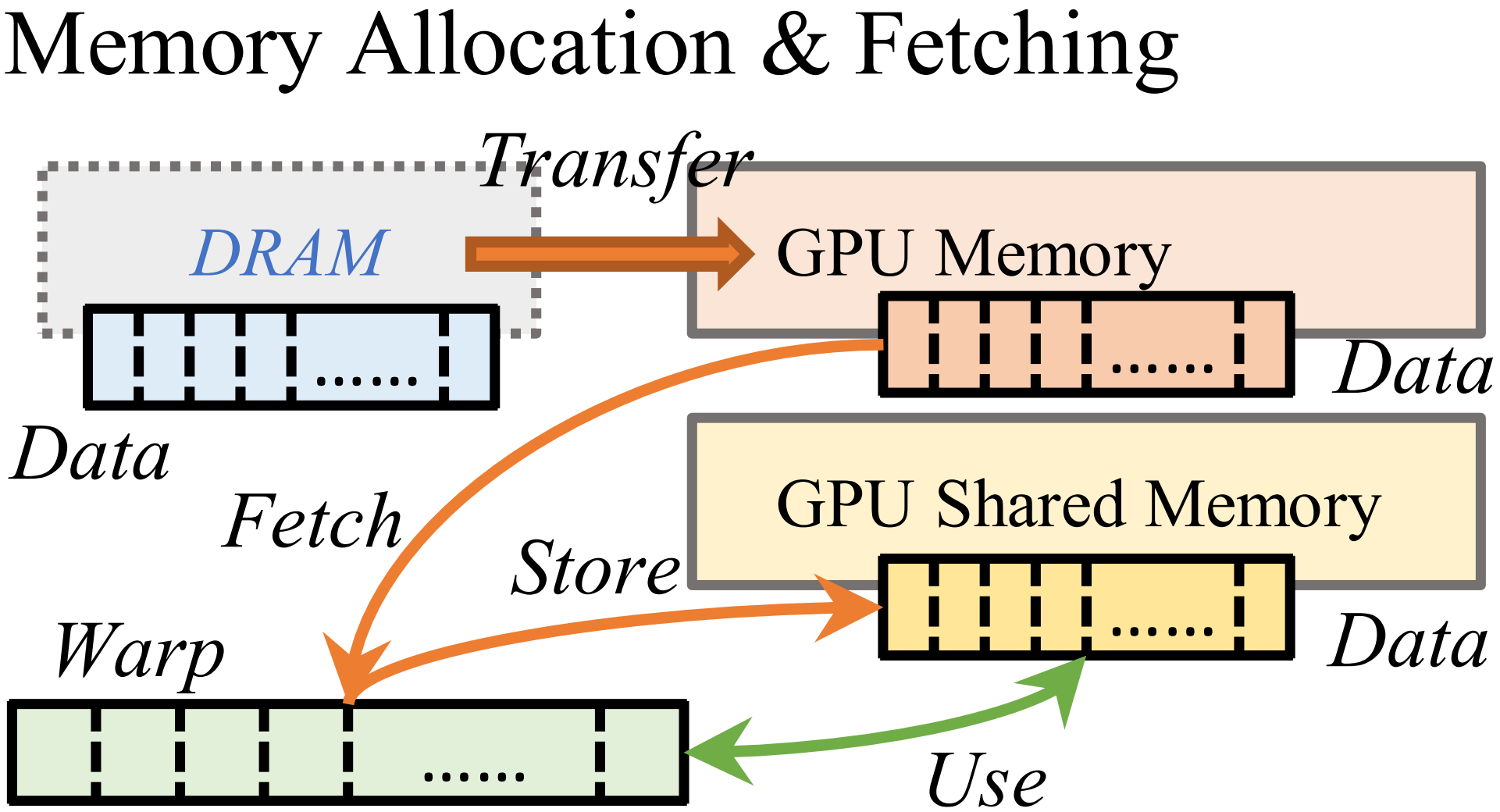


- Hardware intrinsic mapping
- Memory allocation & fetching
- Memory latency hiding
- Loop oriented optimizations
- Parallelization

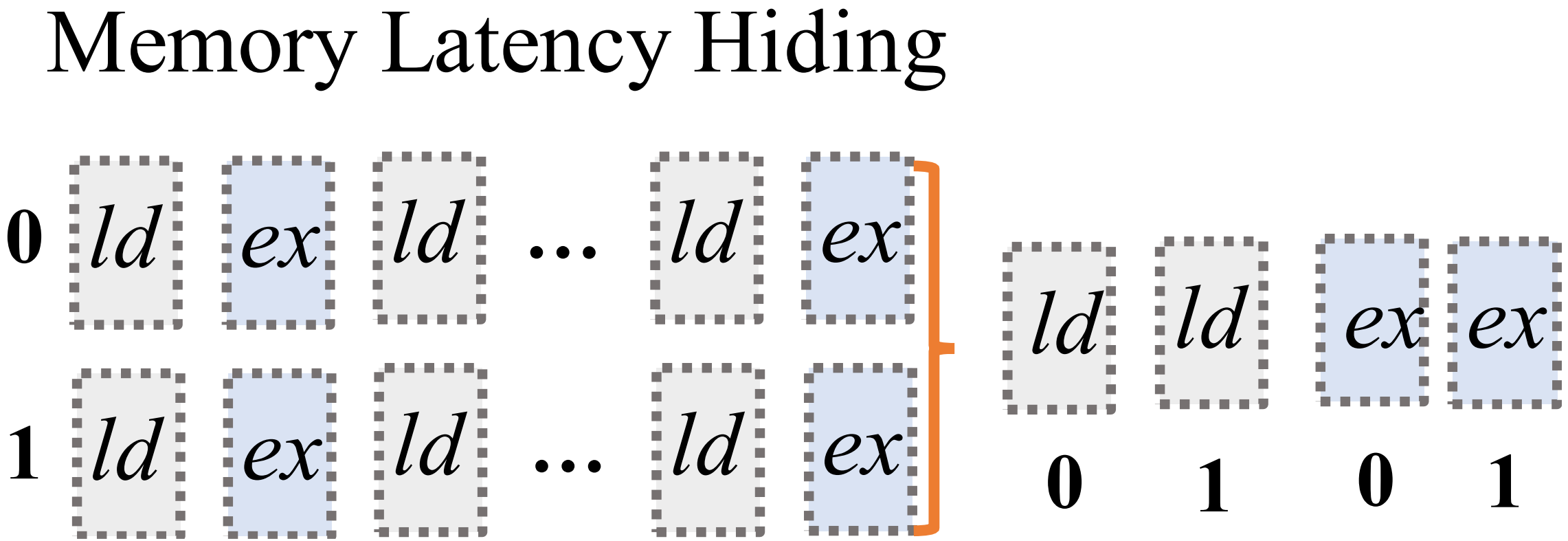
Hardware Intrinsic Mapping



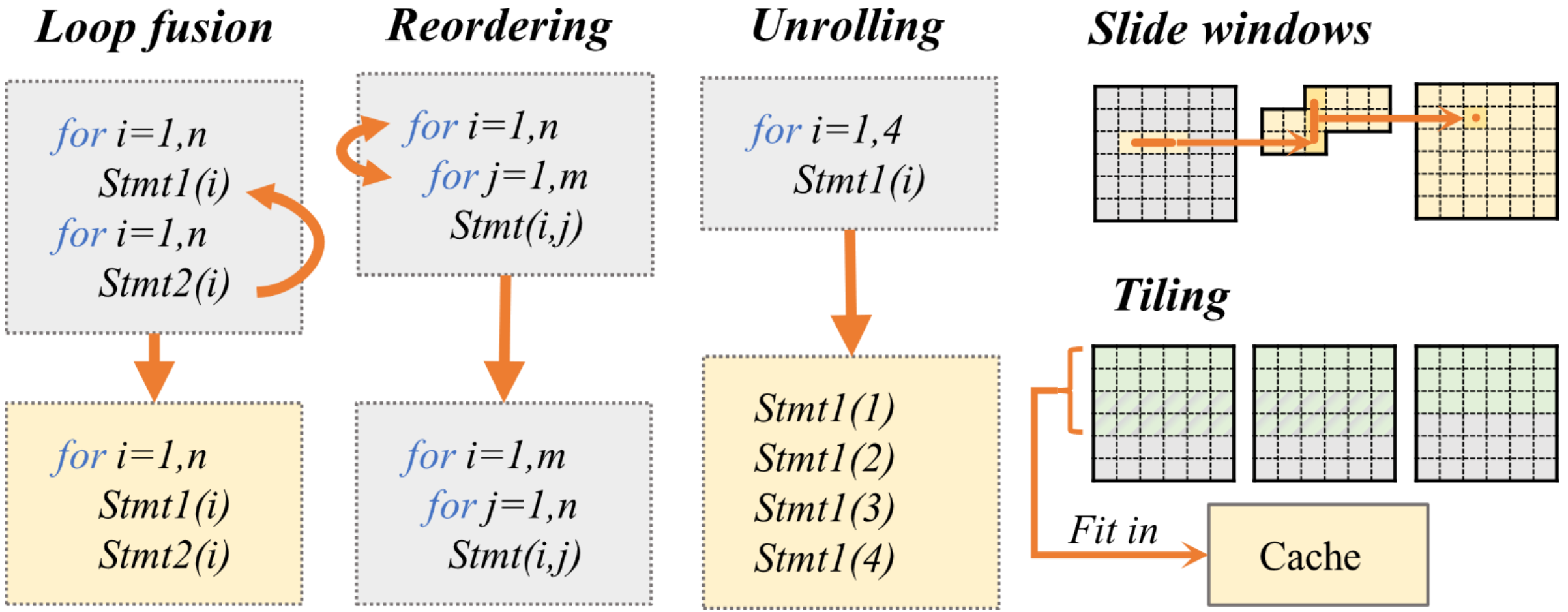
Memory Allocation & Fetching



Memory Latency Hiding

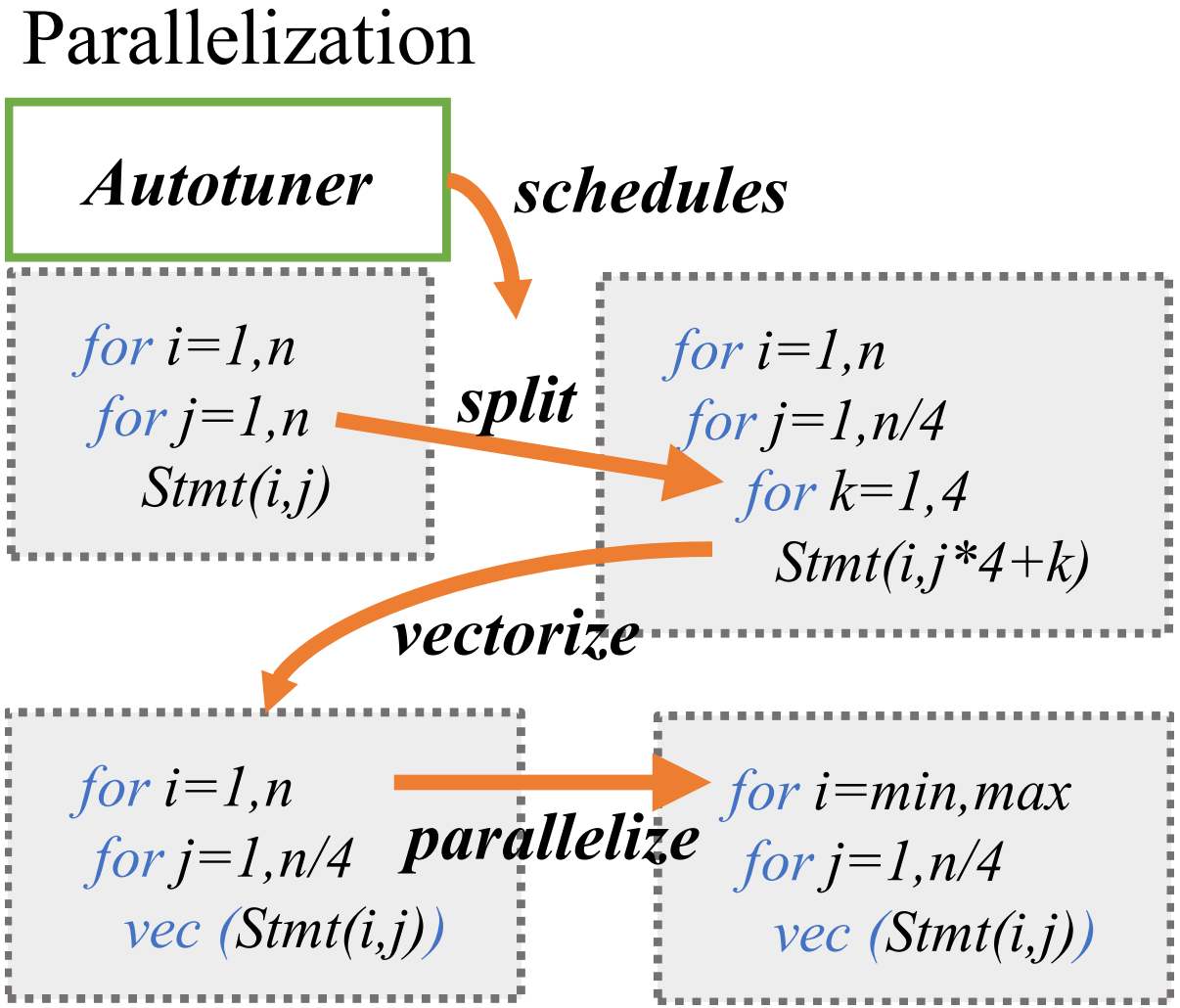


Loop Oriented Optimization Techniques



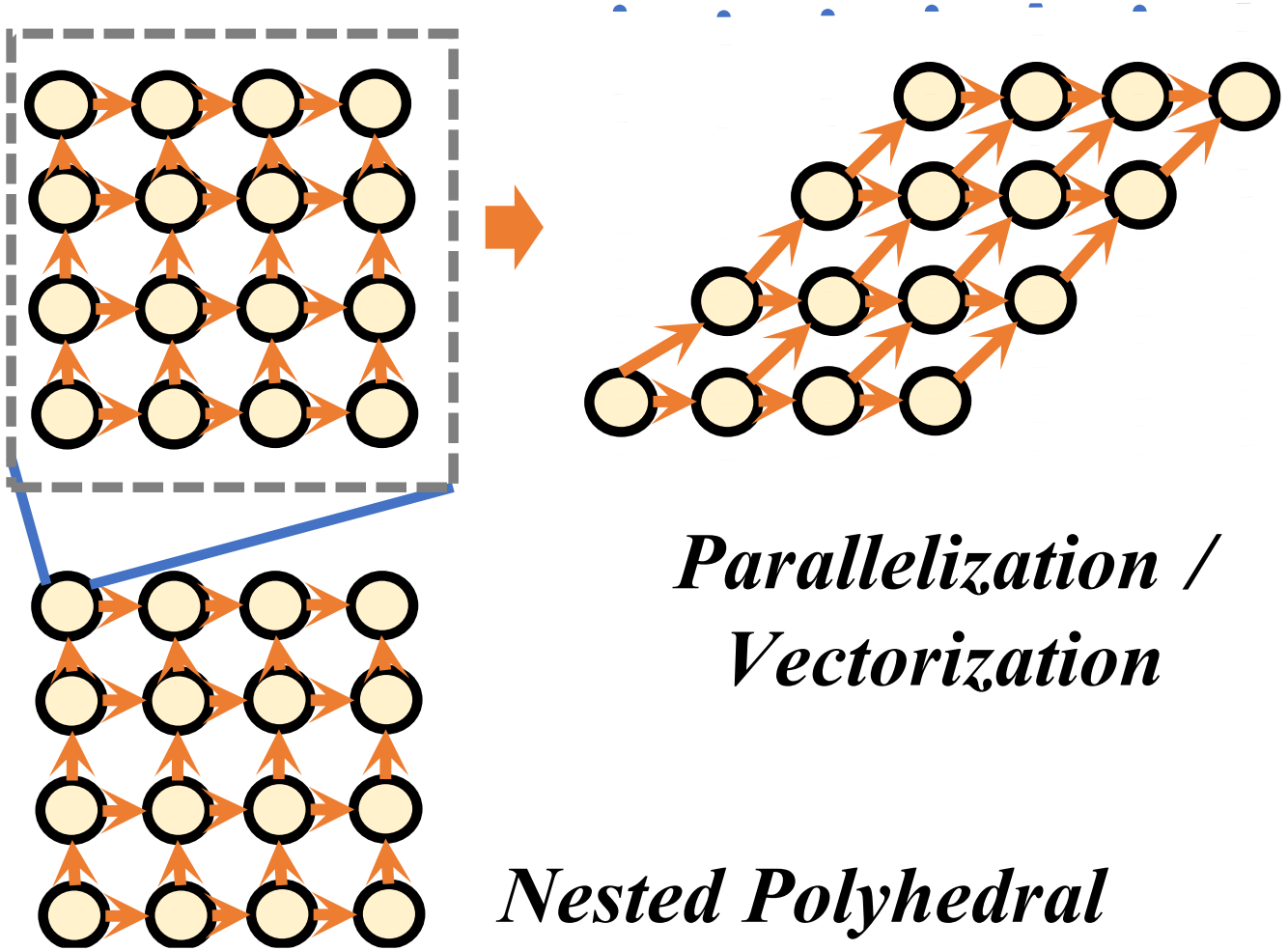
Parallelization

- Halide



Parallelization

- Polyhedral



Comparison of Popular AI Compiler

		TVM	nGraph	TC	Glow	XLA
	Developer	Apache	Intel	Facebook	Facebook	Google
Frontend	Programming	Python/C++ Lambda expression	Python/C++ Tensor expression	Python/C++ Einstein notation	Python/C++ Layer programming	Python/C++ Tensorflow interface
	ONNX support	✓ tvm.relay.frontend .from_onnx (built-in)	✓ Use ngraph-onnx (Python package)	×	✓ ONNXModelLoader (built-in)	✓ Use tensorflow-onnx (Python package)
	Framework support	tvm.relay.frontend .from_* (built-in) tensorflow/tflite/keras pytorch/caffe2 mxnet/coreml/darknet	tensorflow paddlepaddle (Use *-bridge, act as the backend)	(Define and optimize a TC kernel, which is finally called by other frameworks.) pytorch/other DLPack supported frameworks	pytorch/caffe2 tensorflowlite (Use built-in ONNXIFI interface)	Use tensorflow interface
	Training support	× Under developing (Support derivative operators now)	✓ Only on NNP-T processor	✓ (Support auto differentiation)	✓ (Limited support)	✓ Use tensorflow interface
	Quantization support	✓ int8/fp16	✓ int8 (include training)	×	✓ int8	✓ int8/int16 (Use tensorflow interface)

Comparison of Popular AI Compiler

		TVM	nGraph	TC	Glow	XLA
IR	High-/low-level IR	Relay/Halide	nGraph IR/None	TC IR/Polyhedral	Its own high-/low-level IR	HLO (Both high- and low- level)
	Dynamic shape	✓ (*Any*)	✓ (*PartialShape*)	×	×	✓ (*None*)
Optimization	Frontend opt	Hardware independent optimizations (refer to Section 4.3) Hardware specific optimizations (refer to Section 4.4) And hybrid optimizations				
	Backend opt					
	Autotuning	✓ (To select the best schedule parameters)	× (Call optimized kernel libraries, no need)	✓ (To reduce JIT overhead)	× (Additional info is already provided in IR)	✓ (On default convolution and gemm)
	Kernel libraries	✓ mkl/cudnn/cublas	✓ eigen/mkldnn/cudnn/ Others	×	×	✓ eigen/mkl/ cudnn/tensorrt
Backend	Compilation methods	JIT AOT (experimental)	JIT	JIT	JIT AOT (Use built-in executable bundles)	JIT AOT (Generate executable libraries)
	Supported devices	CPU/GPU/ARM FPGA/Customized (Use VTA)	CPU/Intel GPU/NNP GPU/Customized (Use OpenCL support in PlaidML)	Nvidia GPU	CPU/GPU Customized (Official docs)	CPU/GPU/TPU Customized (Official docs)

Performance Comparison

